

HSE C++ Advanced 2025 — полный конспект лекций

Конспект 13 лекций курса «C++ Advanced» (ВШЭ, 2025, лектор — Сергей Скворцов). Написан по полным транскрипциям лекций и кадрам экрана с живым кодированием: должно хватать вместо просмотра записи. В каждой лекции: теория, примеры кода, подчёркнутые лектором тонкости и блок «Кратко» в конце.

Оглавление

1. 01 — Введение. Память
2. 02. Динамическая память. Move-семантика
3. 03 — Продолжаем про move семантику
4. 04 — Типы. Шаблоны
5. 05 — Ошибки. Исключения. Noexcept
6. 06. Паттерны. ODR
7. 07 — Метапрограммирование
8. 08 — Многопоточность
9. 09 — Condition Variable
10. 10 — Advanced threads
11. 11 — Atomic. Lock-free алгоритмы
12. 12 — Корутины
13. 13 — Fibers

Лекция 01 — Введение. Память

Курс «HSE C++ Advanced 2025». Конспект полностью заменяет просмотр лекции. Таймкоды в тексте — ориентир по записи.

1. Организационное (кратко, [00:38–21:47])

- **Формат курса:** ~14 лекций и семинаров, ~10 **маленьких ДЗ** (дедлайн — воскресенье 23:59, каждую неделю) и 3 **больших ДЗ** (~1000+ строк кода, дедлайн 2–3 недели, после дедлайна — сразу 0 баллов). В конце — **коллоквиум** (декабрь, теория: всё, что было на лекциях; полвопросов объявят заранее).
 - **Штраф за опоздание** в маленьких ДЗ — экспоненциальный: +1 час → 99%, +5 часов → 95%, на следующий день → 90%, через сутки → 80%. Тесты открыты, проверка через GitLab CI. Писать локально, не «тестировать через тестирующую систему» — она для оценки, а не для отладки.
 - **Защиты перед ассистентами** (нововведение): у некоторых сдающих выборочно проверяют, понимают ли они свой код. Если не можете объяснить написанное — обнуляется неделя или всё БДЗ. Лектор прямо просит **не использовать ИИ для решения задач**: смысл курса — «набить руку», а на защите придётся объяснять каждый фрагмент.
 - **Списывание:** ловится хорошо; списывание хотя бы одной задачи, попытки расшатать систему или подгонка кода под тесты (вместо написания хорошего кода) караются вплоть до заявления в учебный офис и обнуления всей недели.
 - **Тесты на семинарах** (2–3 вопроса в начале/конце) — бонусные, до 10% сверх основных. Есть бонусные задачки (например, «самый быстрый поисковый индекс») — следить за каналом.
 - **Канал в Telegram — главный источник правды**; чат — только для взаимопомощи и проблем с окружением (иначе сделают read-only). Сайт с дедлайнами: cpp-hse.net; репозиторий: gitlab.com/hse-cpp/cpp-advanced-hse.
 - **Окружение:** Windows не поддерживается вообще; нужен **Linux или macOS** (пара задач — Linux-специфичные, около многопоточности). Все решения проверяются **санитайзерами** — и в системе, и у вас локально. Активно пользоваться **GDB** (в консоли) — без дебаггера на этом курсе тяжело.
 - Курс основан на курсе по C++ ШАД (с тех пор разошлись). Цель — не знание номеров пунктов стандарта, а **правильная интуиция** и много практики; главный справочник — `cppreference`.
-

2. Память — главная абстракция C++ ([21:47–26:09])

Почему лекция и весь курс начинаются с памяти: это **самая главная абстракция C++**, вокруг неё построен весь язык. В отличие от Python/Go/Node.js, где про расположение объектов обычно не думают вообще, в C++ вы обязаны при написании кода понимать, **где живёт объект и как используется память**. Трейдофф: язык, прячущий память, — проще, но медленнее и с меньшим контролем; C++ даёт максимальный контроль — и вместе с ним максимум ответственности.

- По оценкам, **~80% всех уязвимостей и ошибок** связаны с неправильной работой с памятью (команда Chrome оценивала долю таких багов у себя очень высоко). В лучшем случае программа падает, в худшем — злоумышленник получает контроль над процессом (вплоть до исполнения произвольного кода через, например, выход за границу буфера).
- **Санитайзеры** (появились в 2010-х) ловят значительную часть таких ошибок — но только если код тестируют. Правило курса: писать аккуратно и всегда проверять с санитайзерами.
- Раст пытается решить проблему безопасностью на уровне компилятора, но борьба с borrow checker дорого стоит; Carbon (язык от Google) пытается починить C++, сохраняя совместимость, — перспективы неясны. Миллиарды строк C++ уже написаны, и с ними надо уметь работать.

3. Объекты ([26:09–28:15])

Почти всё, с чем работает программа на C++, — **объекты**.

- Объект **создаётся вызовом конструктора** и **разрушается после завершения деструктора**. Промежуток, когда объект жив, — его **время жизни (lifetime)**. Доступ к объекту вне времени жизни — **undefined behavior (UB)**.
- Объект — это кортеж из:
 1. **представления (хранилища)** — выровненного региона памяти размером `sizeof(T)` байт;
 2. **времени жизни** внутри этого хранилища;
 3. **типа**.
- Любая переменная в программе — либо объект, либо ссылка на объект. Другого не бывает.
- **Объект прибит к последовательному участку памяти**: под ним лежит непрерывный массив байтов. Не бывает объектов, «собранных» из кусков в разных местах.
- При **копировании и перемещении** (в т.ч. через `std::move`) возникает **другой объект** — новый, по своему адресу. Объект не может «переехать»: если по-новому адресу живёт «тот же» объект — это на самом деле другой объект.

4. Что такое память: три уровня абстракции ([28:15–34:33])

Лектор подчёркивает: критично разделять, **как память устроена физически, как её видит ОС и как её видит ваша программа**.

- **Физически** это микросхемы DRAM рядом с процессором. Память энергозависима: ток «вытекает» из ячеек, поэтому контроллер регулярно (порядка каждые ~100 мс) пробегается и обновляет их. Внутри одной плашки структура сложная — не массив.
- **ОС** даёт каждому процессу **изолированное адресное пространство** (например, 2^{64} байт виртуальных адресов), внутри которого расположено несколько непересекающихся регионов: код, данные, куча, стек и т.д. Программа не имеет доступа к памяти других программ — детская мечта «выйти за границу массива и поправить соседний процесс» не работает. Разные программы могут использовать одни и те же числовые значения адресов с разными данными. Страничка памяти типично 4096 байт.
- **C++** определяет язык в терминах **абстрактной виртуальной машины** — максимально общо. На уровне языка память — это **набор объектов**: читать можно только объекты (или массивы объектов). Взять и прочитать произвольный байт нельзя — там, скорее всего, нет объекта, и язык это запрещает. То, как память реально устроена в ОС, стандарт C++ не определяет.

5. Байт — это `char` ([34:33–38:22])

- В C++ байт по определению — это `char`, и `sizeof(char) == 1` всегда (размер и измеряется в байтах).
- А вот **сколько бит в байте** — вопрос. Формально стандарт явно минимум не пишет, но логической цепочкой выводится: в байт помещается базовая кодовая единица UTF-8, а у неё минимум 256 значений $\Rightarrow$ байт **не менее 8 бит**. На практике всегда ровно 8 («покажите мне систему, где не 8»), но формально стандарт разрешает и больше (9, 10, 16...).

- **Практическое правило: не пишите в коде число 8 для «бит в байте».** Используйте константу:
 - сиюшнюю `CHAR_BIT` (`<climits>`), или
 - плюсовую — `std::numeric_limits<unsigned char>::digits`.

```
#include <limits>

int main() {
    return std::numeric_limits<unsigned char>::digits; // это и есть "бит в байте"
}
```

6. Ручное управление жизнью объекта: placement new ([38:22–47:15])

Обычно жизнью объектов управляет компилятор (автоматические переменные) или контейнеры. Но ею можно управлять и руками — с помощью **placement new**: размещения объекта **по конкретному адресу**, который вы передаёте сами.

Со слайда «Ручное управление жизнью объекта» (подпись лектора: «*Не повторять дома*»):

```
char* buf = ...; // какой-то сырой буфер байтов
new (buf) std::string("Hello, world!"); // placement new: строим объект в buf
...
reinterpret_cast<std::string*>(buf)->~string(); // ручной вызов деструктора
```

Разбор:

- Обычный `new` сам выбирает место (динамическая память) через аллокатор; запись `new (ptr) T(args...)` — это «**ручной вызов конструктора**», для которого указали, где будет создан объект: `this` внутри конструктора будет равен `buf`.
- Результат placement new — указатель на созданный объект, **численно равный** переданному адресу; дальше объект используется как обычный.
- **Кто создал руками — тот и разрушает руками.** Прямой вызов деструктора: `ptr->~T()`. Тонкость: `std::string` — это `using` для `basic_string`, поэтому в реальном коде деструктор называется `~basic_string()`:

```
char buf[1024];
std::string* s = new (buf) std::string("Hello, world!");
std::cout << *s << std::endl;
reinterpret_cast<std::string*>(buf)->~basic_string(); // именно basic_string!
```

- Если регион памяти разрушить (например, выйдет из области видимости массив на стеке), **не** вызвав деструктор размещённого в нём объекта, — деструктор не вызовется вовсе: утечка ресурсов или UB, в зависимости от того, какими гарантиями обладает класс (в деструкторе может сидеть нетривиальная логика — например, освобождение файлового дескриптора).

Тривиально разрушаемые типы ([43:43–47:00])

Вопрос: для каких объектов деструктор можно **не** вызывать и просто освободить память? Формальный ответ — проверка `std::is_trivially_destructible_v<T>`:

```
#include <type_traits>
#include <string>

struct NonTriviallyDestructible {
    ~NonTriviallyDestructible() {} // даже пустой!
};

struct TriviallyDestructible {
    ~TriviallyDestructible() = default;
};
```

```
static_assert(std::is_trivially_destructible_v<int>);
// static_assert(std::is_trivially_destructible_v<NonTriviallyDestructible>); // ошибка
компиляции
static_assert(std::is_trivially_destructible_v<TriviallyDestructible>);
static_assert(!std::is_trivially_destructible_v<std::string>);
```

Ключевая тонкость, на которой лектор делал акцент: **недостаточно, чтобы деструктор был пустым**. Если пользователь хоть как-то его *определил* (даже с пустым телом) — тип уже не тривиально разрушаемый. Нужно, чтобы деструктора **не было вообще** либо он был `= default`. (И если в классе появятся нетривиально разрушаемые поля, «default»-деструктор сам станет нетривиальным.)

Зачем это надо — оптимизация: разрушая `std::vector<int>`, не нужно линейно пробегать элементы и звать пустые деструкторы — можно просто освободить память. А вот `std::vector<std::string>` обязан позывать деструктор каждому элементу.

`static_assert` vs `assert`: `static_assert` проверяется на этапе компиляции (поэтому редактор/компилятор ругается сразу); обычный `assert` — проверка в рантайме, и её применение здесь «вообще странное, не надо её использовать».

7. Выравнивание ([52:04–61:19])

Представление объекта — **выровненный** регион памяти. **Выравнивание (alignment)** — это требование типа на адреса, по которым может жить объект этого типа: адрес обязан делиться на `alignof(T)`. У `char` выравнивание 1 (живёт по любому адресу), у `std::string` — типично 8.

Почему `placement new` в произвольный буфер `char` — ошибка: массив `char buf[1024]` расположен по адресу, не обязательно кратному 8; создавая в его начале `std::string`, мы получаем объект по невыровненному адресу — **формально UB**. На практике это может работать (в демо на лекции даже санитайзер ASan не поймал; возможно, поймал бы MSan), но «может сломаться когда угодно».

Почему выравнивание вообще нужно:

- Процессор работает **кэш-линиями** (~64 байта, типичные значения; константы лектор давал «для понимания порядка»). Данные, лежащие поперёк двух кэш-линий, обрабатываются вдвое дороже. На старых архитектурах невыровненный доступ ломался совсем.
- Бывают и требования корректности (некоторые атомарные операции и протоколы требуют выровненности).

Выравнивание можно **запросить больше**: `alignas(N)`. Пример с лекции:

```
struct alignas(2048) Overaligned { /* ... */ }; // alignof(Overaligned) == 2048
```

(и вообще `alignof` может быть большим — в демо делали `struct alignas(1024 * 1024 * 1024)`).

Как правильно жить с `placement new` — выравнивать сам буфер под нужный тип:

```
alignas(alignof(std::string)) char buf[1024];
static_assert(alignof(std::string) == 8);
static_assert(1 != 8); // потому выровненность char-буфера не спасает сама по себе

std::string* s = new (buf) std::string("Hello, world!");
```

И общий приём — «сторож»-обёртка, выровненная и размером ровно как `T` (удобно для контейнеров/`std::optional`-подобных вещей):

```
template <class T>
struct AlignedStorage {
    alignas(T) char data[sizeof(T)]; // выравнивание и размер — как у T
};
// внутри такого сторожа уже можно placement-new'ить объекты типа T
```

8. Переиспользование памяти и UB вокруг него ([61:19–66:54])

- По одному и тому же адресу (в одном и том же хранилище) могут последовательно жить **разные объекты**: разрушили старый — создали новый (реинициализация). Так работают `std::vector`, `std::deque`, `std::optional` под капотом.
- Но это ходьба «вплотную к UB»: из-за проблем с формальным описанием таких сценариев **до C++20 корректно реализовать `std::vector` было нельзя** — например, у константных полей адрес по стандарту не может меняться, а вектор переиспользует память, создавая новый объект по тому же адресу. В C++20 починили, но писать такой код по-прежнему нужно очень аккуратно и только если вы пишете низкоуровневый контейнер (в домашке будет дек — там придётся).
- Следствие: **численного равенства указателей недостаточно**, чтобы считать, что указывают они на один и тот же объект. Указатель — это указатель на конкретный объект (хранилище + тип + время жизни); если по адресу пересоздали объект — это формально уже другая история.

9. Виды памяти (storage duration) ([66:54–70:07])

Четыре вида (слайд «Виды памяти»):

1. **Automatic storage duration** — жаргон «на стеке»: обычные локальные переменные (`std::string s;`).
2. **Static storage duration** — жаргон «глобальный»: живут всю программу.
3. **Dynamic storage duration** — жаргон «в куче»: классический `new`.
4. **Thread-local storage duration** (C++11) — лектор пообещал рассказать позже, «когда дойдём до многопоточности».

Про динамическую память — слайд «Динамическая память»:

- Выделяется и освобождается **руками**; объекты живут столько, сколько напишет программист.
- Уровни снизу вверх:
 - **Сырые операторы**: `operator new` / `malloc` — возвращают **нетипизированный массив байтов**, объектов там ещё нет (`void* ptr = operator new(1024);`). Отличие: `malloc` при ошибке возвращает `nullptr`, `operator new` кидает исключение.
 - **Типизированный `new expression`** (`new int`, `new char[32]`, `new std::string{"Hello, world!"}`) — выделяет память **и вызывает конструкторы** (для массива из 1024 строк вызовется 1024 конструктора). Здесь уже пишется тип — потому что создаётся объект этого типа.
 - **Обёртки**: `std::vector`, `std::string` и другие контейнеры — за вас выделяют и освобождают память.
- **Практическое правило**: никто практически никогда не пишет динамическую память руками — используйте обёртки. Динамическая память уместна **только внутри реализаций контейнеров**; пользователю выдавайте удобные высокоуровневые обёртки. (Исключения бывают, но редко.)

```
void* ptr = operator new(1024);           // сырые 1024 байта, объектов нет
new std::string{"Hello, world!"};        // new expression: память + конструктор
std::string ss;
ss.~basic_string();                      // так выглядит ручной вызов деструктора
```

Кратко

1. **Память — главная абстракция C++**: в отличие от других языков, вы обязаны понимать, где живёт объект. ~80% уязвимостей и ошибок — ошибки работы с памятью; проверяйте код санитайзерами.
2. **Объект = хранилище (выровненный регион `sizeof(T)` байт) + время жизни + тип**. Доступ вне времени жизни — UB. Копия и перемещённая версия — **другой объект** по другому адресу; объект не «переезжает».
3. **Три уровня памяти не путать**: физическая DRAM, изолированное виртуальное адресное пространство процесса (набор регионов: код, данные, куча...), и память на уровне языка — набор объектов; произвольный байт читать нельзя.
4. **Байт — это `char`, `sizeof(char) == 1`**; бит в байте — минимум 8, на практике всегда 8, но в коде пишете `CHAR_BIT` / `std::numeric_limits<unsigned char>::digits`, а не 8.
5. **Placement new** (`new (buf) T(...)`) — ручной вызов конструктора по заданному адресу; руками создали — руками разрушайте: `ptr->~T()` (для `std::string` — `~basic_string()`).

6. Деструктор можно не звать только для `std::is_trivially_destructible_v<T>` типов: пустой, но определённый пользователем деструктор убивает тривиальность; нужен отсутствие деструктора или `= default`. На этом строится оптимизация `vector<int>`.
7. **Placement new требует выравнивания**: адрес должен делиться на `alignof(T)`, иначе UB (на практике — может работать, пока не сломается). Выравнивайте буфер: `alignas(alignof(T)) char buf[N];` или сторож `struct AlignedStorage { alignas(T) char data[sizeof(T)]; };`. Процессор работает кэш-линиями (~64 байта) — невыровненные данные дороже.
8. **Реинициализация объекта по тому же адресу** — приём контейнеров, но тонкий (до C++20 формально ломал `std::vector`); равенство адресов $\neq$ один и тот же объект.
9. **Четыре storage duration**: automatic («стек»), static («глобальный»), dynamic («куча»), thread-local (позже). Динамическая память: `operator new / malloc` — сырые байты; `new T(...)` — память + конструктор; в обычном коде — только обёртки (`std::vector`, `std::string`).

Лекция 02. Динамическая память. Move-семантика

Виды памяти

В C++ есть четыре больших класса памяти (storage duration):

- **Automatic** — жаргонно «на стеке»;
- **Static** — жаргонно «глобальный»;
- **Dynamic** — жаргонно «в куче»;
- **Thread-local** (C++11) — обсудим после темы многопоточности, пока считаем, что его нет.

Самый сложный из них — динамический; автоматический и статический простые и понятные.

Динамическая память

Динамическая память — память, которая выделяется и освобождается **руками** (или под капотом, но через абстракции, которые прячут выделение/освобождение). Объекты живут столько, сколько напишет программист: вы сами контролируете время жизни. Это удобно, но требует не забывать освобождать память.

Три уровня работы с ней:

1. **Самый low-level** — `operator new / malloc`: возвращают указатель на сырые байты, объектов там ещё нет.
2. **Типизированный new-expression** — вы явно пишете тип (`new int{42}`, `new int[100]`): под капотом одновременно выделяется память и создаётся объект.
3. **Обёртки** — `std::vector`, `std::string` и т.п. То, что по-хорошему надо использовать почти всегда.

```
int* kek = new int{42};
delete kek;

int* array = new int[100];
delete[] array;

std::unique_ptr<int> ptr(new int{19});

// Calls operator new(sizeof(int) * 100);
std::vector<int> v(100);

std::unordered_map<int, int> x;
// calls new std::pair<const int, int>(1, 2);
x[1] = 2;
```

Правило new/delete

Если есть слово `new` — должно быть слово `delete` (кроме сверхредких историй). За каждым `new` идёт ровно один корректный `delete`.

- Два раза `delete` — undefined behavior.
- Ноль раз — утечка памяти. Утечка безопасна (не UB), но допускать её не нужно практически никогда. Бывают редкие сценарии, когда с утечкой программа проще или эффективнее — но для этого надо очень хорошо понимать, что делаешь.

Единственный «легальный» случай написать `new` без `delete` — передать указатель в `std::unique_ptr`, который сам позовёт `delete` в деструкторе. Но писать `std::unique_ptr<int> ptr(new int{19})` некрасиво: глаз цепляется за `new`, и надо тратить силы на проверку корректности. Поэтому придумали `std::make_unique` — обёртка, которая под капотом делает `new`, всегда возвращает `unique_ptr` и не позволяет «потерять» указатель.

delete и массивы

Правило выбора `delete` очень простое:

- выделяли **один объект** (`new int{42}`) — освобождаем обычным `delete`;
- выделяли **массив** (`new int[100]`, квадратные скобки) — освобождаем `delete[]`.

`delete arr;` после `new int[100]` — undefined behavior, причём физически: попытка освободить регион памяти, который вы не выделяли.

Почему нужен именно `delete[]`: в отличие от одиночного `delete`, компилятору нужно понять, **сколько деструкторов вызвать**. При `new` сначала выделяются сырые байты, потом вызывается конструктор; при удалении — наоборот: сначала деструкторы для всех элементов (важно для объектов с нетривиальным деструктором, например `unique_ptr`), и только потом освобождение памяти. Поэтому такие массивы «прикапывают» служебную информацию: в начало выделенного блока кладётся число элементов, а пользователю возвращается указатель не на начало блока, а на первый элемент. Если позвать обычный `delete` на этот указатель, аллокатор попытается освободить не тот регион, который выделял, — всё сломается.

Отсюда же ошибка `std::unique_ptr<int> ptr(new int[100]);` — `unique_ptr` позовёт обычный `delete`, а не `delete[]`. Нужно писать явно:

```
// [size] [[]]
std::unique_ptr<int[], std::default_delete<int[]>> ptr(new int[100]);
```

Пустые квадратные скобки в первом шаблонном параметре включают вызов `delete[]`.

Deleter — второй параметр unique_ptr

У `std::unique_ptr` есть второй шаблонный параметр — **deleter**, определяющий, как именно освобождать объект (по умолчанию `std::default_delete<T>`). Реализация `std::default_delete` — это ~20 строк «мишуры» для шаблонного контекста, а по смыслу одна строчка: `operator()` принимает указатель и зовёт `delete ptr`. Вы можете написать свою структуру с таким же `operator()`, который делает что-то другое, и просунуть её вторым параметром — логика освобождения кастомизируется.

Уникальность владения

`std::unique_ptr` гарантирует, что при выходе из области видимости (закрывающая скобка, `return`, исключение) выделенная память освободится. Копировать его нельзя: получилось бы два `unique_ptr` на одну память, оба вызовут `delete` — двойное освобождение. Поэтому копирующий конструктор у него удалён:

```
std::unique_ptr<int> Kek() {
    std::unique_ptr<int> ptr(new int{42});
    std::unique_ptr<int> ptr2 = ptr; // ERROR: call to implicitly-deleted copy constructor
    return ptr; // а вот так — можно (см. ниже про move)
}
```

Память из `unique_ptr` можно достать наружу методом `release()`.

Ручное управление памятью: почему это боль и причём тут «закрывающая скобка»

Как жили в С: память выделяли руками, ошибки выделения проверяли на каждом шаге (исключений не было), и нужно было не забыть позвать `free`. Идиоматичный С (например, код ядра Linux) полон цепочек `goto cleanup`: в конце функции написана логика очистки всего выделенного в обратном порядке, и в зависимости от места ошибки вы прыгаете на нужную метку. Это очень больно и очень легко ошибиться.

В С++ первая проблема (проверка ошибок) решена исключениями, а вторая (не забыть освободить) — нет, если использовать сырой `new` / `delete`:

```
int *x = new int[100];
if (!DoSomeWork(a, b, c, d)) {
    return; // WRONG! (утечка)
}
delete[] x;

void f() {
    int* p = new int(7);
    g();    // may throw
    delete p; // okay if no exception
} // memory leak if g() throws
```

Здесь `g()` может выбросить исключение: обработки нет, исключение полетит наружу, строчка `delete p` не выполнится — утечка. Причём не обязательно делать `early return`: достаточно вызвать любую функцию, которая может бросить.

Практическое правило: как только вы выделили сырую динамическую память, вы в большой опасности. Если прямо сейчас не обернуть её в безопасную обёртку — код, скорее всего, будет утекать.

Деструктор самого указателя ничего не делает (его, по сути, нет) — механика работает только через **нетривиальный деструктор** обёртки. Главная фишка С++, по словам лектора, — «закрывающая скобка»: при выходе из блока (скобка, `return`, исключение) вызываются деструкторы всех объектов блока.

Динамическую память выделяем, но сразу оборачиваем во владеющую обёртку, которая сама живёт в **автоматической** памяти (например, `unique_ptr` на стеке): его деструктор будет вызван автоматически и транзитивно позовёт `delete` на внутреннем объекте. Так же освобождаются любые ресурсы: файлы, мьютексы и т.д.

Общий принцип

Никогда (почти) не используйте `malloc`, `operator new`, `new-expression` напрямую. Используйте `unique_ptr` / `make_unique` и контейнеры.

Каждый из этих механизмов когда-то нужен (low-level код, написание контейнера вроде дека из домашки), но только когда вы очень хорошо понимаете, что делаете. Даже в `std::vector` вы не найдёте `operator new` или `new-expression`: они спрятаны за абстракцией **allocator** — неявным вторым шаблонным параметром вектора (по аналогии с `deleter` у `unique_ptr`). `std::allocator` — это ~100 строк кода ради вызова `new` / `delete`, зато позволяет переопределять логику выделения (например, выделять память на GPU, подставив другой аллокатор).

Остальные виды памяти: автоматическая и статическая

Автоматическая называется автоматической, потому что память выделяется и освобождается автоматически за вас: написали `int x;` — она создалась на этой строчке и умерла на закрывающей скобке, `new` / `delete` писать не надо. Почти всегда это стек — небольшой регион памяти (2–8 МБ, не сильно больше; про стек даже в стандарте ничего не написано). Про стек знают и работают с ним все трое: ОС (готовит и настраивает), процессор (сохраняет при вызовах функций), компилятор (вычисляет, где лежат переменные).

Миф: «память на стеке быстрее». Это миф — память та же самая, абсолютно одинаковая. Стек может быть чуть быстрее лишь в том, что *выделить* память там тривиально (она уже выделена), тогда как `new` — это сложная задача: организовать структуру данных, отвечающую на запросы «выдели N байт / освободи вот тот блок» с минимальной фрагментацией и малым overhead. Под этим закопаны сотни человеко-лет — это отдельный большой мир, искусство писать аллокаторы (менеджеры динамической памяти; термин «аллокатор» столкнулся с `std::allocator` — это разные вещи).

Практика: большие объекты (> килобайта) — в динамическую память, на стеке — что-то небольшое и указатели.

Динамическая память нужна в двух случаях: 1. большой объект; 2. нетривиальное управление временем жизни, не совпадающее со структурой кода функции.

Статическая — глобальные переменные и переменные, живущие всю программу. Важное отличие: они **инициализируются по умолчанию**, даже если вы ничего не написали (в отличие от автоматических и динамических). Они занимают место в исполняемом файле: большой глобальный массив на гигабайты раздует бинарник. Использовать статическую память почти всегда можно без — глобальные переменные в целом зло (исключения вроде реализации аллокатора существуют). Понятное применение — константы, которые никогда не меняются; меняющиеся глобальные переменные — почти всегда плохая идея. Для констант вопрос, какие слова написать, отдельный большой (про `static`, `constexpr`, `inline` будет своя лекция): неформально `static` означает «видна только в этом .cpp».

Категория значения. lvalue и rvalue

Теперь теория перед move-семантикой. У каждого выражения в C++ есть **тип** и **категория значения** (value category). Три категории:

1. **lvalue** (left hand side value);
2. **xvalue** (eXpiring value);
3. **prvalue** (pure rvalue).

xvalue и prvalue похожи, для них определена общая категория **rvalue**. Поэтому встречаются два разбиения: `lvalue / rvalue` и более подробное `lvalue / xvalue / prvalue` (prvalue и xvalue немного отличаются, их стоит разделять).

Интуиция (неформальное определение): rvalue — то, что может стоять только справа от присваивания; lvalue — то, что может и слева. Более правильная интуиция: **у lvalue есть адрес, позиция в памяти**.

```
int i = 1;           // i — lvalue, 1 — rvalue (литеральная константа)
int* y = &i;         // y lvalue можно брать адрес — у него есть локация
1 = i;               // ERROR: 1 — rvalue, не может быть слева
y = &42;             // ERROR: 42 — rvalue, нет локации
const char (*ptr)[5] = &"abcd"; // строковый литерал — lvalue (!)
char arr[20];
arr[10] = 'z';       // arr[10] возвращает lvalue reference
int a, b;
(a, b) = 5;          // то же, что b = 5
```

Строковый литерал — lvalue: у него есть адрес (строки живут отдельно в бинарнике), а числа существуют прямо по месту в коде. При этом присвоить строковому литералу всё равно нельзя — определение «lvalue можно писать слева» лишь интуиция.

Из функций: возвращаемое по значению значение — rvalue, временный объект, адрес взять нельзя; а функция, возвращающая `int&`, даёт lvalue:

```
int GetValue() { return 42; }
int& GetGlobal() { static int j; return j; }

GetValue(); // не lvalue: временный объект из функции
GetGlobal(); // ok, int& — lvalue reference, есть локация
```

Ссылки

- **lvalue reference** — классическая ссылка `T&`. Выражение-ссылка само является lvalue (взятие адреса от ссылки даёт адрес исходного объекта).
- Нельзя инициализировать неконстантную lvalue-ссылку константой (`int& r = 10;`) — под ссылке нужен объект с адресом в памяти.
- **const-ссылка** — можно! `void FooConst(const int& x) { ... }` и вызов `FooConst(10);` компилируется. Это специальное правило в стандарте: временный объект создаётся, и это удобно (цепочки функций: `g(f())`, где `f` возвращает `T`, а `g` принимает `const T&`).

Temporary lifetime extension

`const int& ref = 10;` компилируется и превращается под капотом в:

```
int __internal_unique_name = 10;
const int& ref = __internal_unique_name;
```

Это **продление времени жизни временного объекта**: временный объект живёт до конца блока (закрывающей скобки). Поэтому такой код корректен и без UB:

```
const T& x = f(); // временный результат f() живёт до конца блока
```

Правило: так писать не надо, если можно обойтись (`const T x = f();` даст ровно ту же эффективность). Это неявное поведение, которое путает читающего код.

Проблема до C++11

Вернёмся к «как вернуть `unique_ptr` из функции / передать владение». До C++11 у классов с ресурсами были только копирующие конструктор и оператор присваивания. Пример класса-владельца массива:

```
class Holder {
public:
    Holder(int size) {
        data_ = new int[size];
        size_ = size;
    }
    ~Holder() {
        delete[] data_;
    }
private:
    int* data_;
    size_t size_;
};
```

Копирующие методы:

```
Holder(const Holder& other) {
    data_ = new int[other.size_];           // (1)
    std::copy(other.data_, other.data_ + other.size_, data_); // (2)
    size_ = other.size_;
}

Holder& operator=(const Holder& other) {
    if (this == &other) return *this;      // (1)
    delete[] data_;                         // (2)
    data_ = new int[other.size_];
    std::copy(other.data_, other.data_ + other.size_, data_);
    size_ = other.size_;
    return *this;                          // (3)
}
```

Скопировать `unique_ptr` нельзя (два владельца → двойной `delete`). Поэтому для передачи владения люди писали функцию `swap` (меняет местами внутренности двух объектов) для каждого контейнера, и общий паттерн был: создать пустой объект, свопнуть. Проблема фундаментальная: **swap — чисто соглашение, компилятор про него не знает**. При возврате из функции или присваивании он всё равно вызовет копирование (или попытается скопировать `unique_ptr` и сломается).

Был класс `std::auto_ptr` — единственный в мире класс с «нетривиальным конструктором копирования» (принимал `T&`, без `const`) и забирал внутренности у источника, обнуляя его. Это решало задачу, но это подлог: говорится «копирую», а на самом деле забираю всё; случайное копирование молча мутировало источник. `auto_ptr` был настолько ужасен, что его из C++ **удалили** полностью (что само по себе нонсенс — обычно сохраняют совместимость).

rvalue-ссылки и move-семантика

В C++11 появились **rvalue-ссылки** — `T&&`. Это не «ссылка на ссылку», а ссылка **другого типа**, позволяющая отличить конструктор копирования от конструктора перемещения. Разница с копирующим конструктором — в одном амперсанде, но семантика другая: пользователь явно отдаёт владение.

Контракт rvalue-ссылки: вызывающая сторона передаёт объект, из которого вы можете забрать все внутренности — он ей больше не нужен. Единственное, что вы гарантируете, — что для объекта будет вызван деструктор. То есть вы обязаны оставить объект в состоянии, в котором деструктор может корректно отработать (нельзя забрать половину полей, если деструктор ожидает консистентность).

Перемещающий конструктор: забираем данные, у источника обнуляем:

```
Holder(Holder&& other) {
    data_ = other.data_;      // (1)
    size_ = other.size_;
    other.data_ = nullptr;    // (2)
    other.size_ = 0;
}
```

Перемещающий оператор присваивания («удалить левый объект, присвоить себе метаданные правого») — чуть сложнее, главное — не присвоить в себя самого:

```
Holder& operator=(Holder&& other) {
    if (this == &other) return *this;
    delete[] data_;           // (1) освобождаем то, что у нас было
    data_ = other.data_;      // (2)
    size_ = other.size_;
    other.data_ = nullptr;    // (3)
    other.size_ = 0;
    return *this;
}
```

Вариант через `std::swap` тоже можно (деструктор `other` потом всё сделает сам), но считается, что лучше не надо: в коде вроде

```
std::vector<char> buf1 = AllocateBuffer(10_GB);
std::vector<char> buf2 = AllocateBuffer(10_GB);
buf1 = std::move(buf2);
DoTheRealCalculations(buf1);
// buf2 (10 GB) освободится только здесь, в конце функции
```

своп откладывает освобождение огромного буфера до смерти `other`.

Чтобы не дублировать код, можно писать перемещающий оператор присваивания и использовать его в конструкторе: `*this = std::move(other);` (конструктор вызвать повторно на созданном объекте нельзя).

Зачем это нужно: считаем аллокации

```
Holder CreateHolder(int size) {  
    return Holder(size);  
}  
  
int main() {  
    Holder h(10);  
    h = CreateHolder(1000);  
    return 0;  
}
```

Копирующая версия: 3–4 аллокации (минимум 3). А хотим 2: мы создаём ровно два холдера, размером 10 и 1000, лишнее копирование — трогать память и бегать по ней — не в стиле C++. С move-конструктором/оператором временный объект из `CreateHolder` просто «крадётся» — копирования нет.

Есть разрешённая оптимизация **RVO (Return Value Optimization)**: компилятор может не создавать временный объект, а сразу сконструировать результат на месте присваивания. Она примечательна тем, что это **единственная** оптимизация, меняющая наблюдаемое поведение программы (число вызовов конструкторов), поэтому её выделяют отдельно.

`std::move`

Как вызывающая сторона должна отличить «копируй» от «забирай»? Функция `std::move`. Следует из названия: она **ничего не перемещает**. Это просто **каст к rvalue-ссылке**, памяти не касается:

```
int main() {  
    Holder h1(1000);           // h1 — lvalue  
    Holder h2(std::move(h1));  // move-конструктор вызван (на входе rvalue)  
}
```

Эквивалент `std::move(x)` — `static_cast<Holder&&>(x)` (точнее, через `remove_reference`). Реализация в стандартной библиотеке:

```
template<typename T> struct remove_reference { typedef T type; };  
template<typename T> struct remove_reference<T&> { typedef T type; };  
template<typename T> struct remove_reference<T&&> { typedef T type; };  
  
template <class T>  
inline typename remove_reference<T>::type&&  
move(T&& t) {  
    using U = typename remove_reference<T>::type;  
    return static_cast<U&&>(t);  
}
```

Писать `static_cast` вместо `std::move` можно, но не надо: всегда пишите `std::move`, вас не поймут иначе. Любую lvalue-ссылку можно превратить в rvalue-ссылку кастом или `std::move`. Временные объекты (результат функции, объект, созданный по месту) автоматически приводятся к rvalue-ссылке. Для rvalue-ссылок тоже работает temporary lifetime extension (логично: их и принимают для временных объектов).

Rule of 5 / Rule of 0

Компилятор по умолчанию генерирует конструкторы/операторы присваивания по сложным правилам (если нет ни одного копирующего/перемещающего метода или деструктора, объявленного пользователем).

Хорошее правило: либо укажите все 5 специальных методов (rule of 5): копирующий конструктор, копирующий `operator=`, перемещающий конструктор, перемещающий `operator=`, деструктор; либо ничего (rule of 0). Там, где можно, ставьте `= default` или `= delete` — это покрывает подавляющее большинство случаев.

Передача параметров

Правильный способ принимать объект, который вы хотите хранить у себя:

- пользователь не готов отдавать объект → он передаёт `const T&`, вы копируете;
- готов → передаёт через rvalue-ссылку или `std::move`.

Чтобы не дублировать код, можно принимать **по значению** и делать move:

```
class Annotation {
public:
    explicit Annotation(std::string text)
        : value_(std::move(text)) {}
private:
    std::string value_;
};
```

Звучит против «строчки по значению не передают», но работает: если пользователь не готов отдать строку — он явно платит за копию в аргумент; если готов — пишет `Annotation(std::move(s))`, и строка переместится в аргумент, а потом ещё раз `std::move` — в поле. `std::move` бесплатен, два вызова move-конструктора — не беда, зато код компактный и красивый. (Так «модно».)

Никогда не делайте move у const-объектов: `const` от типа «не может отлипнуть», `std::move(text)` на `const std::string` даст `const std::string&&`, и вызовется обычный копирующий конструктор — перф-проблема, причём незаметная.

Классическая ловушка

```
class Annotation {
public:
    explicit Annotation(std::string&& text)
        : value_(text) {} // вызовется string(const string&)!
private:
    std::string value_;
};
```

Почему? Выражение `text` имеет тип `std::string` и категорию значения **lvalue**: оно именованное, у него можно взять адрес. Два амперсанда важны **только для вызывающей стороны**; внутри функции это обычный lvalue, неотличимый от обычной ссылки. Поэтому нужно ещё раз позвать `std::move`:

```
class Annotation {
public:
    explicit Annotation(std::string&& text)
        : value_(std::move(text)) {}
private:
    std::string value_;
};
```

Правило, которое надо запомнить: если хотите передать значение дальше — всегда пишите `std::move`, даже если параметр уже `T&&`. Два амперсанда отличаются для тех, кто вам передаёт значение; для вас внутри функции это то же самое, что lvalue-ссылка.

С rvalue-ссылками `unique_ptr` наконец-то стал возвращаемым: `return ptr;` из функции работает, потому что на `return` значение передаётся во временный объект как rvalue, и вызывается перемещение, а не копирование.

Кратко

1. Четыре вида памяти: automatic («стек»), static («глобальный»), dynamic («куча»), thread-local; самый сложный — динамический.

2. `new` → ровно один корректный `delete`; два `delete` — UB, ноль — утечка (безопасна, но не нужна). Для массивов — только `delete[]`: компилятору нужно знать число деструкторов, поэтому перед массивом хранится служебное поле с размером.
3. Никогда (почти) не используйте `malloc` / `operator new` / `new-expression` напрямую: `std::make_unique`, `unique_ptr` (со вторым параметром-deleter), контейнеры; в контейнерах выделение спрятано за `allocator`.
4. Выделили сырую память — сразу оберните во владеющую обёртку: исключение из любой функции на пути к `delete` = утечка. Главная фишка C++ — «закрывающая скобка»: деструкторы автоматических объектов вызываются всегда, и они освобождают ресурсы (RAII).
5. Стек не быстрее кучи как память — быстро лишь *выделение* на стеке. Большие объекты — в кучу; статические/глобальные переменные — почти всегда зло (кроме понятных констант).
6. У каждого выражения есть тип и категория значения: `lvalue` / `xvalue` / `rvalue` (`xvalue+prvalue = rvalue`). Интуиция: у `lvalue` есть адрес; «слева/справа от присваивания» — лишь историческая интуиция. Строковый литерал — `lvalue`.
7. Константная ссылка может `binds` к временному объекту: работает `temporary lifetime extension` (временный живёт до конца блока). Писать так без нужды не стоит.
8. До C++11 владение передавали через `swap`-идиому или ужасный `std::auto_ptr` (копирующий конструктор, который ворует) — его удалили из языка.
9. C++11: `rvalue-ссылки` `T&&` — контракт «забирай всё, оставь объект пригодным для деструктора». Пишем перемещающий конструктор и `operator=` (не забыть `self-check` и обнулить источник). `std::move` ничего не перемещает — это каст к `T&&`.
10. Ловушка: именованный параметр `T&&` внутри функции — `lvalue`, для передачи дальше нужен ещё один `std::move`. Не делайте `std::move` у `const`. Для принятия «копии или перемещения» — принимайте по значению и делайте `std::move` в поле. Rule of 5 / rule of 0, `= default` / `= delete`.

Организационное

- Очередная домашка (две недели на выполнение); впереди большая домашка — написать умные указатели (там всё выше прочувствуется на практике), а также дек (там разрешены `new-expression` / `operator new`).
- Отдельные будущие лекции: про `static` / `inline` / сборку программ, про `constexpr`.

Лекция 03 — Продолжаем про move семантику

HSE C++ Advanced 2025. Конспект полностью покрывает содержательную часть лекции (таймкоды указаны по записи).

1. Введение и напоминание: зачем нужна move-семантика ([08:23]–[12:30])

Move-семантика нужна, когда мы хотим передавать значения в другие части кода (функции, переменные) **не копируя, а перемещая** внутреннее состояние объекта, то есть отдавая ресурсы, которыми объект владеет.

Ключевые моменты повторения прошлой лекции:

- Смысл имеет только для объектов, **владеющих ресурсами** (вектор, файл, строка). Обычные числа перемещать бессмысленно — это так же дорого, как копировать.
- Пример: `std::string` хранится как три поля (указатель на начало, размер, `capacity`), но эти указатели могут указывать на гигабайт данных. Скопировать три указателя сильно проще, чем копировать гигабайт.
- **rvalue-ссылка (`T&&`)** — это контракт между функцией и вызывающей стороной:
 - ▶ вызывающая сторона разрешает делать с объектом *что угодно* (например, забрать все его поля и записать туда другие), гарантируя, что *позовёт на нём деструктор*;
 - ▶ принимающая сторона обязуется оставить объект в **консистентном состоянии**, на котором можно вызвать деструктор.

После move объект находится в так называемом **moved out** состоянии: на нём можно либо вызвать деструктор, либо присвоить туда что-то новое (реинициализировать). Деструктор вызывается языком как

обычно, вручную вызывать его не нужно — это именно гарантия, на которую вы можете рассчитывать внутри функции, принимающей rvalue-ссылку.

Move-семантика сильно упростила много кода и «выпилила костыли» языка.

2. Реализация перемещающего `operator=` : три варианта ([13:20]–[24:00])

Вариант 1 — через `swap` (можно, но есть тонкость)

```
Holder& operator=(Holder&& other) {
    std::swap(data_, other.data_);
    std::swap(size_, other.size_);
    return *this;
}
```

Идея: мы ничего не удаляем и не зануляем — **знаем, что пользователь позовёт деструктор** над `other`, и деструктор «сделает своё дело» с нашим старым состоянием, которое теперь лежит в `other`.

Лектор явно спросил: «хороший ли это оператор присваивания?» — и разобрал, что **не идеальный**: проблема не в самоприсваивании (при `swap` оно даже безвредно, просто лишние `swap`) и не в `double free` (это было бы и без `swap`). Настоящая проблема — **время жизни данных**:

- `std::swap` — очень тупая штука, примерно такая:

```
template <typename T>
void Swap(T& lhs, T& rhs) {
    T tmp = std::move(lhs);
    lhs = std::move(rhs);
    rhs = std::move(tmp);
}
// в идеале: lhs.Swap(rhs), но для самописных типов это тот же код
```

- После `swap` наше старое состояние (возможно, гигабайтный массив) остаётся жить в `other` до тех пор, пока вызывающая сторона не уничтожит объект. **Мы продолжаем держать данные достаточно долго, и это может стоить заметных ресурсов.**
- Пример из лекции: два `Holder` по гигабайту, один внутри другого, и тяжёлое вычисление. Внутри функции будет занято 8 байт метаданных «на ровном месте» вместо необходимых 4 — потому что оба объекта фактически владеют памятью, которую никто уже не использует.
- Важная тонкость: **в конструкторе перемещения такой проблемы обычно нет** — там вы, скорее всего, `swap`аете с «пустым» состоянием нового объекта, а вот в перемещающем `operator=` проблема реальна, т.к. объект мог уже чем-то владеть.

Вариант 2 — «удалить левое, забрать правое, занулить правое»

```
Holder& operator=(Holder&& other) {
    if (this == &other) return *this;
    delete[] data_; // (1) освобождаем своё старое состояние сразу
    data_ = other.data_; // (2) забираем метаданные правого
    size_ = other.size_;
    other.data_ = nullptr; // (3) зануляем правого
    other.size_ = 0;
    return *this;
}
```

Здесь память освобождается **сразу**, а не «когда-нибудь» деструктором `other`.

Вариант 3 — `std::exchange` (аккуратная запись варианта 2)

Лектор: «очень часто именно в перемещающих операторах присваивания и конструкторах пригождается странная функция» `std::exchange`.

```
Holder& operator=(Holder&& other) {
    if (this == &other) return *this;
    delete[] ptr_;
    ptr_ = std::exchange(other.ptr_, nullptr);
    size_ = std::exchange(other.size_, 0);
    return *this;
}
```

Как работает `std::exchange(obj, new_value)` : - записывает `new_value` в `obj` ; - **возвращает старое значение** `obj` , которое там было на момент вызова.

То есть «swap + запоминание предыдущего значения»: сначала запоминаем предыдущее значение, потом записываем новое в исходный объект и возвращаем предыдущее. Сигнатура:

`template<class T1, class T2> T1 std::exchange(T1& obj, T2&& new_value)` . Лектор пошутил, что это «единственное применение `std::exchange`, которое я знаю» — но при написании аккуратных перемещающих операций это очень удобно.

Напоминание про `std::move` : он ничего не делает

Важное разъяснение по вопросам студентов: **`std::move` ничего не перемещает, ничего не разрушает и вообще ничего не делает** — это чисто подсказка компилятору, просто `cast`:

```
// весь std::move по сути:
static_cast<T&&>(t);
```

Деструктор после `std::move` вызывается в конце области видимости как обычно — «здесь деструктор не вызывается, это просто `cast` к ссылке».

3. Правила пяти и нуля, `= default` и `= delete` ([26:14]–[31:50])

В некоторых случаях компилятор генерирует конструкторы/операторы за вас, но **правила генерации очень сложные**, «**прямо реально сложные, там не угадать**». Практическое правило:

Либо вы пишете все пять специальных методов (rule of 5), либо не пишете ни одного (rule of 0).

Пять специальных методов: 1. копирующий конструктор; 2. копирующий оператор присваивания; 3. перемещающий конструктор; 4. перемещающий оператор присваивания; 5. деструктор.

Если вы реализуете **хотя бы один** из них (или объявите его `= default` / `= delete`), вы должны позаботиться обо всех остальных.

- `= delete` — например, запрещает копирование объекта.
- `= default` — просит компилятор сгенерировать метод; он сгенерирует его **element-wise**: скопирует/переместит все члены по очереди.

Тонкость, которую лектор подчёркивал: «прописать `= default` и вообще не прописать — не одно и то же». Правила автогенерации сложные: если вы нарушили rule of 5 (например, написали свой копирующий конструктор, а перемещающий не написали), компилятор может **не сгенерировать** перемещающие операции неявно. Тогда их нужно явно попросить: `= default` . Демонстрация в live-coding: класс с двумя строками, у которого написаны кастомные копирующие операции, — перемещение перестает компилироваться, и `= default` для перемещающего конструктора/присваивания всё чинит.

Также разъяснение по синтаксису:

```
TwoStrings s2 = s1; // это НЕ оператор присваивания, это конструктор копирования!
// эквивалентно:
TwoStrings s2(s1);
```

Запись через `=` при создании нового объекта путает, но это вызов конструктора.

Практические рекомендации лектора:

- **Лучше rule of zero** — «99% классов удовлетворяют правилу rule of zero, не пишите ничего специального». Свое копирование/перемещение в 2025 году писать — маргинальный случай.
- Если всё же пишете: **копирование лучше всегда запрещать** (= delete) — «копирование вообще почти никогда не нужно».
- **Перемещение нужно чаще**, чем копирование; очень частый паттерн — копирующие операции = delete, а перемещающие — своя реализация.
- Дефолтное перемещение делает ровно то, что ожидается: мувает s1 в аналогичный s1 другого объекта, s2 в s2, и т.д.

Почему компилятор «прав», неявно удаляя копирование

Если у класса нетривиальное владение ресурсами и вы написали хоть один нетривиальный специальный метод, компилятор **неявно отключает (delete) остальные**. И он молодец: дефолтное копирование Holder скопировало бы только указатель и размер (не глубоко!), и деструктор дважды удалил бы одну и ту же память. Это именно та ошибка, которую демонстрировал компилятор: *«object of type „Holder“ cannot be assigned because its copy assignment operator is implicitly deleted»*.

4. noexcept и перемещение ([31:41]–[33:00])

Правило, которое лектор просил **просто запомнить**:

Всегда, когда пишете перемещающий конструктор или перемещающий оператор присваивания, пишите noexcept.

Почему:

- Это **сильно влияет на производительность** именно в этом случае. Если в классе noexcept не написан, то std::vector таких объектов **при релокации будет копировать их, а не перемещать** — ему требуется слово noexcept на перемещающих операциях, чтобы дать строгую гарантию безопасности исключений.
- Из перемещения **нельзя кидать исключения** — «если вы кидаете исключение из перемещения, значит вы что-то точно не так делаете».
- В копировании noexcept написать, скорее всего, нельзя: там вы выделяете память, и аллокация может зафейлиться. А перемещение — почти всегда просто swap указателей.

5. Пишем свой std::move: проблема передачи временных объектов ([33:00]–[37:40])

Наивная реализация:

```
template <typename T>
T&& Move(T& ref) {
    return static_cast<T&&>(ref);
}
```

Она работает для lvalue:

```
int main() {
    Holder h1{1e9}; // h1 is an lvalue
    Holder h2{1e9};
    h2 = Move(h1);  // ок: приняли lvalue-ссылку, превратили в rvalue-ссылку
}
```

Но с **временным объектом не работает** — нельзя передать временный объект по T&. Получается несимметрично: наш Move работает с lvalue, а настоящий std::move — и с временными объектами тоже.

Первая попытка «принимать всё» — перегрузки (в целом работает, но некрасиво). Ошибка, которую показывает компилятор при промежуточных попытках, очень показательная: он жалуется, что присваивание неявно удалено — см. раздел 3, это следствие правила пяти.

6. Forwarding (universal) references and reference collapsing ([38:56]–[47:30])

Решение — шаблонная rvalue-ссылка. Но тут начинается «очень странное поведение языка».

Наблюдение: амперсанд «пропадает»

Пишем:

```
template <typename T>
T&& Move(T&& ref) {           // шаблонная T&& — универсальная ссылка
    return static_cast<T&&>(ref);
}
```

При вызове с lvalue `h1` компилятор выводит `T = Holder&`, и хотя мы писали `T&&`, тип параметра получается `Holder& & → Holder&`. «Хотя мы принимали два амперсанда, а получился один». Куда пропал амперсанд? Это **reference collapsing** — схлопывание ссылок.

Почему компилятор выводит `T = Holder&`

Интуиция (лектор подчёркивал: «это реально нелогично, но вот какую интуицию можно получить»):

1. Компилятор пытается вывести `T` так, чтобы вызов вообще подошёл.
2. Сначала пробует `T = Holder`: тогда параметр — `Holder&&` (rvalue-ссылка), а она **может принимать только rvalue**. Но `h1` — lvalue, не стыкуется → ошибка.
3. Пробует `T = Holder&`: параметр становится `Holder& &&` — «rvalue-ссылка на rvalue-ссылку» — и по правилу языка схлопывается в `Holder&` (lvalue-ссылку). Теперь lvalue `h1` биндится. Подходит!

Это просто **правило языка, зафиксированное стандартом**: «почему так происходит? потому что в языке так написано. Это просто надо принять».

Таблица схлопывания ссылок

комбинация	результат	комментарий со слайда
<code>T& + &</code>	<code>T&</code>	Lvalues are persistent
<code>T& + &&</code>	<code>T&</code>	Lvalues are persistent
<code>T&& + &</code>	<code>T&</code>	Preserving what users want without copy
<code>T&& + &&</code>	<code>T&&</code>	Preserving what users want without copy

Главное правило, которое «нетривиально, надо запомнить»:

Lvalue-ссылка сильнее rvalue-ссылки. Если в типе встречаются и lvalue-, и rvalue-ссылочность, результат — lvalue-ссылка. Только `rvalue + rvalue` даёт rvalue.

Зачем вообще `T&&` в шаблоне? Потому что **универсальная ссылка** (forwarding reference) — конструкция `template<typename T> ... f(T&&)` — принимает и lvalue, и rvalue: при передаче rvalue она ведёт себя как rvalue-ссылка, при передаче lvalue — как lvalue-ссылка. Если бы мы написали `T&`, мы не могли бы передавать временные объекты, а `move` обязан уметь работать с временными. Если бы написали «обычную» (нешаблонную) `T&&` — не могли бы передавать lvalue.

Починенный Move через `std::remove_reference`

Теперь понятно, почему наивный `static_cast<T&&>` ломался: при `T = Holder&` каст был к `Holder& && = Holder&` — «мы move превратили lvalue-ссылку в lvalue-ссылку. Бессмысленно».

Нужна магическая конструкция `std::remove_reference_t<T>` — тип, который «откусывает все ссылки» от переданного типа: если `T = Holder&`, то `std::remove_reference_t<T> = Holder`. Тогда каст делается к

`Holder&&` — честная rvalue-ссылка, и всё работает как ожидается. Лектор: «один раз надо поизгаляться и написать `remove_reference` два раза» (в возвращаемом типе и в касте):

```
template <typename T>
std::remove_reference_t<T>&& Move(T&& ref) {
    return static_cast<std::remove_reference_t<T>&&>(ref);
}

int main() {
    Holder h1{1e9};
    Holder h2 = Move(Holder{123}); // теперь работает и с временными объектами
}
```

Реальный `std::move` устроен именно так — «чуть сложнее, чем `static_cast`, который я рисовал».

Константность: `move` от константы бесполезен

- `std::move` от константы даёт **константную rvalue-ссылку** (`const T&&`) — «самый бесполезный тип, который можно придумать».
- Почему бесполезен: из константного объекта мы **ничего не можем достать изнутри** — ни послать поле, ни переместить. Остаётся только читать, а это ничем не отличается от константной lvalue-ссылки.
- Зачем она вообще есть в языке? «Просто для симметрии — её странно было бы запрещать, но использовать тоже странно». Отсюда практическое следствие: `move` от `const`-объекта молча превращается в копирование.

7. Move на примитивных типах ([48:30]–[50:23])

- На **примитивных типах** `move` ничего не делает, это то же самое, что копирование:

```
int x = 5;
int y = std::move(x); // == int y = x;

std::string_view s1 = "abc";
std::string_view s2 = std::move(s1); // тоже бессмысленно:
// string_view — это два указателя, примитивные поля
```

`std::string_view` хранит только примитивные объекты (указатели), поэтому перемещать его смысла нет — как и любые типы из «элементарных» полей.

8. Куда что можно передавать: таблица биндинга ссылок ([50:23]–[53:00])

Слайд с таблицей «Parameter × Argument» (параметры: `&`, `const&`, `&&`, `const&&`; аргументы: lvalue, const lvalue, rvalue, const rvalue) и живой пример:

```
void f1(std::string& s);
void f2(const std::string& s);
void f3(std::string&& s);
void f4(const std::string&& s);

std::string s("Hi"); // lvalue
const std::string cs("Hi"); // const lvalue

f1(s); // OK
f1(cs); // ERROR
f1(std::move(s)); // ERROR
f1(std::move(cs)); // ERROR

f2(s); // OK
f2(cs); // OK
f2(std::move(s)); // OK
```

```
f2(std::move(cs));    // OK

f3(s);               // ERROR
f3(cs);              // ERROR
f3(std::move(s));    // OK
f3(std::move(cs));   // ERROR

f4(s);               // ERROR
f4(cs);              // ERROR
f4(std::move(s));    // OK
f4(std::move(cs));   // OK
```

Выводы лектора: - **Неконстантная lvalue-ссылка** — принимает только неконстантный lvalue. - **Константная lvalue-ссылка** — «универсальная» функция: **всё что угодно** (lvalue, const lvalue, rvalue, const rvalue); для временного объекта продлевается время жизни. - **Rvalue-ссылка** — только «результат `std::move`», т.е. rvalue (и только неконстантные: `const rvalue` не биндится). - **Константная rvalue-ссылка** — любые rvalue (и обычные, и константные); «не знаю, зачем вы это делаете, но забиндить можно». - Интуиция «вырабатывается дальше, но нужно немножко кода пописать».

Важнейшее уточнение: когда **T&&** — НЕ универсальная ссылка

Шаблонная rvalue-ссылка — это не всегда rvalue-ссылка. Но universal reference — это только та ссылка, у которой прямо на месте параметра стоит T&& с шаблонным параметром T.

```
template <typename T>
void foo(T&&);           // forwarding reference здесь

template <typename T>
void bar(std::vector<T&&>); // но НЕ здесь: std::vector<T> — не шаблонный параметр функции,
                          // шаблонный параметр здесь только T
```

Логика схлопывания работает **только** когда прямо на месте ссылки написан шаблонный параметр. Если параметр не шаблонный (или это производный от шаблона конкретный тип, например `std::vector<T>&&`), универсальной ссылки не получается.

С константностью происходит аналогично: `const T&` может прилипнуть к lvalue, потому что `T` выводится как `const int` и т.п.

9. Perfect forwarding: `std::forward` ([54:28]–[70:30])

Зачем проксировать аргументы

Частый сценарий — функция, которая оборачивает другую функцию и просто пробрасывает ей аргументы. Живые примеры:

- **emplace_back vs push_back у вектора:** `push_back` сначала создаёт временный объект, потом его перемещает в память вектора — лишнее перемещение. `emplace_back` принимает **аргументы конструктора** и создаёт объект **сразу по месту** в векторе, без перемещения. Но для этого аргументы нужно проксировать в конструктор.
- **Бенчмарк-обёртка TimeIt** : замеряет время выполнения функции, не читая её аргументы сам.

```
template <typename F, typename Arg>
auto TimeIt(F func, Arg arg) {
    auto start = std::clock();
    auto res = func(arg);
    auto end = std::clock();
    return res;
}
```

Проблема: аргумент здесь **скопирован** (и не один раз — при передаче в `TimeIt` и при передаче в `func`). Если `func` умеет принимать `rvalue`-ссылку, мы эту возможность упустили. Но и `std::move` здесь написать нельзя: если пользователь передал `lvalue`, мы его **незаметно для него мувнем** — «вам дали объект, а вы его забрали к себе, это очень странно».

Нюанс, из-за которого всё не так просто: именованная `rvalue`-ссылка — это `lvalue`

Ключевое наблюдение лекции (лектор просил это прочувствовать):

Если функция принимает `std::string&& text`, то **внутри функции `text` — это `lvalue`**: у него можно взять адрес, ему можно что-то присвоить. `Rvalue`-ссылка в сигнатуре — это про **стык функции с вызывающей стороной** (интерфейс снаружи), а не про то, как дальше передавать объект внутрь.

Типичный пример-ловушка со слайда:

```
class Annotation {
public:
    explicit Annotation(std::string&& text)
        : value_(text) {}    // вызовется std::string::string(const std::string&) — КОПИРОВАНИЕ!
private:
    std::string value_;
};
```

Правильно:

```
class Annotation {
public:
    explicit Annotation(std::string&& text)
        : value_(std::move(text)) {}    // превратить lvalue text в rvalue (xvalue, на самом деле)
private:
    std::string value_;
};
```

«Почему? Выражение `text` имеет тип `std::string` и категорию значения `lvalue`: оно именовано, у него можно взять адрес. Поэтому нужно ещё раз позвать `std::move`, чтоб превратить `lvalue` в `rvalue`».

Отсюда **практическое правило**:

Если вы хотите куда-то передать `rvalue`-ссылку дальше — всегда пишите `std::move`, даже если у вас уже `rvalue`-ссылка. Иначе будет копирование.

(Лектор также показал пример с бенчмарком конструктора вектора:

`BenchVectorConstructor(std::vector<T>&& arg)`, внутри `std::vector<T> vec(arg)` — это **копия**; чтобы побенчить `move`-конструктор вектора, внутри нужно `std::move(arg)`. `Move` в сигнатуре важен только на стыке `main` и функции.)

Но в шаблонном прокси-коде `std::move` не годится: для `lvalue` он сделает лишний `move`. Нужна условная логика: `lvalue` передавать как `lvalue`, `rvalue` — мувать.

`std::forward`

В языке есть «ужасная конструкция» — `std::forward`: «это типа `move`, но хитрый `move`».

- Если в `std::forward` передали `lvalue` — он **не** делает `move`, возвращает `lvalue`-ссылку.
- Если передали `rvalue` — возвращает `rvalue`-ссылку (делает `move`).

Важное отличие в использовании от `move`:

```
template <typename F, typename Arg>
auto TimeIt(F func, Arg&& arg) {
```

```

auto start = std::clock();
auto res = func(std::forward<Arg>(arg)); // <- нужно ЯВНО указать шаблонный параметр!
auto end = std::clock();
return res;
}

```

- В `std::move` угловой параметр не нужен, а в `std::forward` **обязательно** передать имя шаблонного типа (как он был выведен в функции, «без амперсанда», т.е. сам `Arg`).
- `forward` работает **исключительно с универсальными ссылками**: у вас должен быть (1) шаблонный параметр, (2) на него повешены `&&` прямо в параметре функции.
- Внутри `forward` — тот же `static_cast`, и там происходит ровно reference collapsing, из-за чего lvalue остаётся lvalue, а rvalue становится rvalue. Если бы `forward` можно было записать «в лоб» без шаблонной магии, он выглядел бы как `static_cast<Arg&&>(arg)` — «упражнение слушателям: проверить дома, что для rvalue получится rvalue, для lvalue — lvalue».
- Забавное наблюдение лектора: **наш первый «неправильный move» `T&& Move(T& ref)` на самом деле был `forward`'ом, а не `move`'ом** — он сохранял категорию значения, а не превращал всё в rvalue.

Отличие `move` от `forward` и когда что использовать

- Нужно **мувнуть** значение, превратить его в rvalue-ссылку — используйте `std::move`.
- Нужно **сохранить категорию значения** (lvalue передать как lvalue, rvalue как rvalue) — используйте `std::forward`.

«Они разные задачи решают. Чаще используется `move`, но `forward` в обобщённом коде постоянно встречается: вы пробрасываете аргументы туда-сюда, не понимая, какой у них реально тип, — и это нормально, вам не важно. Пробрасывайте дальше, потребитель разберётся».

10. Ref-qualifiers: перегрузка методов по rvalue/lvalue ([70:51]–[79:00])

Мы умеем перегружать методы по константности. Оказывается, можно перегружать и **по категории объекта, на котором вызван метод** — это **ref-qualifiers**. Четыре варианта:

```

int&&      Get() &&      { return std::move(value_); } // только на rvalue
const int&& Get() const&& { return std::move(value_); } // только на const rvalue
int&       Get() &       { return value_; }           // только на lvalue
const int& Get() const&  { return value_; }           // только на const lvalue

```

Как это вызывается:

```

obj.Get(); // lvalue-квалифицированная версия
std::as_const(obj).Get(); // const lvalue-версия
Widget{}.Get(); // на временном объекте — &&-версия
std::move(obj).Get(); // rvalue-версия

```

Фича «выглядит безумно» и используется нечасто, но у неё яркий пример — **билдер**.

Пример: поисковый индекс и билдер

Пишем поисковый движок: есть `SearchIndex` (по нему можно искать) и `SearchIndexBuilder`, который его строит. Разделение осмысленно: сначала индексируем кучу документов, потом строим эффективную структуру данных (структуры без онлайн-апдейтов обычно эффективнее), и только потом ищем.

```

class SearchIndex {
public:
    // ...
private:
    std::unordered_map<std::string, std::vector<int>> inverted_index_;
}

```

```
};

class SearchIndexBuilder {
public:
    void AddDocument(std::string str);
    SearchIndex Finish() {
        return SearchIndex{std::move(inverted_index_)};
    }
private:
    std::unordered_map<std::string, std::vector<int>> inverted_index_;
};

int main() {
    SearchIndexBuilder builder;
    builder.AddDocument("abacaba");
    builder.AddDocument("foo bar");
    auto index = std::move(builder).Finish();
}
```

Проблема интерфейса: семантика `Finish` — «вызывается ровно один раз»; после него объектом по-хорошему пользоваться нельзя. Но язык никак не запрещает позвать `Finish` второй раз — и тогда из так же наполненного билдера вернётся пустой индекс: накопленная структура уже увезена `std::move` 'ом в первый индекс. Это нелогично и странно. Идея «а давайте `delete this`» не работает (мы ничего не аллоцировали через `new`; да и деструктор тогда позовётся дважды — и всё сломается).

Решение: ref-qualified `Finish`

```
class SearchIndexBuilder {
public:
    void AddDocument(std::string str);
    SearchIndex Finish() && {
        return SearchIndex{std::move(inverted_index_)};
    }
private:
    std::unordered_map<std::string, std::vector<int>> inverted_index_;
};
```

Теперь `builder.Finish()` **не компилируется**: *«this“ argument to member function „Finish“ is an lvalue, but function has rvalue ref-qualifier»*. Пользователь обязан написать `std::move(builder).Finish()`.

Честности ради, лектор отмечает: `std::move(builder).Finish()` дважды подряд — всё ещё возможно. Но: - за таким кодом **цепляются статические анализаторы** (clang-tidy умеет детектить); - двойной `std::move` в коде бросается в глаза программисту, в отличие от «тихого» второго вызова `Finish` — вы тем самым подчёркиваете, что `Finish` должен быть последним и зваться на уже умирающем объекте. - Идеально было бы `delete this`, но «здесь оно не выражается никак». Это джентльменское соглашение.

Альтернатива: рантайм-проверка через `std::exchange`

```
SearchIndex Finish() {
    if (std::exchange(finished_, true)) {
        throw std::runtime_error("Finish called twice");
    }
    return SearchIndex{std::move(inverted_index_)};
}
private:
    bool finished_ = false;
```

Как работает: первый вызов — `finished_` было `false`, в неё пишется `true`, `exchange` возвращает `false`, ветка не выполняется. Второй вызов — `exchange` возвращает `true` → исключение.

Минус: это проверка **в рантайме** — «как в питоне: запустили программу, она что-то покрутила и упала». Лучше, когда корректность проверяется статически, при компиляции. Поэтому ref-qualifiers нужны в первую очередь для этого (плюс иногда — запретить вызов метода на временных объектах, когда аллоцированный ресурс должен жить дольше, чем пользователь предоставил).

Лектор также упомянул, что в Rust эта проблема решена элегантно: там `self` передаётся в метод **явно** (как обычный аргумент), и его можно забирать по значению, требовать `&mut` и т.д.; перегрузок нет, и «usage after move» не выражается.

11. C++23: deducing this ([81:00]–[91:00])

В C++23 появилась фишка **deducing this** («выводимый this») — лектор признался, что больше всего ждал из C++23 именно её.

Синтаксис:

```
// Обычный вариант с ref-qualifier (до C++23):
SearchIndex Finish() && {
    return SearchIndex{std::move(inverted_index_)};
}

// C++23, deducing this:
SearchIndex Finish(this SearchIndex&& self) {
    return SearchIndex{std::move(self.inverted_index_)};
}
```

Ключевые моменты: - `this` пишется **перед обычным параметром** (`this SearchIndex&& self`) — это пометка, что именно этот аргумент — инстанс класса. Он обязательно идёт первым, но после него могут быть и другие аргументы: вызывается как обычно, `finish(42)` — явный объект неявно подставляется первым. - Раньше `this` передавался неявно, теперь он — **явный параметр** (`self`), как `self` в Python/Rust. Можно навешивать на него константность, ссылки, шаблонность. - Внутри такого метода нет неявного `this`: поля доступны через `self`, `this->` писать нельзя. - Главное преимущество: **self — обычный параметр, его тип выводится**. Можно сделать его **универсальной ссылкой** (`this auto&& self`), и тогда одна шаблонная функция знает, на каком ref-qualifier и с какой константностью её вызвали (lvalue/rvalue/const/не const). До C++23 узнать это было невозможно: `this` — всегда указатель, всегда lvalue, и для разной логики приходилось писать 4 (или даже 8) копирастных перегрузок, пробрасывающих всё в одну внутреннюю функцию.

Живой пример из реального кода (файл `result.h`): метод `Ok()` у `TResult` был реализован **четырьмя** одинаковыми перегрузками, отличающимися только ref-qualifier'ами:

```
const T* Ok() const& {
    return Y_MATCH(std::as_const(*this), const T*) {
        Y_CASE(const T& value) { return &value; }
        Y_CASE(const E& error) { return nullptr; }
    };
}

// + ещё перегрузки &, &&, const&& — полная копия паста
TMaybe<T> Ok() && {
    return Y_MATCH(std::move(*this), TMaybe<T>) {
        Y_CASE(T&& value) { return std::move(value); }
        Y_CASE(E& error) { return std::move(error); }
    };
}
```

С deducing this это сводится к одной шаблонной функции с `this auto&& self` (или явным шаблонным параметром `U`), в которой тип `self` показывает, как нас вызвали, и по нему можно включать разную логику. В примере лектора файл сократился на десятки строк копиясты. Как программно понять, какой тип скрывается под `auto` / `U` — «это, наверное, следующая лекция» (там будут type traits, decltype, concepts).

Лектор: в курсе C++23, по-моему, не включён, «но на дворе 25-й год — если очень нужно, напишите, постараемся включить».

12. Кратко

1. **rvalue-ссылка — контракт**: вызывающий разрешает растащить объект и гарантирует вызов деструктора; принимающий оставляет объект в консистентном (moved out) состоянии. Деструктор вызывает язык, а не вы.
 2. **std::move ничего не делает** — это просто `static_cast<T&&>`. Он не перемещает и не разрушает.
 3. **Перемещающий оператор=**: swap-версия проста, но держит старые данные «до деструктора other»; версия с `delete` + `std::exchange(other.ptr_, nullptr)` освобождает память сразу. `std::exchange(obj, new)` записывает новое значение и возвращает старое — лучший друг при написании move-операций.
 4. **Rule of 5 или rule of 0**, ничего посередине. Где можно — `= default` / `= delete`. 99% классов — rule of 0; копирование почти всегда можно удалить, перемещение нужно чаще (и его часто придётся явно `= default`, иначе компилятор не сгенерирует).
 5. **Перемещающие конструктор и оператор=** — всегда **noexcept**, иначе `std::vector` при релокации будет копировать элементы. Кидать исключения из move нельзя; из сору — почти неизбежно можно (аллокация может упасть).
 6. **Именованная rvalue-ссылка внутри функции — это lvalue**: чтобы передать её дальше как rvalue, снова пишите `std::move`. Ловушка: `Annotation(std::string&& text) : value_(text) {}` — это копирование.
 7. **Universal reference** — только `template<typename T> void f(T&&)`. Для нешаблонных типов и `std::vector<T>&&` это обычная rvalue-ссылка.
 8. **Reference collapsing**: lvalue-ссылка «сильнее» — `& + && → &`; только `&& + && → &&`. Компилятор выводит `T = U&` для lvalue-аргументов — именно поэтому в своём `move` нужен `std::remove_reference_t<T>` (дважды: в типе возврата и в касте).
 9. **std::forward<T>(arg)** сохраняет категорию значения: для lvalue не мувает, для rvalue мувает; требует универсальную ссылку и явное указание шаблонного параметра. Нужен в прокси-коде (`emplace_back`, бенчмарк-обёртки); `std::move` — когда нужно превратить объект в rvalue безусловно.
 10. **Ref-qualifiers (&, &&, const&, const&& после сигнатуры метода)** позволяют перегружать методы по категории объекта; типичное применение — метод-«финализатор» (`SearchIndex Finish() &&`), который обязан вызываться на умирающем объекте. Рантайм-альтернатива — `if (std::exchange(finished_, true)) throw ...`, но она проверяется только при исполнении.
 11. **C++23 deducing this** (`Finish(this SearchIndex&& self)`): делает объект явным параметром, позволяет одну шаблонную функцию вместо 4–8 копиястных перегрузок и даёт знать, с какой ссылочностью/константностью вызван метод.
 12. На **примитивных типах и string_view move == копирование**; move от `const`-объекта даёт `const T&&` — «самый бесполезный тип» и молча превращается в копирование.
-

Организационное

- Следующая лекция — типы, type traits, метапрограммирование (типы на этой лекции не успели начать).
 - Новая «бумажка» (раздатка) будет выдана в ближайшие дни; на ближайшем семинаре — `benchmark CompressedPair`, для которого понадобится механизм **Empty Base Optimization** (пригодится и в большой домашке).
 - Большая домашка — ориентировочно через неделю (домашек три, тянуть не хотят).
-

Лекция 04 — Типы. Шаблоны

Курс «HSE C++ Advanced 2025». Конспект полностью заменяет просмотр лекции. Таймкоды — ориентир по записи.

1. Зачем нужны типы ([00:13–03:41])

Тип в C++ — это множество возможных значений объекта плюс множество операций, которые над ним можно совершать. Тип ограничивает допустимые операции и придаёт объекту понятную семантику. Наличие типов позволяет писать сложные нетривиальные программы: в языках без типов (пример лектора — ассемблер) невозможно понять, что вообще лежит в памяти.

Практическое правило: используйте типы почаще. Не таскайте «просто числа» и «просто строки» по всей программе — заводите много **локальных типов под свои задачи**.

Из типов можно собирать другие типы, комбинируя их. Структура — декартово произведение множеств значений своих полей: для такой структуры валидна *любая* комбинация значений полей, никаких дополнительных инвариантов она не несёт:

```
struct TwoInts {
    int64_t hi;
    int64_t lo;
};
```

Если поверх множества значений появляется **инвариант**, пишем класс: за счёт инкапсуляции поля прячутся в приватную секцию, менять их может только сам класс, и он гарантирует выполнение инварианта. Методы такой тип несёт для того, чтобы инвариант сохранять:

```
class TwoInts {
public:
    TwoInts() : hi_(0), lo_(0) {}
    int64_t GetHigh() const { return hi_; }
    int64_t GetLow() const { return lo_; }
private:
    int64_t hi_;
    int64_t lo_;
};
```

Тонкость, которую лектор подчёркивает отдельно: отличий между `class` и `struct` нет совсем, кроме дефолтного модификатора доступа (`struct` — поля публичные, `class` — приватные). Можно написать `class + public:` или `struct + private:` — получится одно и то же.

2. Размер типов и «пустые» объекты ([03:41–05:20])

- **Любой тип имеет ненулевой размер в байтах.** Байт в C++ — это `char`; стандарт требует только **не меньше 8 бит** в байте, максимум не оговорён (может быть 16 или 32 — на практике таких машин нет, но формально).
- **Пустых типов не существует:** даже пустая структура имеет размер 1:

```
struct Empty {};  
static_assert(sizeof(Empty) == 1);
```

- **Почему так:** в C++ все объекты одного типа обязаны иметь **разные адреса** — zero-sized types, как в Rust, в C++ нет. Зачем нужно это свойство — лектор честно говорит «непонятно»; вероятно, чтобы отличать «ваш» объект от соседнего (в частности, в move-операторе присваивания сравнивают `this` с адресом `other`).

3. Зачем хранить пустые типы: аллокатор и `vector` ([05:20–08:06])

Зачем вообще хранить пустые типы без лишней памяти? Классический пример — `std::allocator`: у него **нет никаких полей**, только набор методов. Содержательно там есть два метода — `allocate` и `deallocate` (выделить и освободить память; всё остальное — «мишура» и deprecated-методы). Хранить в нём нечего, но хранить сам аллокатор нужно: например, `std::vector<T, Allocator>` вторым параметром принимает аллокатор и хранит его у себя.

Как устроен `vector` (по исходникам `libc++`, которые лектор показывал на экране): указатель на начало, указатель на конец (размер) и **compressed pair** из указателя на конец выделенной памяти и аллокатора. Если аллокатор пустой (без полей), `sizeof(std::vector<int>)` — **24 байта**, то есть 3 указателя, хотя хранится 4 поля.

4. Empty Base Optimization (EBO) ([08:06–11:24])

Compressed pair опирается на лазейку в стандарте под названием **empty base optimization (EBO)**. Общее правило: **пустой базовый класс** компилятору разрешено не закладывать лишние байты — размер наследника может равняться сумме размеров его полей.

Почему это не нарушает правило «у объектов одного типа разные адреса»:

- Адрес любого объекта-структуры **совпадает с адресом её первого поля** (и с адресом первой базы). По одному адресу спокойно живут объекты **разных** типов — запрещены только два объекта **одного** типа по одному адресу.
- Поэтому пустая база может «лежать» по тому же адресу, что и сам объект, и по тому же адресу, что и первое поле (это разные типы) — противоречия нет.

Когда EBO не срабатывает — из слайда лекции:

```
struct TwoEmpty : Empty {
    Empty anotherEmpty;    // база Empty и поле Empty — один тип!
};
static_assert(sizeof(TwoEmpty) == 2);

struct Proxy1 : Empty {};
struct Proxy2 : Empty {};
struct TwoEmptyBases : Proxy1, Proxy2 {};
static_assert(sizeof(TwoEmptyBases) == 2);
```

В первом случае пустая база не может совпасть по адресу с первым полем — они **одного типа** `Empty`. Во втором — при двух (пустых) базах размер тоже 1 не выйдет: обе базы должны иметь разные адреса.

Ментальная модель наследования: в первом приближении, если нет виртуальных функций, базы — это «первые неявные поля» объекта. С виртуальными функциями появляется таблица виртуальных функций, и layout становится заметно сложнее.

5. `[[no_unique_address]]` (C++20) ([11:24–17:49])

В C++20 появился атрибут `[[no_unique_address]]` — вешается на поле и позволяет хранить пустые типы без места, причём «более аккуратно», чем EBO, — он позволяет нарушать даже требование об уникальности адресов объектов одного типа:

```
struct Empty {};
```

```
struct Foo {
    [[no_unique_address]] Empty empty;
    int x;
};
// sizeof(Foo) == 4; без атрибута было бы 8
```

Лектор живьём экспериментировал с четырьмя пустыми полями и `int`:

```
struct Foo {
    [[no_unique_address]] Empty empty;
    [[no_unique_address]] Empty empty2;
    [[no_unique_address]] Empty empty3;
    [[no_unique_address]] Empty empty4;
    [[no_unique_address]] int x;
};
```

```
int main() {
    Foo f;
    static_assert(sizeof(Foo) == 4);
    std::cout << &f.empty << '\n' << &f.empty2 << '\n' << &f.x << std::endl;
    assert(&f.empty == &f.empty2);
}
```

Наблюдения из эксперимента: поведение **странное и неочевидное** — адреса пустых полей могут совпадать (в том числе с адресом `int`-поля, «влезая» внутрь него), у каких-то полей атрибут «срабатывает», у каких-то нет; лектор не уверен, что поведение хорошо стандартизировано — «такие поля не занимают места», а как именно это обеспечивает компилятор — на его усмотрение.

Почему это опасно: нельзя сравнивать указатели, чтобы различать объекты. В перемещающем `operator=` обычно пишут `if (this == &other) return *this;` — при `[[no_unique_address]]` у **разных** объектов могут быть одинаковые указатели, и такая проверка перестает работать корректно.

Практические правила лектора: - Атрибут стандартизирован, его можно использовать «безбоязненно» — **но лучше не надо**. - Задачу про `compressed_pair` нельзя сдать через `[[no_unique_address]]` — «это скучно»; смысл задачи — прочувствовать и понять ЕВО, она сильно нетривиальнее и забавнее. - Универсальный (шаблонный) конструктор для такого класса — терпимая практика, но писать его надо **супер-аккуратно**: его легко перепутать с `move`-конструктором и другими конструкторами.

6. Шаблоны: зачем и синтаксис ([18:20–24:08])

Шаблон — механизм **параметризовать типами или значениями на этапе компиляции** другую сущность. Шаблонизировать можно практически любую сущность: классы, функции, переменные (нельзя `namespace` — но это и странно было бы). Чаще всего шаблонизируют классы и функции:

```
template <class T1, class T2>
struct pair {
    T1 first;    // Use of T1
    T2 second;  // Use of T2
};

int x = 0;
pair<int, int> p(x, x);
pair<int, int&> p(x, x); // Because why not.
x = std::max<int32_t>(0, x);
```

Важно: `class` vs `typename` в списке шаблонных параметров. - Слово `class` здесь **ничего не значит**: сюда можно передать **абсолютно любой тип** — указатель, число, ссылку, `void`, лямбду, что угодно. - `class` и `typename` здесь **полностью взаимозаменяемы**, разницы никакой (в отличие от пары `struct / class`, где хоть что-то есть). - Лучше писать `typename`: слово `class` сбивает с толку, намекая, будто нужен именно класс. В стандартной библиотеке обычно пишут `typename`.

Шаблонные переменные — редко используемая, но полезная фишка (понадобится дальше в курсе).

7. Non-type template parameters (NTTP) ([23:09–26:29])

Шаблонными параметрами могут быть не только типы, но и **значения**. Называются они по-исторически нелогично — **non-type template parameter (NTTP)**: сначала были типы, потом «давайте добавим не только типы».

Классический пример — `std::array`: он принимает тип **и размер**, причём размер именно шаблонным параметром:

```
template <typename T, size_t Count>
class Array {
    T buf[Count];
};
```

Зачем размер шаблонным? Чтобы он был **известен на этапе компиляции**: компилятор понимает, какой размер ему передали, ещё при сборке программы. Через конструктор константу «не просунешь» — конструктор работает в рантайме, и компилятор честно ругается понятной ошибкой вида *non-type template argument is not a constant expression*.

Общая рамка: шаблонные аргументы — это «аргументы этапа компиляции», а аргументы функций — рантайма. Если у значения есть рантаймовая природа, шаблонным аргументом оно быть не может.

8. Инстанциация и «утиная типизация» ([25:28–32:57])

Шаблон — это описание того, как выглядят **все** подстановки аргументов: вы описываете много классов/функций разом, отличающихся только местами использования шаблонных параметров (например, `Array` — это пачка классов, отличающихся типом и размером буфера).

- **Инстанциация** — превращение шаблона в конкретный класс/функцию: компилятор берёт тело и заменяет все упоминания шаблонных параметров на переданные аргументы.
- Инстанциация происходит **только при первом использовании** конкретного набора аргументов.
- Это уже зачатки метапрограммирования: вы описываете не столько код, сколько то, **как код появляется**.

Ключевое следствие — **ошибки видны только при инстанциации**:

- Внутри шаблона можно писать довольно странные вещи — пока шаблон не инстанцирован, компилятор не ругается. Например, `Array<void, 42>` невалиден (массив из `void` невозможен), но ошибка появится **только на строке использования**, причём «внутри шаблона»:

```
template <typename T, size_t Count>
class Array { /* ... */ };

// Всё компилируется, пока шаблон не использован:
// Array<void, 42> x2; // <- вот ЗДЕСИ ошибка, внутри инстанцирования
```

- В C++ «утиная типизация»: в отличие от дженериков Rust (где вы явно выписываете ограничения — трейты — на шаблонный аргумент и внутри можете использовать только их), в плюсах никакие свойства `T` заранее не выписываются. Внутри шаблона можно делать с `T` **что угодно**: ожидать у него конструктор от трёх чисел, любой «unknown method» — компилятор согласится, пока шаблон не инстанцируют. «Если что-то выглядит как утка и крикает как утка — это утка».
- Механизмы, ограничивающие допустимые `T`, есть: старый `enable_if` и новые **концепты** (C++20). Но они работают **в одну сторону**: ограничивают множество аргументов, которые пользователь может подставить, а внутри шаблона по-прежнему можно делать более хитрые предположения о типе. Ошибка всё равно «выстрелит» только при компиляции конкретной инстанциации.

Практический пример «утиной» проверки с экрана:

```
template <typename T, size_t Count>
class Array {
    void Foo() {
        T value{42, 42, 42};
        value.unknown_method(); // ок, пока Array<T, Count> не инстанцирован
    }
    T buf_[Count];
};
```

9. C++20: строки как NTTP и компиляция регулярок ([32:57–39:00])

В C++20 сильно расширили допустимые типы NTTP. Теперь можно, например, передавать строки:

```
template <util::fixed_string regex>
class StaticRegularExpression;

StaticRegularExpression<("[0-9a-f]{8}-){3}([0-9a-f]{8})"> regex;
```

Пример выше — UUID: 4 секции по 8 шестнадцатеричных цифр. Прелесть конструкции: строку можно **обрабатывать на этапе компиляции** и в зависимости от её содержимого генерировать разные типы, добавлять функции и т.д.

Почему `util::fixed_string`, а не `std::string` / `std::string_view`? Стандарт для NTTP-типов выписывает требования: все поля публичные, сами поля — фундаментальных или таких же «простых» типов. У `std::string` и `std::string_view` поля приватные — они не подходят. Поэтому нужен свой `fixed_string`: тип с конструктором от строки, удовлетворяющий требованиям. Реализовать его самому — не тривиальная задача.

Зачем это нужно — компиляция регулярок. «Скомпилировать регулярку» — значит преобразовать язык регулярных выражений в конечный автомат для быстрого матча. Обычно это дорогое преобразование делается в рантайме, поверх пишут сложные оптимизаторы. Идея шаблонной библиотеки: пусть **компилятор C++ сам оптимизирует движок регулярок** — пишете простой матчер, подставляете регулярку в шаблон, а оптимизатором занимается C++-компилятор. Это реально работает: библиотека показывает очень хорошую скорость.

Минусы (лектор подчёркивает): - работает **только для регулярок, известных на этапе компиляции** — литералов; пользовательскую строку так не обработать; - безумно медленная компиляция, неудобный дебаг; - за много лет «хороших применений» так и не появилось — «непонятно зачем нужно, но красиво».

10. Вывод шаблонных аргументов ([39:00–46:00])

Шаблонные аргументы (и типы) могут **выводиться** из контекста — их часто можно не писать:

```
template <typename T>
T inc(T a) { return a + 1; }

inc(5); // call inc<int>

int x = -1;
x = std::max(0, x); // call std::max<int>();
```

Вывод бывает и сложным — компилятор умеет выводить шаблонный аргумент из **свойств** типа аргумента:

```
template <int I>
int foo(std::integral_constant<int, I> _unused) {
    return I * I;
}

void bar() {
    foo(std::integral_constant<int, 5>{}); // call foo<5>
}
```

Здесь компилятор видит: функция принимает `integral_constant<int, I>`, а ей передали `integral_constant<int, 5>` — значит, `I == 5`. Тип он вывел не напрямую совпадением, а «разобрав» устройство типа аргумента. Математически это законно: устройство `integral_constant` известно, из аргумента очевидно следует `5`.

Иногда вывод не удаётся:

```
uint32_t x = 0;
x = std::max(0, x); // long compiler error
```

`max` принимает два аргумента **одного** типа `T`. Здесь `0` — это `int`, а `x` — `unsigned int`: по левому аргументу `T = int`, по правому `T = unsigned int` — конфликт. Кастовать в какую сторону — компилятор не решает.

Почему компилятор не «додумывает» (не берёт более широкий тип): потому что такое поведение было бы непрозрачным — иногда угадывает, иногда нет, пользователь не понимает, чего ждать. Правило проще: **если лёгкого вывода не получилось — значит, нужно явно указать тип** (`std::max<uint32_t>(0, x)`).

Текст ошибки при этом nowadays вполне читаемый: `*no matching function for call to „max“ ... candidate template ignored: deduced conflicting types for parameter „Tp“ („int“ vs. „uint32_t“ (aka „unsigned int“))`*. Лектор отмечает: лет 10 назад компиляторы выдавали «ошибки на несколько экранов»; конкуренция GCC и Clang (а потом и пример Rust с понятными сообщениями) заставила всех сильно улучшить диагностику. А вот **address sanitizer** специально очень многословен: он рассчитан на ситуацию, когда ошибка не воспроизводится неделями и по итоговому отчёту её долго разбирают с дебаггером — там чем больше информации, тем лучше (у санитайзеров хорошая вики, как они устроены внутри).

11. CTAD: вывод аргументов шаблона класса (C++17) ([49:30–58:00])

До C++17 у шаблонных **классов** аргументы не выводились. С C++17 появился **class template argument deduction (CTAD)** — аргументы шаблона класса выводятся через конструкторы:

```
template <class T1, class T2>
struct pair {
    pair(const T1& f, const T2& s) : first(f), second(s) {}
    T1 first;    // Use of T1
    T2 second;   // Use of T2
};

pair p{0, 1}; // CE до C++17, pair<int, int> после C++17
```

Компилятор смотрит на аргументы конструктора, находит в них совпадения с параметрами шаблона и выводит `T1 = int`, `T2 = int`. Но дефолтный вывод **не всегда даёт то, что хочется**. Пример с экрана: передали строковый литерал в конструктор, принимающий ссылку — компилятор вывел `U = char[6]` и попытался копировать массив; «дефолтного вывода не хватает, нужно что-то более хитрое».

Тонкость про «универсальную ссылку»: если написать параметр `T&&` у **нешаблонного** конструктора шаблонного класса — это **не** универсальная ссылка: сам конструктор не шаблонный, и вывод работать не будет. Чтобы был forwarding reference, шаблонным надо делать сам конструктор.

Решение — **deduction guide**, «способ направить компилятор на правильный вывод»:

```
template <class T1, class T2>
struct pair {
    pair(const T1& f, const T2& s) : first(f), second(s) {}
    T1 first;
    T2 second;
};

template <typename T1, typename T2>
pair(const T1&, const T2&) -> pair<const T1&, const T2&>;

pair p{0, 1}; // pair<const int&, const int&>
```

Конструкция из двух строк: слева от `->` — «прототип конструктора» (как будто шаблонная функция с тем же именем, что у класса), справа — **какие именно аргументы шаблона класса выводить**. Имена параметров гидов (`T1`, `T2` здесь) никак не связаны с параметрами самого класса — это независимые сущности.

Логика работы: при создании объекта в «конструктор-подобном» режиме компилятор **сначала смотрит на deduction guides** и лишь потом идёт искать конструкторы. В примере выше дефолтный (неявный) гид от конструктора `pair(const T1&, const T2&)` вывел бы `pair<int, int>`; явный гид переопределяет вывод — получается пара **ссылки** `pair<const int&, const int&>`.

Лектор подчёркивает: CTAD и deduction guides — в отличие от «красивых» штук вроде строк-NTTP, — **реально полезная** фишка, но дефолтного вывода обычно не хватает, особенно в случае со ссылками, и гиды приходится писать.

Оргвопросы (коротко, [63:30])

- Скоро будет выдана большая домашка («наконец-то»).
- Третья (маленькая) домашка тоже выдана сегодня.

Кратко

1. **Тип** = множество значений + множество операций. Заводите много локальных типов под задачи; структура — декартово произведение полей без инвариантов; инвариант → класс с инкапсуляцией.
2. `struct` и `class` отличаются **только** дефолтным доступом (`public / private`).
3. В C++ **нет пустых типов**: `sizeof` любого типа ≥ 1 (байт = `char` ≥ 8 бит), потому что объекты одного типа обязаны иметь разные адреса (в Rust есть zero-sized types — здесь нет).
4. Пустые типы полезны (`std::allocator` без полей); `vector` хранит аллокатор в **compressed pair** — за счёт **ЕВО** пустая база не занимает места, поэтому `sizeof(vector<int>) == 24`.
5. ЕВО ломается, если пустая база конфликтует по типу с первым полем (`TwoEmpty`) или баз несколько (`TwoEmptyBases`) — размер растёт.
6. `[[no_unique_address]]` (C++20) убирает место под пустое поле, но поведение неочевидно и может ломать сравнение указателей (`this == &other`); «можно использовать, но лучше не надо».
7. Шаблоны параметризуют сущности **типами и значениями на этапе компиляции**; `class` и `typename` в параметрах взаимозаменяемы — пишите `typename`; NTTP — значения-константы времени компиляции (пример: `std::array<T, N>`).
8. **Инстанциация** — подстановка аргументов при первом использовании; все ошибки в теле шаблона проявляются только в момент инстанциации; C++ — «утиная типизация»: ограничений на `T` нет (`enable_if` / концепты ограничивают только допустимые аргументы, но не тело).
9. C++20 разрешил сложные NTTP (например, строки через самописный `util::fixed_string`) — отсюда регулярки, компилируемые в конечный автомат на этапе компиляции: быстро, но только для compile-time-регулярок и с тяжёлой компиляцией.
10. Вывод шаблонных аргументов: у функций — из аргументов (в т.ч. из «устройства» типа, как `foo(integral_constant<int, 5>) → foo<5>`); при конфликте типов (`std::max(0, x)`) вывод не происходит — **явно указывайте тип**. С C++17 аргументы шаблона класса выводятся через конструкторы (CTAD), а **deduction guides** позволяют задать вывод явно и сильнее дефолтного — особенно важно при ссылках.

Лекция 05 — Ошибки. Исключения. Noexcept

Откуда берутся ошибки

Практически любая операция в программе может пофейлиться. Классический пример — аллокация памяти: она не пройдёт, если запросили слишком много (80 ГБ на машине с 32 ГБ). Но может упасть и по «независимым от вас» причинам: при сильной фрагментации аллокатор не найдёт подходящий непрерывный кусок. На практике в современных условиях аллокация фейлится только при действительно большом запросе, но никто не мешает аллокатору вести себя менее предсказуемо — например, в embedded-окружениях с малой памятью, где фрагментация влияет сильнее. Даже такая простая операция фейлится — значит, обработка ошибок unavoidable.

Как с ошибками живут в C: `goto cleanup`

Типичный подход до исключений — ввести коды ошибок и завести «канал» для их передачи. Пример из ядра Linux: функция готовит страничку в файловой системе для записи. Пока идут подготовительные операции, фейлиться нечему, а дальше выполняется операция, возвращающая указательную структуру, которую проверяют на ошибку. Классический приём ядра (и C в целом): очистка при ошибке выносится в конец функции, и при каждой ошибке делается `goto` на соответствующую метку:

```
err = -EIO;      /* ошибка ввода-вывода */
goto out;
...
```

```
err = -ENOSPC; /* no space */
goto out7;
```

При разных ошибках — разные метки, и правило такое: **чем дальше по коду функции случилась ошибка, тем выше должна быть метка**. Если ошибка случилась после захвата семафора, прыгаем не на `error`, а на `out7` — семафор освобождается; если до захвата — на `error`. Синтаксис меток ровно такой же, как у `case` в `switch` — метки в `switch` работают как метки в `goto`, только их нужно располагать в порядке выполнения. Можно даже сделать несколько цепочек обработчиков с `return` друг за другом — компилятор не ругается на «мёртвый» код.

Удобно ли так жить? С одной стороны, это очень явно («explicit лучше implicit», как говорит Python — но «если Python так говорит, не факт, что это правильно»). С другой — это **очень ошибкаопасный подход**: каждый раз нужно держать в голове функцию целиком и супераккуратно проверять, из какой точки какую метку прыгать. Непонятно, зачем жить в мире, где код надо так писать.

Четыре способа вернуть код ошибки из функции

Пусть у нас есть `openFile(path)`, которая должна вернуть файл или ошибку. Слайд лектора перечисляет варианты:

```
// 1. Output parameter
File* openFile(const char* path, Error* error);

// 2. Return error code
Error openFile(const char* path, File** result);

// 2.1 Tuple (Value, Error), e.g. <charconv>
struct OpenFileResult {
    File* File;
    Error Error;
};
OpenFileResult openFile(const char* path);

// 3. Thread-local (global) state (errno)
File* openFile(const char* path);
```

1. **Output-параметр**: файл возвращаем, ошибку записываем в переданный указатель. Проблема: если вернулся ненулевой файл и что-то записалось в ошибку — непонятно, что произошло.
2. **Код ошибки как возвращаемое значение**, результат — через output-параметр (`File**` — двойная звёздочка, потому что хотим вернуть `File*`). Так выглядит очень много С-шных функций; сисколы устроены похоже (обычно в одном возвращаемом значении намешаны и результат, и ошибка).
3. **Структура из двух полей** — так выглядят `to_chars` / `from_chars` из C++17: одно поле — результат, второе — ошибка.
4. **Глобальное/thread-local состояние** — `errno`: функция возвращает только «маркер» ошибки (`nullptr`), а сама ошибка лежит сбоку в глобальной переменной (`fopen` именно так).

Вариант 4 лектор разбирает отдельно: `errno` — глобальная переменная «номер ошибки». Это удобно? Конечно нет. Правда, с многопоточностью проблем нет (в современных реализациях `errno` — thread-local), это скорее свойство самой `fopen`. Лектор демонстрирует живой код:

```
#include <unistd.h>
#include <cerrno>
#include <cstdio>
#include <iostream>

int main(int, const char* argv[]) {
    FILE* f = fopen(argv[1], "r");
    if (f == nullptr) {
        std::cout << "Got error: " << strerror(errno) << std::endl;
```

```

    } else {
        std::cout << "Opened file" << std::endl;
    }
}

```

Главная беда этого подхода видна, если погуглить, как используется `foren`: **никто не обрабатывает ошибку**. В дефолтном коде открывают файл и сразу читают из него. «Очень легко забыть» — сишники, может, и привыкли, но на плюсах будет хуже. Дополнительно возвращаемое значение само по себе неинформативно: `nullptr` говорит, что ошибка есть, но чтобы понять какая — надо ещё достать `errno`.

Математическая проблема: произведение вместо суммы

Лектор предлагает взглянуть как математик, а не инженер: у математика функции ошибок не возвращают. Тип `struct { File* file; Error error; }` — это **произведение (декартово произведение) множества всех возможных файлов на множество всех возможных ошибок**. Осмысленно ли это? Нет: функция никогда не вернёт одновременно и валидный файл, и ошибку; зато тип допускает и «файл + ошибка», и «ничего + ничего» — все эти состояния бессмысленны, но язык их разрешает.

А хочется не произведение, а **сумму** (объединение): «либо результат, либо ошибка» — то есть по сути `std::variant`. В C такого механизма нет (`union` без шаблонов — безумно неудобно), поэтому в C очень страдают.

Лектор приводит **Go** как язык, где это та же проблема, возведённая в стандарт: там нет исключений, и все функции возвращают и значение, и ошибку:

```

f, err := os.Open(path)
if err != nil {
    // ...
}

```

Это джентльменское соглашение: при фейле функция возвращает нулевое значение и ошибку. Но это то же декартово произведение, только замаскированное. В Go функции могут возвращать три значения, одно из которых ошибка, и иногда даже «осмысленно», что часть знаний приезжает сбоку; ненулевое число функций возвращает и значение, и ошибку одновременно — порицается это исключительно на словах.

Язык не проверяет за вас, что множество возвращаемых значений адекватно. Вы используете такую семантику не потому, что она хороша, а потому что язык слаб.

Остальные проблемы кодов ошибок

- **Легко пропустить обработку.** Даже в Go, где ошибку явно вернули, её легко забыть обработать (там стоят линтеры, ловящие необработанные ошибки).
- **Нудно.** В Go каждая нетривиальная функция из одной строчки превращается в четыре (плюс пятый пробельчик, чтобы читалось). Явная обработка и оборачивание ошибок с контекстом — это полезно, но писать это очень неприятно.
- **Производительность.** Каждый `if` — это performance penalty, даже если он обычно срабатывает. Это противоречит ключевому принципу C++: *не платишь за то, что не используешь* — если программа не фейлится, странно платить за ошибки.
- **Теряется контекст.** Программа возвращает ошибку с самого низа: «no such file or directory». Какой файл? Зачем программа его открыла? Почему без него не работает? Приходится запускать `strace` и догадываться, что она пыталась прочитать свой конфиг. В хорошем Go-коде ошибка накапливает контекст: «я инициализировалась, для этого читала конфиг, для этого открыла файл с таким именем, его не оказалось» — как ошибка компиляции в Rust или Clang. Но это требует **ручной разметки в каждом месте**, где ошибка возникает и пробрасывается вверх.
- **Предопределённый набор кодов ошибок** — «**вообще поражение**»: фиксированная классификация, в которую не влезешь со своими ошибками.

Коды ошибок в современном C++

В C++ есть `std::errc` — обёртка над `errno` с более читаемыми именами: не `EALREADY`, а `connection_already_in_progress`, не `EPIPE`, а `broken_pipe` и т.п. Заголовок `<system_error>`; в `<charconv>` — `from_chars` / `to_chars`, возвращающие `std::from_chars_result`:

```
struct from_chars_result {
    const char* ptr; // куда дочитали / где случилась ошибка
    std::errc    ec;  // код ошибки, «красиво названный по-плюсовому»
};
```

Лектор критикует сигнатуру `from_chars`: результат возвращается в последний параметр **по ссылке**, «очень же понятно, какой из трёх параметров меняется» (сарказм). Почему `from_chars` (C++17!) возвращает ошибку, а не исключение? Парсить числа будут часто, и невалидные числа — **довольно вероятный результат** этой функции, а исключения тормозят, когда ошибка происходит: они сильно дороже кода ошибки. Плюс со структурой будут лишние копирования: если бы результат писался сразу в число, компилятор бы оптимизировал. Мелочь, но характерная: C++ почти не поддерживает удобный механизм «современных» ошибок без исключений.

Зачем придумали исключения: проблема конструкторов

Исключения в C++ появились вместе с классами и конструкторами. Цепочка такая: конструктор **ничего не возвращает** — как ему сообщить об ошибке? (В Rust конструкторов как таковых нет: есть функции, возвращающие инстанс класса, и они могут вернуть что угодно — значение, ошибку, несколько ошибок.) Остаётся либо «плохой вариант» (создать объект-заглушку), либо метод `isValid()`:

Псевдокод лектора: конструктор вектора выделяет память, запоминает `capacity`, а пользователь обязан перед использованием позвать `isValid()`. Чем это плохо? **Это надо проверять руками, а программисты ленивы и постоянно что-то забывают**. Аналогия: «поднимите руки, кто обрабатывал результат `printf`» (он возвращает число напечатанных символов) — поднимает один человек. Так и здесь: никто не позовёт `isValid`, все будут создавать вектор, а потом он **тихо взорвётся**, когда память не выделится — и удачи в отладке, когда вектор внезапно инициализировался `nullptr`.

Правило лектора: методы вида `isValid` — зло. Объект должен быть валиден сразу после создания любого экземпляра класса, без дополнительных ручных проверок.

Отсюда и исключения: раз конструкторы ничего не возвращают, для репорта ошибки нужен другой канал.

Исключения: синтаксис и раскрутка стека

Кидать можно **любой объект** (можно `int`, можно что угодно, кроме `void`):

```
throw 42;
```

Ловим через `try` / `catch`:

```
#include <stdio>
#include <iostream>

class Noisy {
public:
    Noisy() { puts(__PRETTY_FUNCTION__); }
    ~Noisy() { puts(__PRETTY_FUNCTION__); }
};

void Bar() {
    throw 42;
}

void Foo() {
```

```
Noisy n;
try {
    Bar();
} catch (int i) {
    std::cerr << "Got exception " << i << std::endl;
}
}
```

При выбросе исключения **автоматически вызываются деструкторы всех объектов на пути раскрутки стека** до ближайшего подходящего `try / catch`. Исключения красиво играют с деструкторами (RAII): если ресурсы освобождаются в деструкторе, то независимо от того, как вы вышли из скоупа — нормально или через исключение — они будут почищены. Семантика такая: объект выкидывается до ближайшего по вложенности подходящего `catch`, и вы можете пролететь много вложенных функций — особенно в плохо написанных программах, где `try / catch` стоит аж в самом начале, и любая ошибка пропускает значительные части программы.

Необработанное исключение: `std::terminate`

Нюанс, который лектор демонстрирует экспериментом: если кинуть `Noisy` и сразу выбросить исключение без `try / catch`, деструктор не вызывается. Программа падает с `SIGABRT`:

`libc++abi: terminating due to uncaught exception of type int`. Когда для исключения не находится подходящего `catch`, рантайм C++ вызывает `std::terminate`, которая терминирует программу — **и стек не раскручивается, деструкторы не вызываются**. Стандарт говорит, что деструкторы *могут* позваться, а могут нет — `implementation-defined`. Обычно не зовутся, потому что реализация устроена так: **сначала** рантайм ищет ближайший `catch`, **потом** раскручивает к нему стек, а не идёт пошагово, пока не встретит `catch`.

Правило лектора: нельзя писать код, рассчитанный на то, что деструктор будет вызван всегда. В Rust, кстати, прямо написано, что `Drop` может не позваться. В C++ деструкторы обещают звать «почти всегда», но нельзя делать так, чтобы безопасность внешней системы (например, ОС) обеспечивалась только деструктором: если вашу программу прибудут снаружи (`SIGKILL`), никакой деструктор не поможет — система про исключения C++ не знает. Надеяться можно лишь на то, что *либо программа завершится, либо деструктор рано или поздно вызовется*.

Поэтому паттерн «деструктор как последняя линия обороны» оформляют явно — лектор показывает класс, который сам себя добивает, если работа не была завершена:

```
class Foo {
public:
    void Finish() {
        Finished_ = true;
    }
    ~Foo() {
        if (!Finished_) {
            abort();
        }
    }
private:
    bool Finished_ = false;
};
```

Исключение из деструктора

Следующий эксперимент: `Noisy` кидает исключение **из деструктора** (`throw 42;` в `~Noisy()`), а вокруг — `try / catch`. Всё равно `terminate`, хотя, кажется, `catch` написан. Причина: исключение вылетает из деструктора **в момент, когда уже идёт раскрутка стека по другому исключению**, а во время раскрутки допускается **только одно активное исключение** — второе немедленно даёт `terminate`.

Поэтому **деструкторам по умолчанию неявно приписан noexcept** (компилятор: «Destructor has an implicit non-throwing exception specification», а на `throw 42` внутри: «„~Noisy“ has a non-throwing exception specification but...»). Почему именно деструкторам? Интуиция аудитории верна: объект полужив, а мы кидаем ещё исключение.

Важная тонкость: не любое исключение из деструктора запрещено. Если **внутри** деструктора исключение выброшено и **поймано до выхода из него** — всё валидно. Лектор показывает «вся программа написана как функция `myMain`»:

```
void MyMain() {
    // вся программа, может даже кидать исключения
}

struct World {
    ~World() {
        try {
            MyMain();
        } catch (...) {
        }
    }
};
```

Здесь деструктор `World` (глобального объекта) вызывает `myMain` в `try / catch` — исключение не вылетает за пределы деструктора, и всё работает, даже с неявным `noexcept`. Проблема только если исключение **вылетает за пределы** деструктора — тогда мы возвращаемся в раскрутку стека со вторым исключением, и всё ломается.

noexcept : две формы

`noexcept` — ключевое слово с **двумя формами**:

1. **Спецификатор** `noexcept` / `noexcept(константное выражение)` : «функция не кидает», условно — «функция `noexcept`, если `bool` внутри `true`». По умолчанию у обычных функций его нет; у деструкторов он стоит неявно.
2. **Оператор** `noexcept(выражение)` : возвращает `bool` — правда ли выражение помечено как `noexcept`.

Ключевых слов мало, поэтому одно слово на две роли. Отсюда знаменитая конструкция, которую лектор показывает в реальном коде на GitHub:

```
noexcept(noexcept(f(...)))
```

Внутренний `noexcept` — оператор, проверяющий, является ли вызов `noexcept`; внешний — спецификатор. Обёртывание функции в лямбду с такой спецификацией — очень популярный код; смысл: «лямбда `noexcept` тогда и только тогда, когда обёрнутые функции `noexcept`».

Теперь можно сделать `noexcept(false)` в деструкторе и кидать исключения легально:

```
~Noisy() noexcept(false) {
    puts(__PRETTY_FUNCTION__);
    throw 42;
}
```

Хорошо ли это? **Нет, это очень опасно**: проблема-то никуда не ушла. Если вы кидаете исключение из деструктора, вы либо супер уверены, что активных исключений в этот момент нет, либо не понимаете, что делаете — «скорее всего, второе». Много кода на GitHub делает это «зачем-то».

Как узнать, что раскрутка уже идёт

Если всё же пишете `noexcept(false)` в деструкторе, обычно проверяют, что сейчас нет активных исключений. До C++17 для этого была функция `std::uncaught_exception()` (начиная с C++11 — `noexcept`), в C++17 задеприкейчена, в C++20 удалена. Вместо неё с C++17 — `std::uncaught_exceptions()` (с `s`),

возвращающая **число** активных непойманных исключений (и `constexpr` с C++26). Семантика по `crtpreference`: функция определяет, есть ли в текущем потоке исключение, которое брошено и ещё не вошло в подходящий `catch` — то есть идёт ли прямо сейчас `stack unwinding`. Иногда кидать исключение даже при `uncaught_exceptions() > 0` безопасно — если оно будет поймано до выхода из деструктора.

Бонус-наблюдение с экрана: внутри `libc++` даже деструкторы стандартных потоков обвешаны проверками — `catch (...)` вокруг `cleanup`-кода и заголовки вида `_LIBCPP_HAS_NO_EXCEPTIONS`: библиотека сама вынуждена защищаться от исключений из деструкторов и уметь собираться с выключенными исключениями.

Кратко

1. Фейлить может любая операция — даже аллокация памяти (перепросили/фрагментация), поэтому обработка ошибок обязательна.
2. С-подход «коды ошибок + `goto cleanup`» (как в ядре Linux) явный, но ошибкаопасный: метки надо расставлять в точном соответствии с точкой фейла.
3. Четыре канала возврата ошибки: `output`-параметр, код возврата + `File**`, структура-«кортеж» (как `from_chars_result`), `thread-local errno` (как `fopen`) — у всех свои неудобства, и `errno` в реальном коде массово игнорируют.
4. Главная математическая беда: такие интерфейсы возвращают **произведение** типа «результат × ошибка», а нужна **сумма** («либо результат, либо ошибка» — вариант). Go возводит это произведение в стандарт.
5. Прочие беды кодов ошибок: легко пропустить обработку, нудно писать, платим за ошибки даже когда их нет (против принципа «не платишь за неиспользуемое»), и теряется контекст ошибки (Go-стайл оборачивание лечит, но требует ручной разметки).
6. `std::errc` / `<system_error>` и `from_chars` / `to_chars` в C++17 — коды ошибок, а не исключения, по соображениям производительности: ошибки — вероятный результат парсинга, а исключения при ошибке дороги.
7. Исключения придуманы ради конструкторов: конструктор ничего не возвращает, а метод `isValid` никто не позовёт — объект должен рождаться валидным.
8. При исключении деструкторы объектов на пути раскрутки стека вызываются автоматически — база RAII. Но непойманное исключение → `std::terminate` → `SIGABRT` без раскрутки и деструкторов (рантайм сначала ищет `catch`, потом раскручивает).
9. На вызов деструктора нельзя полагаться (например, `SIGKILL` извне); критичную «незавершённость» обрабатывайте явно (`abort()` в деструкторе, как у `Foo`).
10. Деструкторы неявно `noexcept`: исключение, вылетевшее из деструктора, — мгновенный `terminate`; кидать можно, если исключение поймано до выхода (`try { MyMain(); } catch (...) {}`). Для этого — `noexcept(false)` + проверка `std::uncaught_exceptions()` (C++17; старая `uncaught_exception` задеприкейчена в C++17, удалена в C++20). У `noexcept` две формы: спецификатор и оператор; идиома `noexcept(noexcept(...))` популярна в обёртках.

Орг. момент

Лекция — повтор темы обработки ошибок с первого курса с углублением, чтобы дальше строить более современные подходы. Много живого кодинга в `nvim` (`exceptions.cpp`, `main.cpp`), реальные падения (`SIGABRT`, вывод `libc++abi: terminating due to uncaught exception`), просмотр `errc.h`, исходников `libc++` + (`fstream`), `crtpreference` и реального кода на GitHub (`yandex/perforator`). Отвлечения: похвала Go («классный язык, лучше Python как вспомогательный к плюсам»), Rust (`Drop` может не позваться; конструкторов нет), Haskell («никто не знает»), опрос про обработку результата `printf` и упрёк, что никто не смотрит выхлоп компилятора — «ошибки здесь, компилятор предупреждает, что вы отстрелили себе ногу».

Лекция 06. Паттерны. ODR

Что такое паттерны проектирования

Паттерн проектирования — типовой метод решения задач, которые часто возникают в коде. Понятие общее для всей разработки, не только для C++. Паттерн — это не изобретение, а *обнаружение*: одна и та же

идея независимо возникает в разных кодовых базах, и ей дают название, чтобы проще было общаться с другими программистами и классифицировать решения.

Важные оговорки лектора: - Паттерны — высокоуровневые идеи; их почти всегда хочется **модифицировать под себя**. - **На паттерн не надо молиться**. Это просто инструмент, который иногда позволяет удобнее структурировать код, а иногда — нет; уместность надо решать каждый раз заново. (Лектор шутит словами Страуструпа: «есть языки, которые все ругают, а есть языки, которые никто не использует» — так же и с паттернами.) - Классическая книга — «Design Patterns» «банды четырёх» (Gamma, Helm, Johnson, Vlissides), но она про ООР в целом; здесь разбор более прикладных, плюсовых вещей.

Template Method (шаблонный метод)

«Шаблонный» здесь — не про `template`, а про «общий метод для ряда классов». Ситуация: есть несколько наследников одной базовой сущности, и в них хочется выполнять действия по одному и тому же алгоритму.

Идея: общий алгоритм описывается в базовом классе как обычный метод, а **точки кастомизации** — отдельные методы, которые наследники переопределяют, чтобы чуть поменять поведение. Сам алгоритм остаётся в базовом классе и может быть сколь угодно сложным.

```
class Animal {
public:
    void DoStuffInTheMorning() {
        GetUp();
        MakeSound();
        AskForFood();
        GoForAWalk();
    }
};

class Dog : public Animal {
public:
    virtual void MakeSound() override {
        Bark();
    }
};
```

Почему это ценно: в других языках, где наследования нет или оно устроено иначе, в базовом классе обычно нет логики, и подобное выразить сложно. Лектор отмечает: **в Rust такое выразить нельзя**. В C++ это естественно: вы сами, переставляя код в поисках аккуратного решения, легко приходите к такой структуре. Это позволяет кастомизировать поведение сложного многошагового алгоритма, не трогая его общую структуру.

Singleton — самый спорный паттерн

Singleton — класс, гарантирующий существование единственного своего представителя и предоставляющий доступ к нему. Это «классик, которым плотно обязан почти любой код в мире», и потому непонятно, полезен он или вреден. Когда синглтон осмыслен (обсуждение с аудиторией): - **Аллокатор**. Память — глобальная сущность: одно адресное пространство, один набор плашек. Сущность, выделяющая память из системы, обычно одна. Хотя это упрощение: разные аллокаторы сильно по-разному ведут себя под разными нагрузками (один быстро отдаёт память, второй эффективно освобождает её разом), и лектор видел бинарники, где рядом живут несколько аллокаторов. - **Настройки приложения / окно**. Код приложения обычно запускается один раз; в игре окно ровно одно. Но класть окно в синглтон *не обязательно* — и лектор соглашается, что это скорее плохая идея. - **Логер** — самый показательный пример. Это сущность, которая централизованно сохраняет отладочные сообщения программы.

Дилемма логера: либо протаскивать его через всю программу (поле `logger` в каждом классе, приём в каждом конструкторе — «очень много кода на всю кодовую базу, и вы точно где-то заблудитесь: пишете подсистему, а логер в неё забыли дотащить, и надо просовывать его через 10 классов»), либо завести условно-глобальный синглтон, доступный из любой точки.

Практический вывод лектора: **без глобальных логеров тоже можно жить** — протаскивание не настолько страшно, зато проще тестировать: можно создать два экземпляра приложения с разными настройками и

сравнить поведение. С синглтоном («приложение точно одно») тесты с двумя экземплярами разваливаются. Но в отдельных случаях (память, логгер) синглтон всё же осмыслен.

Проблема: глобальные переменные и единицы трансляции

Зачем вообще отдельно выделять синглтон — разве глобальная переменная не решает всё? Рассмотрим корректный вариант:

```
// x.h
#pragma once
int& GetX();
void IncX();
```

```
// x.cpp
int x = 0;
int& GetX() { return x; }
void IncX() { ++x; }
```

`x` живёт в одном `.cpp`, а другие файлы ходят к нему через функции. Но что будет, если написать `int x = 0;` прямо в `x.h`? Каждый `.cpp`, инклюдящий заголовок, получит **свою** переменную `x`, и на этапе линковки программа не соберётся: линкер пишет `duplicate symbol '_x' in: ... a-...o ... b-...o` — `clang++: error: linker command failed with exit code 1`.

Здесь слово **символ** означает любую сущность, которую видит компоновщик: функцию или переменную. Два разных объектных файла объявили один и тот же глобальный символ `x` — двойное определение.

Чтобы понять, почему это происходит, нужно вспомнить этапы сборки:

- **Препроцессор** (часть компилятора) раскрывает все директивы препроцессора; `#include` — это **просто вставка содержимого файла**. Можно выполнить все подстановки руками — результат не поменяется (демо: `clang++ -E kek.cpp | rg -v '^#\d+' | wc -c` даёт ~1.69 млн символов из маленького файла с `<iostream>`).
- Каждый `.cpp` — отдельная **единица трансляции** (translation unit), компилируется независимо. Всё ломается только на линковке, когда оба файла уже скомпилированы.

Объявления и определения. One Definition Rule

- **Объявление (declaration)** вносит новое имя для сущности: `int foo();`, `class Vector;` (forward declaration), `extern const int SomeExternallyDefinedInt;` — говорит, что «где-то есть такая сущность».
- **Определение (definition)** целиком определяет, как сущность устроена: `int bar() { return 42; }`, `struct StringView { const char* begin; const char* end; };`. **Каждое определение является объявлением, но не наоборот.**
- Перед тем как использовать сущность, нужно её объявить:

```
void bar() {
    foo(); // Error: Use of undeclared identifier 'foo'
}
void foo(); // Declaration
void baz() {
    foo(); // OK
}
```

One Definition Rule (ODR): в финальной программе должно быть *как минимум одно* определение каждой используемой сущности, и *почти всегда больше одного определения во всей программе — ошибка* («почти» — потому что в правиле куча исключений; по-человечески — почти всегда ошибка). ODR знает только про результат подстановки заголовков в `.cpp`-файлы — **про сами хедеры он ничего не знает**.

Что можно писать в заголовочных файлах

Разбора этого вопроса лектор посвящает большую часть лекции. Обходные пути для переменных: `static` делает сущность глобальной не для программы, а для **данного .cpp-файла** — линковка не упадёт, но это

будут независимые `x` в каждом файле, что не то, чего хочется. Вывод: **в заголовочных файлах переменные писать не нужно** — кроме констант (`static constexpr`, `const`) и статических переменных классов.

Что же можно:

1. **Объявления** — сколько угодно.
2. **Определения классов.** Классы и структуры — исключение из ODR: в стандарт специально написано, что можно иметь несколько определений класса в программе, **покуда определения совпадают по-байтово**. Методы внутри этих определений тоже должны совпадать. (С классами правила одинаковости на самом деле ещё сложнее.) Именно поэтому стандартная практика — писать классы в заголовке целиком либо определение класса с объявлениями методов.
3. **Шаблоны — целиком в заголовочных файлах.** Чтобы инстанцировать шаблон и сгенерировать реализацию с подстановкой аргументов, компилятору нужно видеть весь шаблон. Исключение: если шаблон приватно используется ровно в одном `.cpp`, его можно определить один раз там.
4. **`inline`-функции.** Если определить обычную функцию в заголовке (`void Swap(int& a, int& b) { ... }`) и инклудить его в два `.cpp`, линкер снова ругается: `duplicate symbol '__Z4SwapRiS_'`. Это **mangled name** плюсовой функции: имя, в которое закодированы namespace и типы аргументов; `c++filt <<< __Z4SwapRiS_` даёт `Swap(int&, int&)`.

Ключевое слово `inline` — вопреки названию, оно *не* гарантирует встраивание функции по месту вызова и практически не связано с оптимизацией inlining. Оно означает: «эту функцию можно определять несколько раз, я обещаю, что все определения по-байтово одинаковые; иначе — undefined behavior». То есть это разрешение нарушать ODR при условии одинаковых тел. Если в программе случайно окажутся две `inline`-функции с разным телом — «поражение»: иногда линкер это ловит, чаще нет, и это самые безумные ошибки.

Почему тогда методы, определённые внутри класса в заголовке, линкуются без явного `inline`? Потому что **методам класса `inline` добавляется неявно** — так же, как шаблонным функциям. Отсюда правило: **явно помечать методы `inline` не нужно** («если вы это видите, так писали люди, не понимающие, как это работает»).

Тонкость про шаблоны: они «немножко заифаны» в стандарте — все шаблонные функции неявно `inline`.

Чем опасны нарушения ODR

Линкер обычно не проверяет, что одноимённые определения действительно совпадают (казалось бы, «надо просто посчитать хэши от всех функций», но этого не происходит). Пример из практики: в двух далёких `.cpp` определены структуры с одинаковым именем, но разным набором полей (разного размера), и в обеих единицах трансляции используется `vector` этой структуры. Ключ инстанциации совпадает, поэтому линкер случайно выберет одну из двух — например, версию для узкой структуры применит к широкой. Методы вектора от разных инстанциаций перемешаются, и вы будете дебажить «сломанный вектор», хотя проблема — нарушение ODR.

`#pragma once` и include guards

`#pragma once` не связана с несколькими единицами трансляции: она защищает от **повторного включения заголовка внутри одной единицы трансляции**. Это нестандартное расширение, но единственное, которое поддерживают все и его использование не особо порицается. Формально чуть лучше — классические **include guards**:

```
#ifndef UNIQUE_KEY_H_
#define UNIQUE_KEY_H_
// ... содержимое заголовка ...
#endif
```

Минус: громоздко, и **ключик должен быть реально уникальным** — лектор видел кодовые базы, где ключом берут «какую-то рандомную строчку». `#pragma once` делает то же самое «по-нормальному», без необходимости придумывать ключ; как именно реализация распознаёт файл — зависит от компилятора, но адекватные реализации что-то похожее делают.

Цена раздельной компиляции: LTO, ThinLTO, PGO

Разделение на `.cpp` и заголовки — компромисс. Когда `.cpp` видит только объявления, компилятор не может делать предположений об устройстве функций и **хуже оптимизирует**: чем больше компилятор видит, тем лучше оптимизирует. С другой стороны, программы разбиты на тысячи `.cpp`, каждый компилируется независимо: поменяли одну строчку — пересобирается маленький кусочек, остальное закешировано.

Отсюда практическое правило: **маленькие функции, которые редко меняются, но часто используются, стоит выносить в заголовочный файл** — вы размениваете время компиляции всех пользователей класса на производительность. Но «если у вас 20 маленьких методов — возможно, определение „маленькости” сбилось»; лучше в хедере всё-таки ничего не писать, строгого ответа нет.

Если бы всё собиралось одним `.cpp`, мы бы получали **порядка 10% дополнительной производительности**. Это подтверждает режим **Link Time Optimization (LTO)**: компиляция каждого файла завершается сохранением промежуточного представления, а на финальной линковке оптимизируется вся программа целиком — виден весь контекст, можно делать более агрессивные оптимизации. 10% перфа — «очень много, как пять лет работы ребят в компиляторах», но время сборки сильно растёт. Первый LTO был однопоточным и неюзабельным; **ThinLTO** — баланс: хорошо параллелится, чуть жертвует производительностью; большие бинарники (браузеры, поиск) в нём линкуются «условно полчаса на 80 ядрах». Дальше есть **PGO (Profile Guided Optimization)** — «совсем продвинутая штука, приходите на стажировки».

Практика: во время разработки собираете без LTO, финальную версию — с LTO, по ней гоняете тесты и выкатываете. LTO — проверенная штука, логику не ломает (баги бывают и без него). Альтернатива — **amalgamated-режим**: вся программа руками собирается в один `.cpp`; так работает SQLite — самая распространённая база данных, множество файлов схлопываются в один огромный и компилируются в один бинарник. «LTO для бедных».

Правила работы с заголовочными файлами

- **Шаблоны** — **целиком в заголовочных файлах**. Если заголовок очень большой, его можно разбить на «то, что видит пользователь», и реализацию: в конце первого файла инклюдится второй (можно с трюком через `#define` макроса вокруг `#include` и `#undef` после). Во втором файле для работы редактора и автодополнения дополнительно инклюдят первый — рекурсивно, чтобы IDE понимала контекст.
- **Методы классов**: небольшие — целиком в заголовке (они неявно `inline`), всё остальное — в `.cpp`.
- **Обычные функции**: в заголовке — только объявления, определения — в `.cpp`.
- В `.cpp` всё приватное пишем в `unnamed namespace` — «это как `static`, только по-модному»: сущности добавляется уникальное имя, ни с чем не пересекающееся; работает так, как будто на каждой сущности написан `static`.
- **Переменные в заголовках не писать** (кроме констант и статических полей классов).

Реализация синглтона: эволюция (разбор на примере логера)

Примечание: код в этой части лектор набирал вживую; кадры с финальным кодом не сохранились, ниже — версии по ходу лекции.

Версия 1. Функция, возвращающая ссылку на сущность:

```
// logger.cpp
static Logger logger;           // static: приватен этому .cpp

Logger& GetLogger() {
    return logger;
}
```

Проблемы: глобальное имя `logger` может пересечься с другой переменной; если убрать `static`, другие программисты могут «считать» и прочитать её без функции. И главная — **Static Initialization Order Fiasco**: глобальные переменные в одном `.cpp` инициализируются сверху вниз, а между разными `.cpp` — в каком-то порядке, о котором ничего нельзя сказать. Если из инициализации другой глобальной переменной пойти `GetLogger().Log(...)`, логер может быть ещё не инициализирован. По байке, из-за

обилия любимых в Google глобальных переменных браузер Chrome какое-то время стартовал «много секунд» даже при `return 0` в начале `main` — глобальные переменные пришлось вычищать.

Версия 2 — статическая переменная в функции (решает и порядок инициализации, и лишнюю инициализацию, если логер не используется):

```
Logger& GetLogger() {
    static Logger logger; // инициализируется при первом вызове
    return logger;
}
```

Нюанс: у синглтона «непонятно, когда статические переменные разрушаются» — при завершении программы деструктор другой статической переменной может захотеть писать в уже разрушенный логер. Поэтому для синглтонов часто делают намеренную утечку — деструктор никогда не вызовется. «В Rust любят повторять: *memory leaks are safe*» — утечка безопасна, если она разовая на старте (а вот если программа живёт долго и течёт по гигабайту в день — уже нет).

Версия 3 — класс с ленивой инициализацией:

```
class Logger {
public:
    static Logger& GetInstance() {
        if (!instance) {
            instance = std::make_unique<Logger>();
        }
        return *instance;
    }
private:
    static std::unique_ptr<Logger> instance;
};
```

Лектор: «стало сильно лучше, это хуже, чем статическая переменная в функции, но терпимо»; код **не потокобезопасен** (одновременный заход в `if` — неопределённое поведение; что такое потоки — тема дальше), и динамическая аллокация тут не нужна.

Версия 4 — CRTP (Curiously Recurring Template Pattern, «любопытно повторяющийся шаблон шаблона»). Мотивация: хочется, чтобы `GetInstance` был методом класса, у которого **приватный конструктор** — тогда создать экземпляр в обход синглтона нельзя, а `GetInstance` (метод) имеет доступ. Базовый класс оперирует шаблонным параметром `T`, который сам является его наследником — «родитель знает про наследника»; выглядит странно, но стандартно разрешено и работает (лектор: «кажется, CRTP случайно нашли — бага или фича компиляторов — и решили, что можно много классного кода писать»).

```
template <class T>
class Singleton {
protected:
    Singleton() = default; // protected: зовут только наследники
    static std::unique_ptr<T> instance;
public:
    static T& GetInstance() {
        // инициализация instance лениво, при первом обращении
        return *instance;
    }
};

class Logger : public Singleton<Logger> {
public:
    Logger() { /* ... */ }
};

// использование: Logger::GetInstance().Log("...");
```

Плюс подхода: общая часть выделена в базовый класс, который переиспользуется всеми «синглтонными» классами; рекурсии при внимательном рассмотрении не возникает. Остаются вопросы потокобезопасности («проблемы есть, но потоков у нас пока нет») и дефолтный конструктор.

Версия 5 — финальная. Дефолтный конструктор — сильное ожидание: логер может хотеть писать в файл или в сеть, то есть принимать аргументы. Решение — `variadic templates + std::forward`: `Singleton` пробрасывает произвольные аргументы в конструктор `T`, и можно сделать, например, `initInstance(file)`. «Красиво? Мне кажется, довольно красиво. Синглтон — очень спорный паттерн, но он забавный, его весело реализовывать».

Чем плох синглтон

- **Явное лучше неявного.** Завязывая программу на глобальную переменную, вы делаете много допущений.
- Нельзя создать несколько экземпляров приложения в тестах (а такое реально бывает нужно).
- Сильно связанная программа: **сложно рефакторить** — непонятно, кто какие её части использует. Чем больше выносите в синглтон, тем сложнее потом менять программу; для логера это терпимо, для более хитрых синглтонов — не надо.

Кратко

1. Паттерн — обнаруженный в разных кодовых базах типовой приём; на паттерны «не молятся», каждый раз решают, уместен ли он.
2. **Template Method**: общий алгоритм — в базовом классе, наследники переопределяют точки кастомизации; в Rust такое не выразить.
3. **Singleton** — единственный экземпляр + глобальный доступ; уместен для истинно глобальных сущностей (аллокатор), спорен для логера, плох для «почти глобальных» вещей.
4. `#include` — текстовая вставка; каждый `.cpp` — независимая единица трансляции.
5. **ODR**: минимум одно определение каждой сущности; почти всегда больше одного — ошибка (линкер: `duplicate symbol`).
6. Объявление ≠ определение; каждое определение — объявление, но не наоборот.
7. В заголовках можно: объявления, определения классов (совпадающие по-байтово), шаблоны целиком, `inline`-функции. Переменные — нельзя (кроме констант и статических полей классов).
8. `inline` — не про встраивание, а про «можно несколько определений, тела обязаны совпадать»; методы в классе и шаблоны — неявно `inline`.
9. Нарушения ODR линкер обычно не ловит: две одноимённые структуры разного размера ломают `vector` самым безумным образом.
10. Практика: маленькие популярные функции — в хедер (ценой компиляции), всё приватное в `.cpp` — в unnamed namespace; LTO/ThinLTO возвращает ~10% производительности; синглтон лучше делать статической локальной переменной функции или CRTP-классом с приватным конструктором и пробросом аргументов через `std::forward`.

Оргвопросы

Лектор — Сергей Скворцов. В конце: «сегодня точно уже будет домашка, пока ещё по ошибкам» (в демо мелькала задачка `patterns/any` — свой класс `Any`). Лекция шла ~74 минуты; много времени заняла живая демонстрация в vim: эксперименты с `x.h / a.cpp / b.cpp`, `duplicate symbol`, `c++filt` для mangled-имён, `clang++ -E`; часть демо не показалась с первого раза из-за проблем с DNS на маке.

Лекция 07 — Метaprogramмирование

Что такое метaprogramмирование

Метaprogramмирование — это подход, при котором метaproграмма порождает другую программу в качестве результата своей работы. Мы описываем не сам код, а то, как сгенерировать код.

Инструменты в C/C++: **макросы** (на них построен весь C и C++ — инклюды; кажется архаикой, но встречается часто), **шаблоны** (описывают не код, а то, как сгенерировать код), **constexpr** (вычисления при компиляции) и **метаклассы** (в слайдах «26», по факту ближе к C++29; стандартизаторы пока не продавили).

Макросы: простая текстовая замена

Макросы — это просто текстовая замена, которая ничего не знает про C++. Препроцессор работает до всего остального и оперирует чистым текстом — хоть по ассемблеру, хоть по питону: сначала препроцессор, потом интерпретация текста.

```
#define MAX_SIZE 100
int array[MAX_SIZE];

#define sqr(x) ((x) * (x))
sqr(2 + 3)          // -> ((2 + 3) * (2 + 3))
sqr(heavy_func())   // -> ((heavy_func()) * (heavy_func()))
```

Тонкость: `heavy_func()` вызывается **дважды** — подстановка чисто текстовая. Условная компиляция — проверки на одном из самых ранних этапов компиляции, например выбор платформы (какой заголовок инклудить на Unix, какой на Windows, иначе — `#error`).

assert и почему он не функция

`<cassert>`: `assert(cond)` при невыполнении условия печатает в stdout, где именно случилась ошибка, и завершает программу: `Assertion failed: (5 == 2 * 2), function main, file main.cpp, line 4.`

Почему `assert` не функция? При вызове функции невозможно узнать, из какой строки и файла её вызвали — 30 лет люди не могли это сделать (до C++20 и `std::source_location`). А для ассерта это критично. В C++20 решение:

```
#include <source_location>

void MyAssert(bool cond, std::source_location loc = std::source_location::current()) {
    std::cout << loc.file_name() << ':' << loc.line() << std::endl;
}
```

Тонкости: - Дефолтный аргумент `std::source_location::current()` **обязателен** — без него ничего не работает. Он вычисляется при каждом вызове **в контексте вызывающей функции**: «знает» файл, строку, имя функции (`file_name()`, `line()`, `column()`, `function_name()`). - Но функция всё равно вызывается в рантайме и стоит дорого, особенно когда проверки не нужны — тут макросы сильнее. **## Отключение проверок: NDEBUG**

Программы собирают в двух вариантах: `debug` — со всеми проверками, медленно, но всё проверяет; `release` — проверки отключены, надеемся, что `debug`-версия хорошо протестирована. Макросы позволяют выключать проверки на этапе компиляции:

```
#ifdef NDEBUG
#define assert(e) ((void)0)
#else
#define assert(e) /* настоящая проверка */
#endif
```

Если `NDEBUG` («not debug») определён — ассерт раскрывается в пустоту; если нет — это `debug`, проверяем. В реальном `<cassert>` написано чуть ли не лишнее — достаточно `#define assert(e) ; no ((void)0)` аккуратнее: корректно съедает точку с запятой в любом контексте.

Вывод: с макросами опасно — нужно хорошо понимать, в каких контекстах они будут раскрываться.

Предопределённые макросы и оператор

Предопределённые макросы: `__LINE__` (сдвинули код — значение поменялось), `__FILE__` (полный путь), `__func__` и нестандартный `__PRETTY_FUNCTION__`, который печатает функцию красивее, с аргументами.

Имена, начинающиеся с `_X` или `__x`, использовать нельзя — они зарезервированы за реализацией (undefined behavior по стандарту); clang-tidy сразу ругается «Invalid case style for variable `_X`».

Оператор `#` превращает аргумент макроса в строку — **текстовое написание выражения как его написал программист**:

```
#define ENSURE_MSG(cond, msg) \
    if (!(cond)) { \
        throw std::runtime_error{"Assertion " #cond " failed: " msg}; \
    }
```

Для `ENSURE_MSG(5 == 2 * 2, ...)` получится `"5 == 2 * 2"` — так проще дебажить. Механика: препроцессор подставляет вместо `e` переданный текст, а `#e` оборачивает его в кавычки; препроцессор знает только байтики — если через `#` передать уже строку, вернётся строка в кавычках, а не «нижележащее значение».

Ещё тонкость языка: **подряд идущие строковые литералы без операторов конкатенируются в один** — поэтому `"Assertion " #cond " failed: " msg` работает без сложения строк и копирования памяти.

Макросы с переменным числом аргументов и выбор по количеству

У них в кодовой базе набор макросов: `ensure` кидают исключение, `verify` — убивают программу (abort); причём макрос **перегружается по количеству аргументов** — с условием и с условием+сообщением. Вариадик-макросы: пишем `...`, внутрь передаётся произвольное число аргументов, а `__VA_ARGS__` даёт их список через запятую.

Практическое правило: внутри макросов оборачивайте в скобочки все выражения, переданные снаружи — иначе приоритеты операторов поедут и всё сломается: вы не знаете, какие операторы использовал вызывающий.

Выбор между `ENSURE_1` и `ENSURE_2` — «страшный» хак:

```
#define ENSURE_2(cond, msg) \
    if (!(cond)) { \
        throw std::runtime_error{"Assertion " #cond " failed: " msg}; \
    }

#define SELECT_THIRD(a, b, c, ...) c

#define ENSURE(...) SELECT_THIRD(__VA_ARGS__, ENSURE_2, ENSURE_1)(__VA_ARGS__)

int main() {
    ENSURE(1 == 2);
    ENSURE(1 == 2, "uh oh");
}
```

`SELECT_THIRD` принимает минимум три аргумента и всегда возвращает **третий**. Если `__VA_ARGS__` — один аргумент, третьим «встанет» `ENSURE_1`; если два — `ENSURE_2`. Выбранное имя затем вызывается с тем же `__VA_ARGS__` (при двух аргументах это просто оба через запятую). Определить второй `ENSURE` без `#undef` нельзя — `'ENSURE' macro redefined`.

do { } while(false) и тестовые фреймворки

Макросы массово используются в тестировании — тесты в задачах курса на Catch2 пестрят макросами:

```
#define REQUIRE(cond) \
    do { \
        if (!(cond)) { /* красивый report failure */ } \
    } while (false)
```

Зачем `do { } while(false)`: создаёт **настоящий скоуп** и **требует точку с запятой в конце** — фигурные скобки можно, но `do/while(false)` лучше: макрос нельзя случайно вставить, например, как тело `if` без скобок.

X-макросы: enum → строка

Самое красивое использование макросов. Задача: превратить enum в строку. В лоб — switch с `case Color::Red: return "Red";` для каждого значения.

Почему этот switch «грустный»: добавив новое значение enum, нужно не забыть поправить все такие switch по всему коду. Можно не писать `default` и пусть компилятор ругается — но это надо правильно настраивать в сборке. В C++26 (или позже) это сможет встроенная рефлексия, но до неё далеко.

Правильный, по мнению лектора, вариант — **кодогенерация**: питончик, который распарсит плюсы, найдёт enum-ы и сгенерирует функции `toString` и итерацию по значениям. **Как ни странно, это самый хороший, проверенный, рабочий вариант**; минус — нужно писать и запускать сам генератор.

А чисто на макросах — более вербозно:

```
#define FOR_EACH_COLOR(XX) \
    XX(Red) \
    XX(Green) \
    XX(Blue)

enum Color {
#define POPULATE_ENUM(value) value,
    FOR_EACH_COLOR(POPULATE_ENUM)
#undef POPULATE_ENUM
};

std::string ToString(Color color) {
    switch (color) {
#define POPULATE_SWITCH(value) case value: return #value;
        FOR_EACH_COLOR(POPULATE_SWITCH)
#undef POPULATE_SWITCH
        default: ;
    }
}
```

Макрос `FOR_EACH_COLOR` принимает в качестве аргумента другой макрос — как callback: вместо `XX` подставляется переданный макрос и вызывается со всеми значениями. Теперь `Yellow` добавляется в одно место — enum и switch расширятся автоматически.

Тонкости: - `#undef` обязателен по стилю: вспомогательный макрос создаётся по месту, используется и сразу удаляется, чтобы не загрязнять глобальную область видимости. - Нельзя просто `return #color;` — `#` вернёт текст написания аргумента (`color`), а не имя значения; приходится костылить через switch. - `XX` — просто placeholder; можно писать что угодно.

X-макросы очень популярны в реальном коде — например, в ClickHouse их масса для описания настроек и enum-ов. Второй пример из практики — ref-qualifiers: специализация под **шесть** комбинаций ref-qualifier'ов, которую раньше копировали шесть раз (пример из ydb, слайд):

```
#define Y_FOR_EACH_REF_QUALIFIERS_COMBINATION(XX) \
    XX(/* пусто */) XX(&) XX(&&) \
    XX(const) XX(const&) XX(const&&)

#define Y_DECLARE_REMOVE_CLASS_IMPL(qualifiers) \
    template <typename C, typename R, typename... Args> \
    struct TRemoveClassImpl<R (C::*)(Args...) qualifiers> { \
        typedef R TSignature(Args...); \
    };

Y_FOR_EACH_REF_QUALIFIERS_COMBINATION(Y_DECLARE_REMOVE_CLASS_IMPL)
#undef Y_DECLARE_REMOVE_CLASS_IMPL
// github.com/ydb-platform/ydb, util/generic/function.h
```

Макрос — очень мощный язык: из обычной текстовой подстановки получается «функция высшего порядка», где макрос передаётся как аргумент-колбэк.

Практическое правило по частоте: 99% реального кода — бизнес-логика без своих макросов; но использование готовых макросов встречается постоянно (тестирование, проверки), а свои вы пишете нечасто — только когда начинается нетривиальный код.

Шаблоны: циклы и if на этапе компиляции

В нулевые люди обнаружили, что шаблоны, как и макросы, — мощный язык: на них можно писать циклы и if.

Цикл — факториал через рекурсивную инстанциацию:

```
template <int N>
struct Factorial {
    static const int value = N * Factorial<N - 1>::value;
};

template <>
struct Factorial<0> {
    static const int value = 1;
};
```

Индукция: `Factorial<N> = N * Factorial<N-1>`, а случай `N == 0` — **специализация** (общий шаблон для всех аргументов, но для аргумента 0 — другой код). Компилятор инстанцирует `Factorial<8>...<0>` и перемножает — это и есть «цикл». Результат — константа времени компиляции: годится везде, где ожидается константа, даже в шаблонных аргументах; можно считать и нетривиальные вещи вроде квадратного корня.

Главная проблема — время сборки. История: cpp-файл компилировался 15 минут, потому что там инстанцировалось 64 варианта очень тяжёлой функции поиска для разных режимов работы.

If на типах:

```
template <bool Condition, typename TrueType, typename FalseType>
struct If; // только объявление, без определения

template <typename TrueType, typename FalseType>
struct If<true, TrueType, FalseType> { using type = TrueType; };

template <typename TrueType, typename FalseType>
struct If<false, TrueType, FalseType> { using type = FalseType; };
```

Объявление без определения и две **частичные специализации** (подставлена только часть аргументов).

`If<cond, T, F>::type` сам выбирает ветку на этапе компиляции; условие — такое же константное значение, как в факториале. Исторический контекст: когда поняли, что на шаблонах можно всё, случился **бум метапрограммирования** — безумные библиотеки, где вся логика уносилась в шаблоны и безумно долго компилировалась; отчасти поэтому плюсы до сих пор не любят и боятся. **Сейчас так обычно не пишут, но уметь читать и понимать нужно.** В домашке будет пара задач в такой парадигме; особенно весело писать бинпоиск, стараясь **не инстанцировать лишние шаблоны** и аккуратно отсекал ветки: у компилятора есть лимит на число инстанциаций, и его легко пробить.

SFINAE: проблема выбора перегрузки

Классическая задача — массив с двумя конструкторами:

```
template <typename T>
class Vector {
public:
    Vector(size_t n, const T& value); // n копий value
    template <typename Iterator>
```

```
Vector(Iterator begin, Iterator end); // диапазон итераторов
};
```

Проблема: для массива `int` -ов вызов `Vector<int> v(10, 20);` вызовет конструктор от двух итераторов! При вызове функции компилятор сначала строит **overload set** — множество всех потенциально подходящих функций — и выбирает наиболее подходящую. Итераторный конструктор гибкий: принимает всё, что угодно, двух одинаковых типов; а первый требует каста `int` к `size_t`. Нужно **исключить его из overload set** при определённых условиях.

Концепты это умеют (`requires`), но они появились недавно, а до них писали иначе — и это свойство более фундаментальное, чем концепты. Решение называется **SFINAE** — **Substitution Failure Is Not An Error**, «ошибка подстановки не является ошибкой»:

При подстановке аргументов в шаблон функции, если подстановка даёт невалидный заголовок функции, функция просто не кладётся в **overload set** — а не порождает ошибку компиляции. Странное поведение компилятора, но именно оно позволяет гибко выключать функции из выбора перегрузки.

К итераторному конструктору добавляют второй фиктивный дефолтный параметр:

```
template <bool Condition, typename T>
struct enable_if; // объявление без определения

template <typename T>
struct enable_if<true, T> { using type = T; };

template <typename T>
struct enable_if<false, T> {}; // пустая структура, члена type нет
```

Как работает: если условие `true` — есть `type`, равный второму аргументу; если `false` — структура пустая, и `::type` даёт **ошибку компиляции**. Но ошибка случилась при подстановке в заголовок функции — по SFINAE функция **выкидывается из overload set**, и компилятор рассматривает остальные; если весь **overload set** пуст — тогда уже настоящая ошибка. Теперь `Vector<int> v(10, 20);` вызывает конструктор от `(size_t, const T&)`, потому что итераторный не подошёл.

Куда писать `enable_if`: три способа

`enable_if` можно вставлять в несколько разных мест, у каждого свои плюсы и минусы:

1. **В шаблонный параметр** (как выше, `typename = std::enable_if_t<...>`). Минус: пользователь может явно задать этот параметр и всё сломать.
2. **В тип параметра** — например, параметр `void* = nullptr`. Тоже делают, минусы свои.
3. **В возвращаемое значение** — **самый аккуратный способ**: возвращаем не `T`, а `typename std::enable_if<cond, T>::type`. Но самый громоздкий: функция страшно выглядит, непонятно, что возвращает, пока не прочитал внимательно.

Лучше выбирать один из двух: возвращаемое значение или дефолтный шаблонный аргумент (в типе аргумента по умолчанию `void`, а `void`-аргумент сделать нельзя — приходится брать указатель на него с дефолтом `nullptr`, что некрасиво).## Важный тонкий нюанс: SFINAE и методы класса

Очень тонкий момент (с концептами работает так же). Хочется добавить метод `unref`, только если `T` — не `void` (ссылка на `void` не бывает):

```
template <typename T>
class UniquePtr {
public:
    // ТАК НЕ КОМПИЛИРУЕТСЯ:
    std::enable_if_t<!std::is_void_v<T>, T&> unref();
};
```

Так не работает: `T` здесь — не шаблонный параметр данной функции, он уже известен при инстанцииции самого класса. Правило для запоминания:

В SFINAE/ `enable_if` используйте только те аргументы, которые шаблоны для данной конкретной функции, а не определены где-то выше (в шаблоне класса).

Проблема: при инстанцииции `UniquePtr<void>` компилятор сразу пытается понять, какие методы есть у класса, подставляет `T` во все заголовки — и всё ломается; clang даже подсказывает: «`enable_if` cannot be used to disable the declaration».

Решение: добавить неизвестности — ещё один шаблонный параметр, равный исходному, и вешать SFINAE на него:

```
template <typename T>
class UniquePtr {
    template <typename U = T,
              typename = std::enable_if_t<!std::is_void_v<U>>>
    U& unref();
};
```

С концептами та же история: `requires` после объявления метода не поможет — компилятору, чтобы разобрать функцию, всё равно нужен `T`; правильно опять же завести свой шаблонный параметр метода. Громоздко, но иначе никак.

Концепты и `requires`

Из хорошего: в новых плюсах проще. Почти всегда вместо SFINAE сейчас нужно использовать концепты: пишете `requires` с условием — и под капотом происходит то же, что с `enable_if`: функция исключается из overload set. Плюс можно писать свои концепты — например, проверка наличия метода

```
size: cpp template <typename T> concept HasSize = requires(T t) { { t.size() } ->
    std::convertible_to<std::size_t>; };
```

Разбор: - `requires` бывает «двойной»: `requires(...) { ... }` — параметры нужны, чтобы объявить намерения: просим **фейковый экземпляр переменной**; этот код не исполняется, но на нём можно дёргать методы и писать условия. - `{ t.size() } -> std::convertible_to<std::size_t>` проверяет, что `size()` возвращает `size_t` или тип, преобразуемый в него.

Использовать можно по-разному: `requires` -выражение в объявлении шаблона; даже **прямо по месту в коде** — «самая страшная суть», когда прямо при написании кода проверяется ожидание. Работает, люди так уже пишут, но **так делать не надо — некрасиво**. Внутри `constexpr if` проверка `requires { t.size(); }` работает отлично: если `size` нет, ошибки компиляции не будет, выберется другая ветка; с обычным `if` так нельзя — условие посчитается в рантайме и всё сломается.

`constexpr`-функции

Наконец, с C++11 есть `constexpr` — функции, вычисляемые на этапе компиляции, — **и жизнь стала сильно проще**. Эволюция: - C++11: `constexpr`-функция должна была состоять из одного `return` -а — люди пытались запихнуть всё в одно выражение. - C++14: требования ослабили — можно **циклы**; разрешено почти всё, кроме условно `goto` и `try`. - C++17/20: можно использовать `std::vector` и `std::string` в `constexpr`-функциях — выделять память; главное — освободить её до выхода из функции.

Сейчас на этапе компиляции можно переносить довольно сложные вычисления. Нужно ли это и насколько популярно — открытый вопрос, зависит от кода. Полезно скорее для **библиотечного/общего кода**: предкомпилировать что-то, выбрать одну из нескольких реализаций функции. В домашке есть упражнение — «довольно забавно».

Кратко

- **Метапрограммирование** — программа, порождающая другую программу; в C++ это макросы, шаблоны, `constexpr` (метаклассы — далёкое будущее, C++29+).
- **Макросы — чисто текстовая замена до компиляции**: отсюда возможности и подводные камни (двойной вызов `heavy_func()`, поехавшие приоритеты — оборачивайте аргументы в скобки).

- **assert** — макрос, а не функция: функция не знает место вызова; это умеют `__LINE__` / `__FILE__` и `std::source_location::current()` (C++20) как **дефолтный аргумент, вычисляемый в контексте вызывающего**.
- Проверки отключаются через `#ifndef NDEBUG`, а `do { } while(false)` в макросах даёт скоуп и требует `;`.
- Оператор `#` делает строку из текста аргумента; соседние строковые литералы конкатенируются; **выбор макроса по числу аргументов** — трюк с `SELECT_THIRD(__VA_ARGS__, ENSURE_2, ENSURE_1)`.
- **X-макросы** — макрос, принимающий макрос-колбэк: `enum`, `toString`, 6 специализаций `ref-qualifier`’ов описываются в одном месте; вспомогательные макросы сразу `#undef`. В индустрии это норма (ClickHouse, ydb); лучшая альтернатива — кодогенерация питоном.
- **Шаблоны — тоже язык**: рекурсивные инстанцииции = цикл (факториал), частичные специализации = `if`; цена — **время компиляции** и лимит на число инстанцииций, который легко пробить.
- **SFINAE: ошибка подстановки — не ошибка** — невалидный заголовок шаблонной функции исключает её из overload set; `enable_if` пишут либо в возвращаемый тип (аккуратнее), либо в дефолтный шаблонный параметр.
- В SFINAE/концептах для метода класса можно использовать только шаблонные параметры самой функции: `T` из шаблона класса известен при инстанцииции класса; лечится фиктивным параметром `typename U = T`.
- В новом коде используйте концепты и `requires`; `constexpr`-функции с C++20 умеют даже `std::vector` и `std::string`.

Оргмоменты

Лектор напомнил про домашки: пара задач на метапрограммирование в «шаблонной парадигме» (например, бинарный поиск с аккуратным отсечением инстанцииций) и упражнение с `constexpr`-вычислениями. Также стоит посмотреть, как устроены макросы в тестовом фреймворке Catch2, который используется в задачах курса.

Лекция 08 — Многопоточность

Зачем вообще многопоточность: короткий экскурс в устройство процессора

Лектор начинает с Intel 8086 — одного из первых успешных массовых процессоров. У него **одно ядро**: на кристалле — вычислительное ядро и вспомогательная логика (подсистема работы с памятью и т.п.), а размер платы определялся количеством ножек. Процессор общается с памятью по шине.

Много лет люди так и жили в парадигме «одно ядро». Потом возник **дисбаланс**: ядра ускорялись (росли частоты, полезная работа за такт), а скорость доступа в память — почти нет. Любопытно: ~30 лет назад память была «дешёвой» относительно процессора — доступ занимал примерно те же ~70–100 нс, что и сейчас, а процессор был медленным. Поэтому тогдашние программисты предпочитали **таблички в памяти** вычислениям на лету. Когда процессоры обогнали память, всё перевернулось: теперь память — главный ограничитель производительности.

Правило: произвольное чтение из памяти — это ~70 нс. При частоте 4 ГГц один такт — ~1.25 нс, сложение занимает единицы тактов. Значит, одно обращение к памяти «стоит» ~100 тактов: процессор в это время просто стоит и ждёт ответа.

Решение — **кэш**: маленькая быстрая дорогая память рядом с ядром, поэтому его мало. Свет за секунду проходит 30 см в вакууме — расстояния до памяти сравнимы с размерами машины, и чем ближе память к ядру, тем быстрее (дело не только в скорости света, но и в протоколах, адресации и других физических эффектах). Когда кэша перестало хватать, появились **уровни**: L1 (самый быстрый и маленький, рядом с исполнительными блоками), L2 (побольше), L3 (самый большой и медленный). Сейчас обычно 3 уровня кэша.

Многоядерность и гиперторейдинг

На заре 2000-х в обычных ПК появились **многопроцессорные системы** — два процессора на одной материнской плате (сначала это делали энтузиасты «подпольно», потом производители поняли, что подход правильный, и начали штамповать массово). Но у двух отдельных процессоров две стрелки в память,

дорогая межпроцессорная коммуникация, разные кэши. Поэтому примерно в 2005 году появились **многоядерные процессоры**: несколько ядер внутри одного корпуса, супербыстрое общение между собой, общий доступ к памяти и, возможно, общие кэши.

Типичная раскладка кэшей: у каждого ядра свой маленький L1 и свой L2, а L3 — **общий**. На дизшоте AMD Ryzen 5950 (Zen 3) видно: 8 одинаковых ядер занимают лишь ~30% площади кристалла, остальное — гигантский L3. Чтение из памяти практически всегда идёт через кэш; есть инструкции, читающие мимо кэша (для данных, которые больше не понадобятся), но так никто не делает.

Зачем всё это? Вся многопоточность нужна исключительно ради производительности — точнее, ради сокращения времени работы программы: одного ядра перестало хватать.

Дальше — **гиперторейдинг (hyperthreading)**. Процессор почти всегда простаивает в ожидании памяти (кроме бенчмарков). Идея: на одном физическом ядре запустить **два виртуальных ядра** — два независимых набора регистров, два независимых вычисления; пока первое ждёт память, выполняется логика второго. Тонкости:

- Если вы просто складываете числа, которые всё в кэше, — профит гиперторейдинга сходит на нет: виртуальные ядра делят одни и те же вычислительные блоки. У 5950 (8 физических / 16 виртуальных ядер) можно выполнить ~32 инструкции за такт, но не 64.
- Переключение между виртуальными ядрами делается **на уровне железа** и почти неотличимо от физических ядер с точки зрения ОС.
- Делать 32 виртуальных ядра дорого с точки зрения железа, поэтому ограничиваются удвоением.

Серверные процессоры — это уже 96 ядер (48 физических / 96 виртуальных), а по несколько таких в системе — до 384 ядер и ~1.5 ТБ RAM. Возникает **NUMA (Non-Uniform Memory Access)**: у каждого процессора своя «близкая» память, доступ к памяти соседнего дороже. Все сервера работают так, но на то, *как* вы пишете код, это влияет не сильно (нюансы всплывают при деплое).

Операционная система: вытесняющая многозадачность и переключение контекста

Многоядерные процессоры появились в ~2005, а привычный вид ОС сформировался в 90-х. Как жили на одном ядре 15 лет с кучей «параллельных» программ? Через **абстракцию**: ключевая задача ОС — спрятать от программ тот факт, что ядро одно. ОС исполняет процесс, потом переключается на другой, потом на третий. Переключение происходит, когда:

1. процесс сам решил, что ему хватит (пошёл читать с диска, в сеть, спать);
2. он слишком долго исполнялся — **квант времени** велик, в Linux по умолчанию ~100 мс; спасает то, что программы редко работают 100 мс подряд без обращения к системе.

Подход называется **вытесняющая многозадачность (preemptive multitasking)**: вы пишете код, не думая об этом, а ОС иногда вас вытесняет.

Что происходит при вытеснении? Ключевой факт: чтобы «запомнить, где находилось вычисление», достаточно сохранить **регистры** — несколько сотен байт. Память сохранять не нужно: если вычисление не происходит, память не меняется. Сохранение и восстановление регистров для другого потока — **переключение контекста (context switch)**.

Тонкость про миграцию потоков между ядрами: ОС **не любит перекидывать** поток с ядра на ядро — миграция дорогая, потому что теряются тёплые кэши. Важно: при миграции кэши **не сбрасываются** — кэши адресуются физической памятью (ячейка 1028 одна на весь компьютер), данные остаются валидными, просто прогревать их на новом ядре придётся заново.

Потоки в C++11

В C++11 потоки появились **прямо в стандарте**. До этого потоки были, но запускались средствами ОС и везде по-разному; C++11 унифицировал. Две составляющие:

1. механизм запуска потоков — `std::thread`;
2. примитивы синхронизации — `std::mutex`, condition variable, `std::atomic` и т.д.

В любой программе на старте уже есть **один поток** (тот, что исполняет `main`); остальные создаются поверх него.

Терминологическая ремарка: слово «поток» в C++ перегружено (есть поток ввода-вывода, `stringstream`, и есть `thread`). `thread` правильнее называть **нитью**, но все привыкли к «потoku» — из контекста понятно.

Чем поток отличается от fork? `fork` — механизм Unix-подобных систем для запуска соседних процессов (shell сделал `fork` и запустил `vim`). Потоки с точки зрения системы очень похожи на процессы, но ключевое отличие: у потоков **общая память** — они читают одну и ту же память и даже пишут в неё. У процессов общей памяти нет (можно придумать общий регион, но это регион, а не вся память). У потоков общая **вся** память: общая куча, общий аллокатор. (Бывают «продвинутые форки» — `vfork` / `clone`.)

Первый пример:

```
#include <iostream>
#include <thread>

int main() {
    std::thread t1{[] {
        std::cout << "Hello, ";
    }};
    std::thread t2{[] {
        std::cout << "threads!";
    }};
    t1.join();
    t2.join();
}
```

В конструктор `std::thread` можно передать лямбду (круглые скобки после захвата можно не писать), а можно функцию и её аргументы через запятую — как при обычном вызове; аргументы передадутся в поток.

Первый запуск без `join`'ов: программа **падает** (`libc++abi: terminating`, `SIGABRT`), иногда выводит `Hello, threads!`, иногда `threads!Hello,`, иногда ничего. Почему: **поток обязательно нужно либо `join`, либо `detach`** до конца его объекта.

- `join` — **останавливает текущий поток**, пока не завершится ожидаемый, и освобождает ресурсы, связанные с потоком (внутри ОС это сишная структурка со свойствами потока). Лектор сравнивает с `delete` указателя: `join` — про освобождение ресурсов, а не просто «дождаться».
- `detach` — мгновенно отвязывает поток: при завершении программы его не будут ждать. **Не надо** — неконтролируемо, обычно признак странного, непродуманного кода. **Правило: всегда делаем `join`.**
- Если поток уже завершился к моменту `join` — ничего страшного, `join` всё равно нужен.
- Завершение программы с незажойненными потоками — `terminate`, это ошибка; два `join` подряд — плохо.

Поток **стартует сразу** при создании — метода `run` нет. И `t1`, `t2` исполняются **по-настоящему параллельно** на разных ядрах: никто не скажет, в каком порядке выполнится вывод. Отсюда `Hello, threads!` в одном запуске и `threads!Hello,` в другом. Забавный эксперимент: если убрать `sleep`, почти всегда выводится `Hello, threads!` — `t1` создаётся чуть раньше (на один системный вызов меньше) и успевает напечатать первым.

Нарrens-before: формальный язык про порядок операций

Как рассуждать о порядке в многопоточной программе? Введено отношение **happens-before** — частичный порядок на операциях: про некоторые пары можно гарантированно сказать, что одна происходит после другой. Примеры:

- любые две операции **внутри одного потока** провязаны happens-before (поэтому в однопоточном коде «всё по порядку» и нет проблем);
- `join` — точка синхронизации: операции ожидаемого потока happen-before операции после `join`. Поэтому порядок двух `join` не важен — гарантии получены в любом случае.

Если операция A happens-before операции B, то **результат A виден до результата B**. Красивая визуализация: операции — вершины графа, happens-before — рёбра; где между двумя операциями нет пути — потенциальный race condition.

Race condition — две операции, не провязанные happens-before: нельзя объяснить/доказать, какая раньше, при этом видны результаты исполнения. Важное различие:

- **Race condition сам по себе — это ок** (defined behavior). Просто две параллельные операции, «какая раньше — как повезёт», — неопределённость порядка, но не UB. Лектор: «результат программы, который выводит имя машины, тоже зависит от машины — и это хорошее, определённое поведение».
- **Data race (гонка по данным) — Undefined Behavior**. Это когда разные потоки **меняют общие данные**, не синхронизированные happens-before.

Пример из лекции: `x = 1` happens-before запуска потоков (он случился до них — можно объяснить почему), а записи `x = 2` и `x = 3` в двух потоках между собой не связаны: это и race condition, и, поскольку запись, — **data race**, т.е. UB. На практике мусорное значение получить сложно: процессор делает запись одного значения **атомарно** (от греч. «неделимый») — либо прошла одна запись, либо другая. Но полагаться на это нельзя: описывать конкретное поведение UB — «очень плохое дело».

Проблема с cout из лекции: нельзя из разных потоков писать в `cout`. Внутри у него буфер, накапливающийся перед флашем в систему, — и этот буфер не защищён. Записи «одна не happens-before другой», и TSan ругается, даже если на экране всё выглядит прилично.

ThreadSanitizer — санитайзер, умеющий находить data race'ы

(`clang++ main.cpp -o main -fsanitize=thread`). Лектор называет его «одно из великих произведений человечества»: он находит гонки, которые не каждый программист отыщет вручную, — печатает `WARNING: ThreadSanitizer: data race` со стектрейсами обоих потоков. **Правило: проверяйте многопоточный код под TSan.**

jthread и joinable

Ручной `join` писать неудобно, руками потоки мало кто запускает, а когда запускают — в C++20 появился `std::jthread`: он делает `join` в **деструкторе**. Зачем что-то умное — деструктор уже есть.

Про двойные `join`'ы: бывают проблемы, поэтому есть метод `joinable()` — «позвали ли `join` уже на этом потоке или ещё нет»:

```
if (t.joinable()) {  
    t.join();  
}
```

Важно: `joinable()` **не проверяет, закончилось ли исполнение потока** — он проверяет только, позвали ли уже `join`. Даже если вы знаете, что поток уже вышел, вы всё равно должны позвать `join` — это про освобождение ресурсов.

Философия синхронизации: лучше не синхронизировать вообще

Лучший вариант — **ничего не синхронизировать**: чем больше потоков и взаимодействия между ними, тем менее эффективна программа. Самая эффективная программа с точки зрения утилизации железа — однопоточная. Парадокс разрешается так: если можно однопоточным кодом «добить» все ядра — это идеальная модель параллельности. Именно поэтому **компиляторы однопоточные**: чтобы собрать большой проект (типа браузера), нужны тысячи запусков компилятора, и количеством параллельных запусков управляет верхняя программа (`make`, `ninja`) — она держит столько компиляторов, сколько ядер. На нижнем уровне синхронизации нет, а она дорогая.

Правило: синхронизацию вытаскиваем **как можно выше**; коммуницируем между потоками, шарим и синхронизируем как можно меньше — пока всё корректно. Проблема в корректности — вот для чего нужны примитивы.

Mutex: взаимное исключение

Есть «две школы» защиты данных. Первая — более общая, но более медленная: **mutex** (mutual exclusion; в советских книгах — **критическая секция**: одно и то же, просто есть язык, доставшийся из Союза, а есть переведённый). Аналогия — телефонная кабинка: внутри один человек, остальные ждут.

У мьютекса есть `lock()` и `unlock()`. `lock` говорит: «вы вошли в критическую секцию, другой в неё войти не может»; второй `lock` в другом потоке будет ждать первого `unlock`. Внутри ОС образуется очередь (есть целая наука, в каком порядке пускать потоки после `unlock`). Ключевая семантика: **между `lock` и `unlock` возникает happens-before** — внутри критической секции можно синхронизировать произвольные действия, и UB исчезает.

Cache line: почему наивный мьютекс тормозит

Демонстрация: 4 потока по миллиарду инкрементов общей суммы под мьютексом — «4 миллиона операций в секунду на 4 потоках», т.е. **на порядок медленнее** однопоточного варианта. Лектор отказывается угадывать: надо «расчехлить профилировщик» — профилировать под Linux (`perf`/перфоратор), инструмент скажет, где тратятся ресурсы. Почти наверняка тормозит мьютекс, но есть ещё нюанс — **cache line**:

- Процессор работает с памятью **кэш-линиями по 64 байта**: прочитав 1 байт, он читает 64 и кладёт их в кэш. Поэтому прочитать 1, 4 или 8 байт одинаково дёшево (если не пересекаем границу кэш-линии).
- Когда вы что-то пишете, вы читаете кэш-линию, меняете, пишете обратно — и должны **инвалидировать** закэшированную копию линии во всех остальных кэшах. Кэши у ядер разные, но кэшируют одну и ту же память: если переменная лежит в четырёх кэшах и одно ядро её поменяло — сбрасывать надо во всех остальных.
- Протокол синхронизации кэшей сложный (подробности — у Ромы Липовского) и дорогой. Когда несколько потоков параллельно инкрементируют одну переменную, всё деградирует до того, что **каждый раз идём прямо в память** — кэши не работают. В однопоточном случае это замаскировано; в многопоточном — всё плохо.

Кстати: кэш хранит значения **по ключу-адресу** (не путать с **TLB** — Translation Lookaside Buffer, кэшем трансляции адресов).

Решение — не шарить кэш-линии между потоками. Кладём счётчики в разные кэш-линии:

```
struct alignas(64) Counter {
    long long value;
};
```

`alignas(64)` означает, что структура требует выравнивания 64 байта — потому что кэш-линия 64 байта.

Убери `alignas` — код останется **корректным**, но медленным: линия постоянно инвалидируется.

«Формально корректный, но медленный» код из-за кэш-линий — большая тема домашних заданий:

помнить про кэш-линии. (Инкремент относительно цикла — мелкая операция, поэтому эффект на демо виден не так сильно, как в реальных задачах, но он очень большой.)

Атомарность операций и почему `x++` ломается, а `x = 2` — нет

Почему в примере с присваиваниями мы не видели мусорных значений, а с инкрементами суммы получаем полную ерунду (ожидаем 4 миллиарда, получаем ~1)? На уровне процессора операции разные:

- запись одного значения в память атомарна** — выполняется целиком, нельзя выполнить половину;
- инкремент — не атомарен**: сначала прочитать, потом изменить, потом записать обратно. Два потока читают одно значение, оба инкрементируют, оба пишут — одно прибавление теряется.

Оба варианта — data race и UB, но проявляются по-разному из-за атомарности записи на уровне железа.

Кратко

- Память — главный ограничитель производительности: доступ ~70 нс против ~1.25 нс на такт; поэтому кэши L1/L2 (у ядра) и общий L3.
- Гиперторейдинг: 2 виртуальных ядра на физическом помогают только при ожидании памяти.
- ОС даёт вытесняющую многозадачность; переключение контекста — сохранить/восстановить регистры (сотни байт); миграция потока между ядрами дорога. Большие серверы — NUMA.
- В C++11 появились `std::thread` и примитивы синхронизации; поток стартует сразу и его нужно либо `join`, либо `detach` — **всегда делаем `join`**, `detach` не используем.
- `join` — про освобождение ресурсов ОС (как `delete`), а не только «дождаться»; `joinable()` проверяет лишь, позвали ли уже `join`. C++20: `jthread` делает `join` в деструкторе.

- **Happens-before** — частичный порядок операций. Отсутствие связи = race condition (это ок); несинхронизированная запись = **data race** = UB. Проверяйте код ThreadSanitizer'ом.
- Синхронизация дорогая: чем меньше коммуникации и общих данных между потоками, тем быстрее; идеал — синхронизация «наверху» (как make/ninja вокруг однопоточных компиляторов).
- `std::mutex` даёт happens-before между lock/unlock, но медленный; инкремент не атомарен (чтение+изменение+запись), запись одного значения — атомарна.
- Процессор работает кэш-линиями по 64 байта; параллельная запись в одну линию — инвалидации и работа напрямую с памятью. Разносите счётчики разных потоков по разным линиям (`alignas(64)`).

Оргвопросы

Тема безумно обширная и не помещается даже в целый курс: для глубины «от нижнего уровня до верхнего» (протоколы синхронизации кэшей и т.п.) лектор рекомендует курс Ромы Липовского про конкурентность (точно есть в Шаде). Ближайшая домашка и следующий проект выкладываются в тот же день. Профилировать — под Linux (perf/перфоратор), не под macOS.

Лекция 09 — Condition Variable

Семафор как мотивация

Сегодня продолжаем говорить про многопоточность: тема — condition variable и то, как с её помощью строятся сложные примитивы, на которых пишется «почти весь» многопоточный код. С одним mutex'ом это делать неудобно, поэтому в качестве упражнения пишем **семафор**.

Семафор — примитив, очень похожий на mutex, но с одним принципиальным отличием: он позволяет находиться в критической секции **нескольким потокам одновременно** — не больше, чем заданное число `n`. Реальная задача, где такая семантика нужна — **rate limiting**: вы пользуетесь внешним сервисом и знаете по соглашению, что ему нельзя слать больше, чем, скажем, 4 параллельных запроса. Каждый поток перед запросом делает `Enter`, делает запрос, затем `Leave` — и в любой момент времени в сервис «вылезает» не больше 4 запросов.

Через семафор делается и mutex: достаточно записать в `n` единицу. В стандарте семафор есть с C++20 — `std::counting_semaphore`, который делает ровно это. Но давайте напишем его сами.

Первая версия: поля и Leave

Заведём mutex, который защищает счётчик. `Enter` и `Leave` вызываются из разных потоков, поэтому просто так менять `counter_` нельзя. Счётчик — это количество доступных «токенов»: сколько ещё потоков может зайти.

```
class Semaphore {
    std::mutex mutex_;
    int counter_ = 0;
};

void Semaphore::Leave() {
    std::unique_lock<std::mutex> guard(mutex_);
    counter_++;
}
```

Для гарантии, что mutex всегда корректно разлочивается, используется RAII-обёртка. Практическое правило лектора: **не забывайте давать имя guard-объекту**. Конструкция

```
std::lock_guard<std::mutex>(mutex_);
```

создаёт *временный объект без имени*, который разрушается на первой же точке с запятой — mutex тут же разлочится, и защиты нет. Это очень частая ошибка: у себя в кодовой базе авторы однажды нашли ~30 вхождений кода, «защищённого» именно так. Пишите `std::lock_guard guard{mtx_};` — с именем.

`std::lock_guard` — максимально простая обёртка: в конструкторе `lock()`, в деструкторе `unlock()`, никаких методов — реализация помещается на экран целиком. `std::unique_lock` сложнее: умеет перемещаться, его можно разлочить (`unlock()`) и залочить обратно; хранит указатель на `mutex` и булев флаг «владеем ли» (в заголовке `libc++` — `mutex_type* __m`; `bool __owns`; , конструкторы с `defer_lock_t` / `try_to_lock_t`, методы `lock`, `try_lock`, `unlock`, `swap`, `release`). **Правило: если хватает `lock_guard` — используйте его, он проще.** `unique_lock` нужен там, где нужны его возможности (как раз в `cv.wait`).

Наивный Enter: mutex + sleep

Пишем `Enter`: лочим `mutex`, проверяем счётчик; если токен есть — забираем и выходим. А если нет? Ждать. Как? `Sleep`. А сколько спать? «Пока не проснёшься». А кто разбудит? Непонятно — нужен «будильник». Попробуем наивно:

```
void Enter() {
    for (;;) {
        std::lock_guard guard{mtx_};
        if (counter_ > 0) {
            counter_--;
            return;
        }
        std::this_thread::sleep_for(std::chrono::milliseconds(2));
    }
}
```

Первая попытка в процессе написания была ещё хуже: проверка `counter_` делалась до захвата `mutex`'а — формально это race condition, а конкретнее **data race** (незащищённый доступ к `counter_` из разных потоков), то есть undefined behavior. Race condition сам по себе «окей» как термин, data race — это UB. Вторая ошибка — попытка залочить уже залоченный этим же потоком `mutex` внутри цикла: это deadlock «в одном потоке»; `mutex` по умолчанию так не умеет (есть рекурсивный `mutex`, но лектор называет его странной вещью, которую никогда использовать не надо).

Код со `sleep`'ом корректен, но плох. Почему:

1. **Сжигаем CPU впустую.** Каждый ждущий поток раз в 2 мс просыпается, лочит `mutex`, проверяет, засыпает. Если один поток держит семафор минуту-две, остальные всё это время «греют воздух»; сегодня основной ограничивающий ресурс вычислений — электроэнергия.
2. **Задержка огромная.** 2 мс — это время ответа хорошо оптимизированного простого сервиса по сети; за 2 мс можно сложить несколько миллионов чисел.
3. **Магическая константа.** Откуда 2 мс? «Гиперпараметры» — любой такой параметр в синхронизации — плохой знак.
4. **Спать «чуть-чуть» система не даёт.** Вместо 2 мс получите скорее всего больше: на Windows минимум порядка 16 мс; сильно зависит от системы и версии. Единицы миллисекунд поспать часто нельзя вообще.

std::atomic: что умеет и чем не помогает

Возникает идея: читать `counter_` без data race. Для этого есть `std::atomic` — альтернативный механизм синхронизации. `Mutex` умеет синхронизировать «весь код», `atomic` — только `int` и простые типы, похожие на `int`. `Atomic` позволяет атомарно, неделимо, менять счётчик: есть методы `load`, `store` и куча других (в подсказке IDE: `wait`, `is_lock_free`, `compare_exchange_strong/weak`, `exchange`, `fetch_add`, `fetch_and/or/sub/xor`, `load`, `notify_all/notify_one` — у всего параметр `memory_order` по умолчанию `seq_cst`). Это работает быстрее `mutex`'а, потому что процессор умеет эффективно делать атомарные операции для чисел без data race.

Но здесь `atomic` **не решает главную проблему**: ждать всё равно приходится циклом, тратя CPU — на атомарных переменных нельзя спать. (У `std::atomic` есть метод `wait`, но лектор просит «забудьте про него пока».)

Отдельная тонкость: атомарность работает **только в рамках одной переменной**. В `mutex`'е можно поменять две переменные рядом и гарантировать «либо обе, либо ни одна» — с `atomic` так нельзя. `Mutex` — более сильная, но более дорогая конструкция. И ещё: мысленный приём для поиска гонок — представьте себя шедулером ОС, который исполняет потоки по одной инструкции в любом порядке, и попробуйте

сломать программу в этой модели. Так сразу видно исполнение, где поток проверил `isFree == true`, заснул, второй поток прошёл то же самое, и оба «вошли» в критическую секцию.

Busy loop: exchange, fetch_add и livelock

Если мы готовы тратить CPU, но хотим обойтись без mutex — пробуем чисто на атомарных операциях:

```
class Semaphore {
    // ...
    std::atomic<bool> is_free_{true};
    std::mutex mtx_; // ...
};

void Enter() {
    while (!is_free_.exchange(false)) {
        // busy loop
    }
}
```

`exchange` — неделимая операция: читает значение, возвращает его и записывает новое. Это сработало бы для `mutex`'а (токен всегда один), но для общего семафора нужен счётчик. На атомарных типах есть операторы типа инкремента, но под капотом они зовут методы — **полезно всегда писать метод явно**: инкремент — это `fetch_add`, «атомарно добавляет и возвращает предыдущее значение».

```
void Enter() {
    while (counter_.fetch_sub(1) <= 0) {
        // ...
    }
}

void Leave() {
    counter_.fetch_add(1);
}
```

Здесь появляется новая проблема — **livelock** (не **deadlock**!): у систем есть свойства того, как они «продвигаются», и здесь ломается *liveness* — свойство, что система делает что-то полезное. Пример: счётчик маленький, а потоков тысяча; все сделали `fetch_sub`, счётчик стал сильно отрицательным, и теперь каждый поток по одному `fetch_add` 'у гоняет единичку по кругу — «единица спряталась» между двумя методами. Формально все потоки имитируют бурную деятельность, но суммарного прогресса нет. В happy path штука работает, но довести систему до такого состояния можно, и это плохо.

Решение на CAS — `compare_exchange_weak/strong`. Это «страшная» конструкция, посмотрим внимательно:

```
void Enter() {
    int guess = counter_.load();
    do {
        while (counter_.compare_exchange_weak(guess, guess - 1) <= 0) {
            // busy loop
        }
    } while (/* ... */);
}
```

Логика CAS: три сущности — сама атомарная переменная, «угаданное» значение (`prev` / `guess`, передаётся **по ссылке**) и новое значение (по значению). Если угадали (в переменной лежало то же, что в `guess`) — в переменную записывается новое значение, возвращается `true`. Если нет — возвращается `false`, а в `guess` записывается **актуальное значение переменной**, чтобы помочь угадать на следующей итерации.

Про `weak` и `strong`: одинаковые методы с чуть разными гарантиями. Правило: **в цикле — weak** (эффективнее, но иногда фейлится сам по себе, поэтому нужен ретрай), **вне цикла — strong**. Сравнение — всегда на равенство. Уточнение к коду: раз новое значение пишется только при успехе,

аккуратнее сначала загрузить актуальное значение и лишь потом делать compare-exchange (первый набросок тратил лишнее).

Что получили: liveness-проблему решили, но гарантия тонкая — **каждый конкретный поток** гарантии завершения не получает (другие успевают перезаписывать счётчик), но из всех конкурирующих потоков кто-то один прогресс делает. Классификация многопоточных алгоритмов:

- **lock-free** — формально блокировок нет; системно прогресс делает хотя бы один поток;
- **wait-free** — каждый поток завершается за ограниченное количество инструкций. Самые лучшие (эффективные и понятные) алгоритмы, но написать хороший wait-free очень сложно, и не всегда возможно; здесь, скорее всего, нельзя.

Углубляться в это стоит на курсе Ромы Липовского. Но главное: **вторую проблему — сжигание CPU — CAS не решает**. Это по-прежнему busy loop.

Практическое правило про busy loop

Никогда не пишите busy loop в обычном коде. Исключения: embedded-системы; общение с GPU — там допустимо, потому что ресурс дороже процессорного времени (время видеокарты): вы прямо ждёте, пока карта запишет что-то в память.

И важно: «как разбудить поток хорошо» — люди в целом не придумали; идеального механизма «установки» потоков нет. Есть mutex и есть удобный механизм ожидания по условию, к которому мы идём.

Магическая функция, которую хочется

Вернёмся к версии с mutex. Мы лочим mutex, проверяем счётчик; если токена нет — хочется сказать компилятору: «**разбуди меня, когда счётчик станет больше нуля**». Заметим, что счётчик защищён mutex'ом, и это не следует ни из типов, ни из значений — только из кода. Значит, «магической» функции нужно передать и mutex, чтобы предикат проверялся под ним. И ещё деталь: **верни мне управление с заложенным mutex** — если предикат выполнен, не разлочивайте его, я сам разлочу. Итого сигнатура мечты:

```
SleepUntil(mtx_, [this]() -> bool {  
    return counter_ > 0;  
});
```

Тогда Enter превращается в компактный корректный код. Почему нельзя, например, сначала вызвать функцию «проверь под mutex и верни bool», а ждать потом? Потому что после выхода из функции мы mutex'ом **не владеем**, и counter_ мог поменяться — проверка и ожидание разъезжаются. Переход из состояния «я сам проверяю предикат» в состояние «меня надо будить» должен быть атомарным относительно предиката.

std::condition_variable

Стандартной функции SleepUntil нет, но есть очень похожая конструкция — **std::condition_variable**. Лектор шутит, что одна из самых сложных задач в многопоточном программировании — правильно называть mutex'ы и conditional variable; с mutex'ом непонятно (предлагают просто mutex / mtx), а CV — за условие/событие: у нас это not_empty_ (предикат «есть токен»).

Интерфейс (автодополнение — главный способ писать код на C++): native_handle (для low-level, неинтересно) и пять содержательных методов: notify_all, notify_one, wait, wait_for, wait_until. Сигнатура нужного wait похожа на мечту:

```
template <class Predicate>  
void wait(std::unique_lock<std::mutex>& lock, Predicate pred);
```

Принимает unique_lock (не lock_guard! — именно потому, что его нужно уметь разлочить и залочить обратно) и предикат; возвращает управление **с заложенным mutex'ом**, когда предикат истинен. С точки зрения пользователя как будто mutex вообще никто не разлочивал — но под капотом wait разблокирует mutex, пока ждёт, и блокирует обратно перед возвратом.

Важная деталь реализации: `wait` с предикатом — просто удобная обёртка. Если посмотреть в стандарт, там буквально написано `while (!pred()) wait(lock);` — предикат проверяем **мы сами**, CV про предикат ничего не знает, она знает только про `notify`. Low-level примитив — это `wait`, принимающий только `mutex`.

Notify: one и all, где их вызывать

Чтобы спящий кто-то проснулся, ему нужно подсказать, что предикат, возможно, поменялся. Два ключевых метода:

- `notify_one` — разбудить одного из спящих на этой CV (какого именно — стандарт **не гарантирует**; реализация обычно будит дольше всех ждущего, но это деталь);
- `notify_all` — разбудить всех.

Если делать `notify_all` там, где хватило бы `notify_one`, — потратим больше CPU: все проснутся, проверят предикат, и все, кроме одного, заснут обратно. Если нужна понятная гарантия «кто проснётся» — `notify_all` + выбор сами: например, явная очередь и проверка «первый ли я», либо более низкоуровневые библиотеки. На Unix-like системах всё реализовано через **pthread**s — low-level библиотеку, где у примитивов чуть больше настроек; на практике дефолт работает хорошо.

Сама реализация CV хитрая: она сложно дружит с `mutex`-ом и использует примитивы ОС; «сделать её самому поверх атомиков» не получится — нужен примитив ОС, который умеет «проснуться и сразу залочить `mutex` без ухода в ядро».

notify внутри или снаружи критической секции?

Вопрос на понимание: можно ли вынести `notify_one` за пределы `mutex`-а? Механически можно: пробуждённый поток всё равно сначала должен залочить `mutex` для проверки предиката. CV так умеет: поток просыпается, видит свободный `mutex` и лочит его сразу, без засыпания. Если же `notify_one` делать **внутри** критической секции, разбуженный поток проснётся, попробует залочить `mutex` — а он занят нотифайером, — и поток заснёт второй раз: «проснулись и сразу заснули», неэффективно.

Но у выноса `notify` наружу есть тонкая опасность. Пример: потоки-воркеры в конце работы делают `notify_one`, а поток-сборщик результатов ждёт на CV и, посчитав, что всё завершено, делает `delete this`. Если `notify` вынесен за пределы `lock`-а, возможна последовательность: сборщик проверил предикат, **заснул уже после проверки**, затем пришло уведомление — сборщик проснулся, сделал `delete this`, объект разрушился, а другой поток после этого обращается к его полям. UB. Если же `notify` делается под `mutex`-ом, такой разъезд невозможен.

Практическое правило: класть `notify` внутрь критической секции — меньше думать, чуть менее перфоманно; выносить наружу — более перфоманно, но надо аккуратнее думать и не пропустить баг (особенно с разрушением объектов). Выбирайте осознанно.

Почему wait обязан быть под mutex: потерянные уведомления

Разберём, что сломается, если проверять предикат не под тем же `mutex`-ом (или, например, сделать `counter_` атомарным и убрать `lock` перед `wait`). Исполнение: два потока заходят в `Enter` (`counter_ = 0`), оба проверили `counter_ > 0` — ложь — и оба собираются ждать. Первый **уже проверил** счётчик и вот-вот заснёт; в этот момент переключение контекста: приходит `Leave`, увеличивает счётчик, делает `notify_one` — а будить ещё некого, очередь пустая. Уведомление **не накапливается**: `notify_one` «не стакается», оно просто сигнализирует «проснись кто-нибудь сейчас». Потом оба потока засыпают в `wait` — и спят бесконечно, хотя счётчик уже 1.

Отсюда ключевое требование: **предикат проверяется под тем же `mutex`-ом, что защищает данные, и `wait` вызывается под этим `mutex`-ом**. Тогда невозможно состояние «предикат уже истинен, а я собираюсь спать»: пока мы держим `mutex`, `Leave` не сможет ни поменять счётчик, ни сделать `notify`. Именно поэтому здесь атомарная переменная не помогает, а мешает: `mutex` выполняет две роли (по сути одну) — защищает операции над `counter_` и «привязывает» condition variable с условием на этот счётчик; кодом мы гарантируем, что CV сигнализирует именно о свойстве этого счётчика. Можно ли сделать счётчик атомарным и убрать `lock_guard`? Сломается ровно по схеме выше: защита перехода «проверил → засыпаю» пропадает.

Spurious wakeups

Последнее и очень важное: `wait` может вернуть управление, даже если никто не делал `notify` — так называемые *spurious wakeups* («ложные пробуждения»): системе так удобно, она может вас случайно разбудить. Гарантии очень «смешные»: возврат из `wait` сам по себе ничего не значит.

Отсюда железное практическое правило: **предикат всегда проверяем после пробуждения; `wait` всегда вызываем в цикле (или через перегрузку с предикатом) — иначе код некорректен.** Без предиката это не работает.

Итоговый семафор:

```
#include <condition_variable>
#include <mutex>

class Semaphore {
public:
    Semaphore(int n) : counter_{n} {}

    void Enter() {
        std::unique_lock<std::mutex> lock{mtx_};
        not_empty_.wait(lock, [this]() -> bool {
            return counter_ > 0;
        });
        counter_--;
    }

    void Leave() {
        std::lock_guard guard{mtx_};
        counter_++;
        not_empty_.notify_one();
    }

private:
    int counter_ = 0;
    std::mutex mtx_;
    std::condition_variable not_empty_;
};
```

Проверим его моделью «злобного шедулера»: любые перестановки инструкций между потоками не ломают логику — предикат всегда проверяется под `mutex`-ом, `notify` делается под `mutex`-ом, ложные пробуждения отсекаются предикатом.

Кратко

- **Семафор** допускает в критической секции до `n` потоков (пример — `rate limiting` к внешнему сервису); при `n = 1` это `mutex`. В стандарте с C++20 есть `std::counting_semaphore`.
- **Не забывайте имя у RAII-guard'a**: `std::lock_guard(mtx_)`; — временный объект, разлочивается на точке с запятой. Хватает `lock_guard` — берите его; `unique_lock` нужен для `cv.wait` (можно `unlock/lock/перемещать`).
- **Data race** — это UB (в отличие от просто `race condition`); доступ к общим данным только под `mutex`-ом или через `std::atomic`.
- **`std::atomic`** даёт быстрые атомарные операции над одной простой переменной (`load`, `store`, `exchange`, `fetch_add`, `compare_exchange_*`), но не синхронизирует несколько переменных и не умеет ждать — только жечь CPU.
- **Никогда не пишите `busy loop`** в обычном коде (допустимо в `embedded` и при ожидании GPU); `sleep` с «магической константой» — тоже плохо: спать миллисекунды система не даёт (на Windows — порядка 16 мс).
- **Livelock ≠ deadlock**: все потоки движутся, а прогресса нет (пример — гонка за `fetch_add` при сильно отрицательном счётчике). Lock-free гарантирует прогресс хотя бы одного потока; wait-free — каждого за ограниченное число инструкций (почти недостижимо).

- **CAS:** `compare_exchange_weak` (в цикле, может фейлиться сам по себе) / `strong` (вне цикла); сравнение на равенство; при неудаче актуальное значение пишется в «угадываемую» переменную.
- **`std::condition_variable` + `wait(unique_lock, predicate)`** = та самая «магия»: спим, пока предикат ложен, просыпаемся с залоченным `mutex`-ом. Предикат проверяем мы сами в цикле — CV про него ничего не знает.
- **Notify делаем под `mutex`-ом**, если не готовы аккуратно рассуждать (риск разезда с разрушением объекта); вынос наружу — оптимизация с нюансами. `notify_one` будит одного (кого — не гарантировано), `notify_all` — всех; для понятных гарантий берите `notify_all` + свою очередь.
- **Уведомления не накапливаются:** `notify` «в пустоту» теряется. Поэтому `wait` обязан выполняться под тем же `mutex`-ом, что защищает предикатные данные.
- **Spurious wakeups существуют:** возврат из `wait` ничего не значит; предикат проверяем всегда — `wait` только в цикле/с предикатом.

Оргвопросы

Вчера выложили БДЗ и домашку по потокам. План: в конце недели — небольшая домашка по `condition variable`; ещё пара небольших домашних на потоках; после схемы — диплекс-декодер с дедлайном в самый конец сессии («под Новый год»). Записи лекций про многопоточность выложат на диск (и, вероятно, в репозиторий). На этой неделе финальный рассказ про Коллоквиум.

Лекция 10 — Advanced threads

Потоки: что это такое и почём они стоят

Когда мы пишем многопоточный код, мы создаём потоки — и вдруг оказывается, что **создавать поток дорого** (оценка порядка 5 микросекунд, на практике может быть и дольше). Что такое поток физически? Это **контекст исполнения**: состояние всех регистров плюс собственный стек. Всё. Память у всех потоков общая, своего у потока только регистры и стек. Ядро хранит состояние регистров для каждого потока в системе, стеки лежат в обычной памяти.

Дальше работает **вытесняющая многозадачность**: когда вы пишете код, вы не думаете, в каком порядке исполняются потоки. В ядре есть шедулер, который решает, какой поток когда запускать и какого снимать; при снятии ядро сохраняет состояние потока и когда-нибудь продолжает его исполнение. Типичные цифры со слайда:

- создание потока — ~5 µs;
- переключение потоков ядром — ~3–100 µs, плюс каждое переключение — это «общение с ядром»;
- большие серверы живут на ~1000–2000 потоках от силы;
- ядро может «уйти в себя»: внутри шедулера красно-чёрное дерево, при балансировках бывают спайки активности.

Почему нельзя создать 100 тысяч или миллион потоков? Ключевых соображений два. Во-первых, **переключение потока — дорогое занятие**: если каждый поток исполняет супер-короткие задачки (сложил два числа и уснул), вся система будет только тем и заниматься, что переключать потоки туда-сюда (плюс проблемы с кэшами). Во-вторых, память: в Linux каждый поток получает 8 МБ стека. Память — хитрая абстракция, физические страницы выдаются не сразу, но всё равно на стеки сотен тысяч потоков уйдёт значительная часть памяти системы.

Шедулер — очень сложный кусок кода; лектор утверждает, что в мире почти нет людей, которые понимают его целиком (в Google пытались настроить его под себя и принести патчи в open source — полностью не получилось, у них свой планировщик). В Linux сейчас шедулер — **Completely Fair Scheduler**: он раздаёт квоты процессам (у процесса сколько-то потоков), и примерно раз в 100 мс происходит итерация планирования: каждому желающему исполняться потоку выдаётся квант времени. Если процесс съедает больше своей квоты — его снимают с исполнения. Поток исполняется, пока не израсходует квант или пока не уснёт. А спят потоки много: читают диск, ходят по сети, пишут логи, ждут мьютексы. **Очень редко поток реально выжирает весь свой квант** — обычно он засыпает раньше, и тогда он убирается из очереди исполнения до пробуждения.

Парадигма «поток на задачу» → пул потоков

Много лет люди писали код в парадигме «на каждую сущность — по потоку»: например, веб-сервер на каждый запрос пользователя заводил новый поток. Это внезапно очень дорого, и последние лет 15 все перешли к простой идее:

Динамически создавать потоки почти всегда плохо. На старте программы создаём фиксированное небольшое количество потоков и больше не создаём; дальше задачи раскидываем в этот набор потоков.

Главное правило лекции: **вы всегда хотите унифицировать, откуда у вас берутся потоки. Потоков должно быть мало, и создаваться они должны условно в одном месте.** Идеально, когда число потоков совпадает с ограничением сверху на вашу программу: на ноутбуке это физические ядра, а в продакшене (docker, kubernetes, виртуалка в облаке) у процесса есть квота на ядра — и число потоков должно соотноситься с этой квотой, не сильно её превышая. Если вы где-то в глубине программы «по месту» решили запустить поток — вы почти наверняка делаете что-то не так, надо пересматривать подход.

Лектор рассказал реальную историю: был класс «базового поиска», внутри которого создавались потоки. Кто-то взял этот класс и раз в минуту стал пересоздавать его над свежей базой — и каждую минуту плодил кучу потоков. Код разваливался: безумное количество утечек, месяцами чинили. Плюс чисто инженерные неудобства: заходишь в gdb, делаешь backtrace всех потоков — там 2000 потоков, и терминал ломается.

Что в стандартной библиотеке нужно, а что — нет

Люди долго исследовали, как удобно писать многопоточный код, и насаждали в стандартную библиотеку странные вещи. **Реально нужных примитивов очень мало: потоки, мьютекс, кондвар (condition variable) и атомики. Всё остальное — странное.**

Пример «странного» — параллельные алгоритмы из C++17: в стандартных алгоритмах появился параметр `execution`-политики, например `std::execution::par`, и можно написать параллельный `std::for_each` или сортировку. Несколько месяцев все радовались статьям «плюсы умеют параллельно сортировать», а потом оказалось, что идея не работает, её толком никто не реализовывает, и профит есть только в вырожденных случаях. Проблема: количество потоков — это то, что мы хотим конфигурировать сами. Может показаться, что «число потоков = числу ядер» — хорошее правило, но нет: у программы может быть лимит на ядра (квота), а если запустить два параллельных `for_each` в двух местах — получите x2 потоков. Очень неконтролируемая сущность, годная разве что для hello world. Десятки человеко-лет ушли на то, чтобы добавить и реализовать эту штуку, — и её никто не использует. Идея «простой интерфейс снаружи, сложную задачу решим под капотом» разбилась о нюансы.

Проблема дорогого переключения и как её обходят

Даже с контролируемым числом потоков остаётся проблема: в потоки дорого засовывать маленькие вычисления. Туда нужно класть большой кусок работы; **если вы синхронизируете потоки каждые 100 инструкций — это поражение, профита никакого нет.** Проблему долгого пробуждения потока (задача выполнялась — поток уснул на мьютексе или кондваре) в Google решили радикально: сильно пропатчили ядро, чтобы потоки переключались супер-дешево, и живут на пропатченном ядре. Патчи приносили в open source, но их нормально не превьюили и не приняли — где-то в рассылках 10-летней давности лежат сырые диффы, которые к сегодняшнему ядру уже не применяются.

Весь остальной мир делает иначе — появились абстракции поверх фиксированного пула потоков:

- **асинхронное программирование** (JavaScript и т.п.) — удобные обёртки над I/O-операциями;
- **пул задач**: мы отделяем место исполнения задач от самих задач — на старте заводим 10 потоков и отправляем туда задачи, каждый поток исполняет много разных задач;
- **корутины и фиберы** — сейчас примерно все живут на них (Go, асинхронный Rust, stackful/stackless корутины в разных C++-рантаймах).

Серебряной пули нет: у всех парадигм свои проблемы. Базовый подход, поверх которого всё строится, — тредпул. Сейчас напишем его руками.

Строительный блок: UnboundedBlockingQueue

Скетч тредпула: `using Task = std::function<void()>;` конструктор принимает `thread_count` и создаёт воркеров, публичные методы `Submit` и `Stop`. Лектор подчеркнул: **не надо делать `Submit` шаблонным по типу функции — просто всегда передавайте лямбду и захватывайте в неё аргументы** (то, что `std::thread` так делает, — странное усложнение). Таску кладем не «на стек», а в очередь — поэтому первым делом нужна потокобезопасная блокирующая очередь.

По ходу лекции в коде нарочно наделали ошибок, чтобы показать грабли:

- если взять `std::lock_guard` в начале метода каждого потока и не отпускать — исполняться будет один поток, и в очередь ничего нельзя добавить;
- наивный `while (true) { ...; if (queue_.empty()) continue; ... }` — это busy-loop: мы не ждём процессорное время, мы должны спать.

Итоговый вид очереди (восстановлено по кадрам с кода):

```
template <typename T>
class UnboundedBlockingQueue {
public:
    void Push(T value) {
        std::unique_lock guard{mtx_};
        queue_.push(std::move(value));
        not_empty_.notify_one();
    }

    std::optional<T> Pop() {
        std::unique_lock guard{mtx_};
        not_empty_or_cancelled_.wait(guard, [this] {
            return !queue_.empty() || cancelled_;
        });
        if (queue_.empty()) {          // очередь отменили и дрэйнили
            return std::nullopt;
        }
        T value = std::move(queue_.front());
        queue_.pop();
        return value;
    }

    void Cancel() {
        std::unique_lock guard{mtx_};
        cancelled_ = true;
        not_empty_or_cancelled_.notify_all();
    }

private:
    std::queue<T> queue_;
    std::mutex mtx_;
    std::condition_variable not_empty_or_cancelled_;
    bool cancelled_ = false;
};
```

Имя `Unbounded` означает, что очередь может расти без ограничений, `Blocking` — что когда получить значение нельзя, поток спит (ровно то, что нужно воркеру). Поэтому здесь нужен кондвар: ждать «непустой очереди».

Тонкости, которые лектор подчеркнул:

- **`notify*` лучше делать под локом.** Формально `notify` можно делать и без лочки — `notify` вообще «про лочку не знает», кондвар связывается с мьютексом только в `wait`. Но почти всегда, если вы пишете обертку над кондваром, легко попасть в ситуацию, что вы удалите объект до того, как разбуженный воркер реально завершит работу (`notify` под локом «проще думать»). Поймать такой баг глазами очень

сложно: нужно увидеть, что раз уженный поток сразу уснул обратно. **Либо пишете notify под локом, либо, если супер уверены в себе, выносите это в отдельный хорошо протестированный код.**

- `wait` бывает двух видов: сырой (принимает только лок) и с предикатом. Второй реализован через первый: `while (!pred()) wait(lock);`. Сырой `wait` подвержен **spurious wakeups** — кондвар может «проснуться», хотя `notify` не было. С предикатом это не проблема: гарантия очевидна — если вышли из `wait` с предикатом, предикат выполнен, лочка залочена, значит после этого никто не мог ничего поменять. Семантика `wait` : он отпускает лочку, спит, обратно её блокирует и проверяет предикат — предикат всегда исполняется под лочкой, и из `wait` вы возвращаетесь с залоченной лочкой.

Что делать с исключениями из задач

Воркер исполняет задачу — а если задача кинула исключение? Обсуждение пришло к тому, что «непонятно, что делать». Варианты: записать в лог (но в **плюсовой стандартной библиотеке нет логгера**, а автор тредпула не знает, в каком приложении его будут использовать: библиотека, печатающая что-то в `stderr`, в чужом приложении — неприятно); расширить интерфейс задачи. Лектор показал идею интерфейса: вся логика задачи — в `Execute`, а в `HandleFailure` пользователь пишет, что делать при ошибке (набросок, словами):

```
struct Task {
    void Execute();           // вся логика задачи
    void HandleFailure();    // что делать, если Execute кинул
};
```

А сам лектор в demo-коде оставил в `catch (...)` смайлик `// :-)` — «делают смайлик, потому что правда непонятно, что сделать» (Google так делает, потому что у них исключений нет).

Остановка пула: убивать потоки нельзя

Воркеры бегут вечно — как их остановить? Убить поток? **Потоки убивать нельзя никогда, ни в коем случае, даже не пытайтесь.** Процессы убивать можно сколько угодно, а потоки — нет, потому что все потоки живут в одном процессе с общей памятью. Простое объяснение: вы взяли и залочили мьютекс, поток убили прямо здесь — мьютекс остался залочен навсегда, дедлок. Поток не «связан» с мьютексом: мьютекс внутри — это атомарная переменная и очередь в ядре. Более того, даже если вы сами мьютексы не используете, они есть внутри: **внутри механизма исключений есть мьютекс** (история: сервис с 20 потоками при шквале исключений начал тормозить на этом внутреннем мьютексе, всё разваливалось, сервис лёг — в `libc++` этот мьютекс потом долго выпиливали), **мьютекс есть внутри аллокатора** (любое выделение памяти).

Поэтому останавливать пул нужно аккуратно: поддержка cancellation в очереди (`Cancel()`) и `join` всех воркеров:

```
void Stop() {
    queue_.Cancel();
    for (auto& worker : workers_) {
        worker.join();
    }
}
```

Дальше — грабли с race condition. В англоязычной литературе это называется **TOCTOU: time of check, time of use** — вы проверили условие (очередь не отменена), а затем, между проверкой и использованием, состояние изменилось: поток уснул на `Pop()`, а очередь уже отменили. Поэтому **все проверки нужно делать под одной лочкой** — именно поэтому предикат ждёт внутри `wait` под мьютексом. При отмене будим всех — **notify_all**, а не **notify_one**, иначе разбудим один поток, а остальные будут спать вечно.

Компактная и выразительная запись воркера — «любимая конструкция лектора»: в `while` можно инициализировать переменную на каждой итерации, и тело исполняется, если она приводится к `true` :

```
void RunWorker() {
    while (auto task = queue_.Pop()) { // optional пуст -> выходим
```

```

try {
    task();
} catch (...) {
    // :-)
}
}
}

```

Ещё один финальный баг, найденный при тестировании: после `Cancel()` воркер проверял флажок `cancelled_` и сразу выходил, хотя в очереди оставались невыполненные задачи. А мы хотим, чтобы **все задачи до отмены успели исполниться**: сначала нужно дрэйнить очередь, и только потом останавливаться. Поэтому в `Pop()` проверяется `queue_.empty()`, а не `cancelled_`: если кондвар разбудил, а очередь пуста — значит, нас отменили и надо выходить. В деструкторе тоже была тонкость: повторный `Stop / Cancel` — нужна идемпотентность (двойная отмена не должна ломать пул).

Напоследок — забавный момент с демо: `std::cout` не thread-safe, и TSAN (`clang++ thread_pool.cpp -std=c++2b -O3 -fsanitize=thread`) честно показал data race внутри `ostreambuf_iterator`. В C++20 для этого есть `std::syncstream` (синхронная обертка над потоками вывода, добавляющая мьютекс в нужное место), но в установленной libc++ его не оказалось.

Куда дальше: future/promise и корутины

Базовый тредпул уместился в ~90 строк и, на удивление, неплохо работает — лектор оценивает, что в мире десятки/сотни миллионов ядер исполняют задачи именно в тредпулах такого уровня. Но «правильный» тредпул — искусство: хочется приоритетов, work stealing (у воркеров дисбаланс; хочется меньше синхронизировать их на одном общем мьютексе, но не терять latency). Возникает стандартный трейд-офф разработки: **обмен пропускной способности (throughput) на latency** — можно максимально эффективно использовать ядра, но исполнять задачи чуть дольше, или исполнять супер-быстро, но неэффективно использовать ядра.

Поверх тредпула хочется удобных примитивов для многопоточного кода. Стандартные `std::future` и `std::promise` существуют и удобны, но — **использовать их не надо, вообще никогда**. Дальше на курсе: как поверх тредпула сделать более современные интерфейсы — корутины разных видов, executors (бонусные задачи).

Кратко

- Поток = контекст исполнения (регистры + стек), память общая; создание ~5 µs, переключение ядром ~3–100 µs — потоки дорогие.
- Сильно больше ~1000–2000 потоков на сервере не создают: дорогое переключение и по 8 МБ стека на поток в Linux.
- **Создавайте потоки мало и в одном месте; число потоков соотносите с квотой процесса на ядра; не создавайте потоки динамически по месту.**
- Из стандартной библиотеки реально нужны только потоки, мьютекс, кондвар и атомики; параллельные алгоритмы C++17 (`execution::par`) — на практике не используйте; `std::future` / `std::promise` — тоже не надо.
- Блокирующая очередь (`UnboundedBlockingQueue`) — базовый строительный блок тредпула; ждать условия нужно через `wait` с предикатом (защита от spurious wakeups), а не busy-loop.
- **notify делайте под локом** (кондвар сам про лок ничего не знает); при отмене/закрытии — `notify_all`.
- **Убивать потоки нельзя никогда**: останется залоченным мьютекс (а мьютексы спрятаны даже в исключениях и аллокаторе) — только аккуратная отмена + `join`.
- **Проверяйте условие и используйте его под одной лочкой (TOCTOU)** — иначе проверка устареет между проверкой и использованием.
- После отмены сначала дрэйните очередь задач, потом завершайте воркеров; проверять надо `queue_.empty()`, а не только флаг отмены.
- Мелкие задачи в потоках не окупаются: если синхронизация каждые 100 инструкций — профита нет; дальше — корутины/файберы и trade-off throughput vs latency.

Орг. моменты. Домашка про кондвары доступна сразу после лекции; будет ещё одна маленькая на lock-free и большой проект — JPEG-декодер (дедлайн 31 декабря, «новогодний проект»). Коллоквиум состоится, он в формуле оценки: две даты на выбор — 13 или 20 декабря (лекторы за 20-е, но 20-е — сессия). Устный: выпадает тема из лекций (список темок пришлют на этой неделе, примерно до следующей лекции включительно), рассказываете тему — 6 баллов, остальное добираете небольшими вопросами на понимание кода (типа «расскажите про one-definition rule» или «что не так с этим кусочком кода»). Защищать домашки не надо.

Лекция 11 — Atomic. Lock-free алгоритмы

`std::atomic` : базовый строительный блок

Сегодня — про lock-free алгоритмы. Начнём с базового строительного блока, из которого под капотом собираются почти все многопоточные алгоритмы, и в частности lock-free и wait-free: это атомики (`<atomic>`).

`std::atomic` — шаблонный класс, в который можно подставлять свой тип. У него есть набор методов; базовые два — `store` (записать значение) и `load` (прочитать), плюс более сложные — `compare_exchange`, `fetch_add`, `fetch_and` и так далее. Минимальный пример:

```
#include <atomic>

int main() {
    std::atomic<int> x;
    x.store(123);
    int y = x.load();
}
```

Почему переменные называются *атомарными*? Потому что все операции над ними обладают свойством атомарности, то есть неделимости: какую бы операцию и из скольких бы потоков вы ни вызывали над одной атомарной переменной, программа не имеет гонок на ней (data races нет) — все операции «выполнятся в каком-то порядке», каким именно — обсудим дальше в контексте модели памяти.

Зачем атомики, если есть мьютекс?

Первый естественный вопрос: зачем вообще атомарные переменные, если того же можно добиться мьютексом? Лектор показал «свой» атомик, реализованный на мьютексе, — он восстанавливается по кадрам так:

```
template <typename T>
class Atomic {
public:
    void Store(T value) {
        std::lock_guard guard{mtx_};
        value_ = value;
    }

    T Load() {
        std::lock_guard guard{mtx_};
        return value_;
    }

private:
    std::mutex mtx_;
    T value_;
};

int main() {
    std::atomic<int> x;
    x.store(123);
}
```

```
int y = x.load();
}
```

Такая реализация даёт ту же гарантию «нет гонок на переменной». Так почему стандартная реализация лучше? Обсуждение в аудитории свелось к двум утверждениям: «мьютекс дороже» и «в наивной версии больше инструкций». Оба требуют проверки.

Spinlock и почему им не надо пользоваться

Первый кандидат на «мьютекс без мьютекса» — spinlock на одной атомике:

```
class Spinlock {
public:
    void Lock() {
        while (locked_.exchange(true)) {
            // крутимся в цикле
        }
    }

    void Unlock() {
        locked_.store(false);
    }

private:
    std::atomic<bool> locked_ = false;
};
```

Он работает очень плохо: пока вы пытаетесь его заблокировать, вы ждёте в цикле и **сжигаете CPU, а сжигать CPU супер-вредно**.

Spinlock в userspace практически никогда не нужен. Исключения — если вы пишете ядро операционной системы (там это неизбежно) или взаимодействуете с железом, которое ожидает только такой протокол (где-то — с видеокартами, на некоторых МК-системах). В реальном современном коде, который пишет большая часть разработчиков, спинлоки не нужны: кто-то думает, что это оптимизация и «лучше мьютекса», — нет, это хуже: вы сжигаете кучу CPU и ничего не выигрываете.

Настоящий мьютекс сложнее: вы напишете его в домашке через хитрый системный вызов **futex**. Идея futex — переносить ожидание в kernel space, но оставаться в user space на happy path: когда мьютекс никто не держит, блокировка происходит вообще без системного вызова.

Сколько стоит мьютекс против атомака: бенчмарк

Ключевое наблюдение лектора про инструкции: процессоры суперскалярные — исполняют много инструкций одновременно, но обычно те, что находятся рядышком. Если какая-то инструкция дорогая, вы быстро выполните всё вокруг неё и дальше будете ждать именно её.

С точки зрения инструкций атомик действительно «дешёвый по коду»: запись — это одна инструкция (move или exchange), добавить — lock add. А вот у Atomic<T> на мьютексе число инструкций оценить почти невозможно: в лучшем случае — пара инструкций (happy path без ухода в ядро), в худшем — системный вызов, внутри которого неограниченное количество инструкций, плюс усыпить и разбудить поток. **Вы не можете дать оценку сверху тому, насколько быстро мьютекс разблокируется.**

Бенчмарк (глупый, но показательный): куча потоков по очереди берут мьютекс, увеличивают чиселку и кладут её обратно; альтернатива — то же самое через атомик, где ++ — это ровно то же самое, что fetch_add. На оси — число потоков от 1 до 16 (одна итерация — количество потоков).

Результаты неожиданные:

- однопоточно инкрементировать число, конечно, быстрее, чем дёргать его из многих потоков — это не вопрос;
- по реальному времени (среднему времени исполнения) mutex-версия выглядит неплохо;

- **но CPU мьютексная версия тратит сильно больше** — куча потоков ждут, пока им разрешат работать, и тратят на ожидание инструкции;
- **атомарная версия — сильно хуже** и по времени, и по процессорному времени: в худшем случае отличие в три раза и больше.

Почему? Сравним happy path и не-happy path. Когда повезло и мьютекс заблокировался без ухода в ядро — записать или прочитать bool (один байтик) сильно дешевле, чем делать атомарный инкремент. Отсюда два правила:

- **Разные атомарные операции имеют разную стоимость.**
- **Если есть атомарная переменная — это не значит, что она быстрее. Это очень сильно зависит от того, что именно вы делаете и с чем сравниваете. Алгоритм вполне может ускориться, если заменить атомарную переменную на мьютекс.**

Это странно и неестественно: когда раскапываешь более продвинутый механизм, ожидаешь, что он будет работать лучше. На практике — совершенно не факт.

Lock-freedom и wait-freedom: определения

Зачем тогда атомики? Во-первых, из них всё сделано. Но главное их свойство — **количество инструкций**: если один алгоритм — это всегда одна инструкция, а второй — «сколько-то инструкций, из которых в худшем случае системный вызов», то можно осмысленно сравнивать, какой из них ведёт себя лучше в разных ситуациях. Отсюда приходят понятия **lock-freedom** и **wait-freedom**.

Определение lock-freedom (как его дал лектор): алгоритм является lock-free, если при любом исполнении, если его достаточно долго «покрутить», хотя бы один поток сделает прогресс. «Полезность» прогресса зависит от алгоритма — в бенчмарке это, например, «число поинкрементировалось».

Из этого определения есть следствие-критерий, который на практике проверять сильно проще:

Если вы в любой момент останавливаете любой поток (просто подключили дебаггер, и он перестал исполняться), то остальные потоки всё равно продолжают делать прогресс — система не ломается.

Проверим критерии на примерах:

- `Atomic<T>` на мьютексе **не lock-free**: 100 потоков могут вызывать `store`, и остановка любого из них ничего не ломает, но если остановить поток, который в этот момент *владеет мьютексом*, — прогресса в системе нет, всё сломалось.
- Замена мьютекса на spinlock из атомиков **ничего не меняет**: если удачно заблокировать поток, который сделал `locked_ = true` и держит «блокировку», — прогресс потерян. Всё то же самое по факту.

Это не абстракция: на практике свойство вылезает постоянно, когда вы пишете достаточно низкоуровневый код или гарантируете что-то про время исполнения. Классическая ситуация: 100 потоков что-то делают, посередине вставлен мьютекс, на большом масштабе кто-то заблокировал мьютекс — и его отправили надолго спать (процессора не хватило, система долго не исполняла этот поток), и вся программа стоит и ждёт этот мьютекс.

Лектор подчеркнул конкретный пугающий сценарий: **если программа запущена в docker с лимитом процессоров и случайно съедает квоту, шедулер отправляет её в троттлинг — не даёт времени исполнения до следующего кванта планировщика. Троттлинг — это очень часто десятки миллисекунд простоя.** Для C++-программы десятки миллисекунд — это правда много (за 10 мс можно сделать очень много полезного, можно отрендерить несколько кадров). Если в этот момент все потоки стояли на залоченном мьютексе — это сильно бьёт по latency.

Отсюда практический вывод:

- **Lock-free — это не про производительность, а про гарантии. В определении lock-freedom нет ни слова про скорость.** Частое заблуждение: «напишем lock-free — будет супербыстро». Нет, скорее всего будет так же или медленнее; но про такие алгоритмы проще думать с точки зрения интерфейса и гарантий.

- **Lock-free алгоритмы надо применять точечно, по месту, хорошо понимая зачем.** Заменять обычную хешмапу на lock-free хешмапу «везде всегда» — почти наверняка неправильно.

is_lock_free и атомики больших типов

Вопрос «всегда ли атомик lock-free» имеет осмысленный ответ не всегда. Во-первых, у атомика есть запрос этого свойства:

```
bool is_lock_free = std::atomic<int>::is_always_lock_free;
```

Во-вторых, в атомик можно положить не только «поддерживаемый» тип. Возьмём структуру на 100–400 байт — и она, к удивлению, компилируется как `std::atomic<Struct>`. Но:

- у чисел `is_always_lock_free` выполняется;
- у больших структур — не выполняется: внутри появляется мьютекс, одной инструкцией там обойтись нельзя.

Почему стандарт вообще поддерживает такое? Требования к типу в `std::atomic` — тривиальная копируемость, отсутствие сложных конструкторов копирования; формально вы можете делать над таким атомиком compare exchange. Но на практике `std::atomic` от большой структуры — «что-то очень странное, так делать не надо». Поддержка оставлена сознательно, чтобы не ограничивать архитектуры: если бы стандарт требовал, например, «atomic от int — всегда lock-free», то на архитектуре, где это не так, нельзя было бы поддержать весь стандарт. А на всех современных архитектурах с числами `is_lock_free` — true.

Ещё нюанс: у числовых и нечисловых атомиков **разный набор методов**. У чисел методов сильно больше: `fetch_add`, `fetch_and`, `fetch_xor` и т.д. — они позволяют выражать код в специфичные процессорные инструкции. У «нечисел» — только `store`, `load` и `compare_exchange` (плюс `wait / notify`).

wait / notify : обёртка над futex

У атомиков (в том числе `std::atomic<bool>`, который, в отличие от `std::vector<bool>`, нормальный вектор) есть странные методы `notify_one`, `notify_all` и `wait` — очень похожие на методы кондвара. Выглядит так, будто кто-то скопировал методы из condition variable. На самом деле это **C++20-обёртка над futex** — тем самым системным вызовом, на котором сделаны мьютексы. Это удобный механизм его использовать, но:

wait / notify у атомиков — суперспецифичная штука; пока вы не разобрались с futex, лучше их вообще не трогать. Почти никогда вы не хотите это использовать. Реально полезно, когда вы почему-то пишете свой мьютекс или суперспецифичный алгоритм.

Модель памяти: memory_order и sequential consistency

Дальше лектор показал главное, из-за чего «думать про атомики сложно»: даже `load` и `store` принимают аргументы — **memory_order**. В автокомплите у `std::memory_order` есть значения: `relaxed`, `consume`, `acquire`, `release`, `acq_rel`, `seq_cst`.

Ментальная модель атомиков в общем виде — «некоторое безумие» (лектор отсылал к курсу Ромы Липовского как к месту, где этим мозгом нормально занимаются). Для курса выбрана модель, в которой можно адекватно думать: **sequentially consistent**. Это модель исполнения, в которой легко понять, почему получился тот или иной результат, — но она генерирует недостаточно быстрый код. На x86-64 думать в ней — окей; на более слабых (по модели по умолчанию) архитектурах можно получать более эффективный код, компилируя в другую модель. Модель исполнения задаётся для **конкретных операций** над атомарными переменными; по умолчанию используется `memory_order::seq_cst` — самая сильная, самая строгая гарантия, остальные слабее.

Что значит «слабее» — лучше всего показать на примере. Пусть в разных потоках выполняется:

```
// поток 1
x.store(1); // A
y.store(2); // B
```

Если оба стора последовательны и консистентны, какие бывают пары `(x, y)` при чтении другим потоком? `0/0`, `1/2`, `1/0` — но не наоборот. Но если для конкретики поставить одному из сторов `memory_order::relaxed`, гарантии порядка уже нет. `relaxed` — самый слабый memory order: он гарантирует только атомарность самого действия над переменной (нет гонок), а **в каком порядке изменения становятся видимыми другим потокам — как повезёт** (вспомните store buffer, который когда-нибудь флашится в общую память, — непонятно когда).

Лектор подчеркнул связь с компилятором: компилятор оптимизирует код с учётом as-if rule — он не может менять наблюдаемое поведение программы, процессор не «ломает» корректную программу. Если вы как программист ожидаете, что `y` уже равен 1, когда `x` равен 1, — при слабой модели это ожидание не обосновано, и «программа сломана» на уровне вашей ментальной модели. Строгая модель памяти стоит производительности: действительно нельзя лишний раз переставлять инструкции.

Формулировка sequentially consistent модели:

Результат исполнения программы такой же, как если бы мы запускали её на одном ядре: в каждый момент времени выполняется ровно один поток, и операции всех потоков чередуются в некоторый общий порядок.

Чем это хорошо? **Про такую программу удобно думать.** Голова однопоточная: «сначала исполню эту строчку, потом эту, потом вот это место в другом потоке» — вы придумываете конкретное чередование операций, которое либо ломает алгоритм, либо нет. Более слабые модели этот порядок ломают. Даже у `seq_cst` очень много нюансов, но если memory order не писать — используется `seq_cst`. **Потенциально можно написать более сложный код со слабыми memory order и получить более эффективный — но сначала научитесь жить в `seq_cst`.**

Проблема: поддержание максимума в атомике

Практическая задача, ради которой всё это: написать lock-free операцию «поддерживать максимум в атомике». Несколько потоков записывают значения, по результатам в переменной должен остаться максимум.

Наивная попытка — «прочитать, сравнить, записать»:

```
template <typename T>
T FetchMax(std::atomic<T>* max, T value) {
    auto prev = max->load(std::memory_order::seq_cst);
    if (value > prev) {
        max->store(value);
    }
    return prev;
}
```

Корректно ли это в `seq_cst`? Нет. Думать надо так: **вы живёте в sequentially consistent мире и пытаетесь придумать чередование операций, которое ломает алгоритм.** Оно находится легко: пусть в атомике 0, два потока вызывают `FetchMax` со значениями 1 и 2. Оба потока прочитали `prev = 0`; поток со значением 2 успевает всё сделать (store 2); поток со значением 1 уже прошёл проверку — и тоже делает store 1. Итог: после двух параллельных `FetchMax(1)`, `FetchMax(2)` из нуля в атомике осталась единица. Всё сломалось.

Что делать? «Мьютекс!» — но тогда это не lock-free, а lock-free уметь делать важно: в супер-больших программах и в ядре Linux всё на одном глобальном мьютексе блокировать нельзя — система развалится.

Решение — `compare_exchange`.

Compare exchange: семантика

`compare_exchange_weak` / `compare_exchange_strong` — у них два входа и один выход, и они ещё и меняют переменную, так что за сигнатурой надо следить внимательно. Первый аргумент — `expected` — передаётся по ссылке, второй (`desired`) — по значению; в C++ это по коду не видно, «в расте было бы понятно».

Семантика (лектор сначала показал неатомарный эквивалент, чтобы было видно, что происходит):

```
// неатомарная версия, чтобы понять семантику
template <typename T>
bool CompareExchange(T& atomic, T& prev, T value) {
    if (atomic == prev) {
        atomic = value;
        return true;
    }
    prev = atomic;    // обновляем «догадку» реальным значением
    return false;
}
```

То есть: вы сравниваете значение, которое лежит в атомарной переменной, со своей догадкой `prev`. Это происходит атомарно. Если угадали — в переменную записывается `value`, возвращается `true`. Иначе — в `prev` записывается реальное значение, возвращается `false`, и вы повторяете попытку.

`weak` и `strong` в этом курсе «практически ничего не значат», можно использовать и то и другое; **weak чуть лучше использовать, если вызов находится в цикле** (он может ложно *failing*’уть, но в цикле это не страшно).

Теперь правильный lock-free максимум:

```
template <typename T>
void FetchMax(std::atomic<T>* max, T value) {
    T prev = max->load();
    while (!max->compare_exchange_weak(prev, value)) {
        // не угадали: prev обновился, пробуем снова
    }
}
```

Это типичный **CAS-цикл** (compare-and-swap loop): «прочитал — попытался угадать — не угадал, обновил догадку — повторил». Практически весь lock-free код строится поверх этой конструкции.

Отдельные замечания лектора:

- **Compare exchange — очень дорогая операция, самая дорогая среди атомарных:** на процессоре её реализовать по-настоящему сложно (всё должно быть атомарно). Через неё, впрочем, выражаются все остальные операции: `store` — это «compare exchange, если угадали», `fetch_add` — CAS-цикл с добавлением. (Чистый `load` через CAS сделать нельзя — он бы «ломал» переменную; поэтому `load` и `store` всё-таки существуют отдельно.)
- По кадрам видно и другой стиль записи того же цикла — через явный `break`:

```
while (true) {
    auto prev = max->load();
    if (max->compare_exchange_weak(prev, value)) {
        break;
    }
}
```

Лектор отметил: **на практике лучше писать вариант «load внутри цикла перед CAS» — в happy path можно срезать один compare exchange; но думать проще про вариант с догадкой снаружи.** (Оба варианта эквивалентны по смыслу.)

- Является ли такой CAS-цикл lock-free? Да: он очень похож на спинлок, но здесь нет «владельца» — любой поток в любой момент может успешно завершить CAS. Хотя по стоимости это дороже спинлока.

Корректность многопоточных алгоритмов: как доказывать

Как доказать, что многопоточный алгоритм корректен? Запустить на одном ядре — не доказательство. Формально доказывать сложно (это дискретная математика; даже доказать отсутствие бесконечных циклов в коде нетривиально). Лектор упомянул фреймворки формальной верификации — в частности **TLA+**:

формальный язык спецификации алгоритма, который позволяет проверить (и фактически доказать), корректен ли алгоритм.

Практический вывод: **если вы внимательно посмотрели на lock-free алгоритм и не нашли багов — они, скорее всего, всё равно есть.** Для серьёзных структур данных нужны формальные методы или хотя бы стресс-тесты.

Wait-freedom: более сильная гарантия

В CAS-цикле максима есть тонкость: вспомните определение lock-freedom — «хотя бы один поток делает прогресс». Может оказаться так, что у вас есть поток-неудачник: все постоянно успешно обновляют максимум, а один поток просчитал значение, делает CAS — не получилось, снова, и снова, — **и так может крутиться вечно, и это не противоречит lock-freedom.**

Поэтому lock-freedom — недостаточно сильная гарантия. Есть **wait-freedom**: каждый поток делает прогресс за ограниченное количество операций (шагов). Это «самая классная» гарантия, но wait-free алгоритмов очень мало. Примеры wait-free операций — `store` и `load`: они завершаются за конечное число шагов всегда. CAS-цикл `FetchMax` — lock-free, но не wait-free.

Замечание про мьютексы: у `std::mutex` есть `try_lock` и `unlock`, оба выполняются за ограниченное число операций и никогда не блокируются. Можно ли на этом построить wait-free алгоритмы? По сути — нет: либо из такого мьютекса получится спинлок, либо что-то очень странное (иногда — какие-то вероятностные структуры). И общее правило: **чем сильнее требования к алгоритму, тем меньше таких алгоритмов существует.**

Lock-free стек: Push

Теперь напишем что-нибудь настоящее — lock-free стек (wait-free стек — без шансов). Идея: односвязный список, голова — атомарный указатель. Указатель — это не число, но это более сильная штука; с ним можно делать те же операции — атомарно читать и записывать, как набор байтиков, — примерно как атомарный `size_t`, и на любой адекватной архитектуре операции над указателем lock-free.

```
#include <atomic>
#include <optional>

template <typename T>
class LockFreeStack {
public:
    void Push(T value) {
        Node* new_head = new Node{value, nullptr};
        while (!head_.compare_exchange_weak(new_head->next, new_head)) {
            // spin
        }
    }

    std::optional<T> Pop() {
        // ...
    }

private:
    struct Node {
        T value;
        Node* next;
    };

    std::atomic<Node*> head_ = nullptr;
};
```

Две ловушки, на которых лектор остановился:

- По умолчанию в атомарной переменной мусор — не забывайте инициализировать атомарные переменные (`head_ = nullptr`).

- Хитрость в `Push` : мы используем `new_head->next` как «ожидаемое» значение CAS. Изначально `new_head->next == nullptr` — это и есть наша «догадка» о текущей голове. Если угадали (голова не менялась), CAS одним махом записывает новую голову и заодно проставляет ей `next` — старую голову. Если не угадали — `compare_exchange_weak` запишет в `new_head->next` реальную голову, и на следующей итерации догадка снова корректна. Красиво и без отдельного `load` .

Рор тоже пишется через CAS (если прочитанная голова не `nullptr` — пытаемся «перещёлкнуть» голову на `next` ; если не угадали — повторяем):

```
std::optional<T> Pop() {
    Node* cur_head = head_.load();
    while (cur_head != nullptr &&
           !head_.compare_exchange_weak(cur_head, cur_head->next)) {
        // повторяем
    }
    if (cur_head == nullptr) {
        return std::nullopt;
    }
    T value = cur_head->value;
    delete cur_head; // вот здесь начинаются проблемы
    return value;
}
```

И тут же лектор заметил: **в `happy path` лучше писать `поп/пуш` так, чтобы срезать один `compare exchange`, но думать проще про канонический вид**. Проверка на пустой стек обязательна — без неё разыменуем нулевой указатель.

Проблема удаления: гонка и ABA

В коде выше есть фундаментальная проблема: `delete cur_head` . Рассуждение «мы сделали CAS, значит, узел вылинкован из стека и его можно удалить» — неверно. Пока мы между `head_.load()` и CAS спали, другой поток мог сделать полный рор (вылинковать и **удалить** наш `cur_head`), а мы, проснувшись, разыменовываем уже удалённый указатель — `segfault`. В любой точке между чтением головы и CAS может «влезть» что угодно — смотреть надо на каждую инструкцию.

Частичное решение — ограничить структуру: **MPSC** (multi-producer, single-consumer — писать может много потоков, а читать/рор делать один). Тогда в рор никто конкурентно не лезет, и проблема удаления исчезает... но generic lock-free стек так не пишется.

Общее человечество придумало честный lock-free стек, но «любая попытка придумать его адекватным — появляются очень странные вещи». Два механизма:

- **Hazard pointers**: перед разыменованием указатель помечается как «опасный к удалению» и откладывается в отдельное хранилище; удалять его можно только когда известно, что им больше никто не владеет; чистят аккуратно в конце. Само это хранилище — непростая структура (возможно, с мьютексом внутри).
- **Не удалять вообще** (откладывать освобождение до полной тишины) — вариант той же идеи.

И вторая, ещё более смешная проблема — **ABA**. Пусть стек вида `A → B` . Мы прочитали `cur_head = A` и `cur_head->next = B` , и уснули. Пришли два других потока: один сделал полный рор (удалил A и B), второй попушил узлы, а потом `push` создал новый узел — и вот операционная система/аллокатор выдала под новый узел **тот же адрес A** (в цикле «выделил-освободил» указатели постоянно переиспользуются). Теперь в стеке снова «A → ...», мы просыпаемся: «голова — A, всё совпадает», CAS успешно меняет A на B — и стек сломан: мы сравнивали указатели, но по указателю лежит уже совершенно другой объект.

Как решать: **hazard pointers** или **tagged pointers** — зашивать в сам указатель счётчик версии, по которому можно понять, «та ли это» версия узла. Это больно, тяжело и очень хорошо прогревает мозги; подробности — на семинарах.

В конце лекции — финальные замечания: на lock-free стеке можно написать MPSC-очередь; а вот MPSC поверх некоторых других примитивов — «скорее нельзя, попробуйте».

Кратко

- `std::atomic<T>` — строительный блок многопоточных и lock-free алгоритмов: `store`, `load`, `fetch_*`, `compare_exchange`; операции атомарны, гонок на переменной нет.
- **Атомик — не синоним «быстро»**: разные атомарные операции имеют разную стоимость, и наивный `Atomic<T>` на мьютексе в бенчмарке обогнал настоящий атомик в разы; алгоритм может ускориться от замены атомика на мьютекс — бенчмаркайте.
- **Spinlock в userspace не нужен** (ядра ОС и специфическое железо — исключения): он сжигает CPU; настоящий мьютекс строится на `futex` — user space на happy path, ожидание в kernel space.
- **Lock-free = при любом исполнении за конечное время хотя бы один поток делает прогресс**. Проверочный критерий: остановка любого потока не ломает систему. `Atomic<T>` на мьютексе и на спинлоке — не lock-free.
- **Lock-free — это про гарантии, а не про скорость**; ценность — в системах с троттлингом/лимитами CPU (docker), где заснувший владелец мьютекса замораживает всех на десятки миллисекунд.
- **Wait-free — сильнее: каждый поток делает прогресс за ограниченное число шагов** (`store`, `load` — wait-free; CAS-циклы — только lock-free).
- У больших нечисловых типов `std::atomic` может быть не lock-free (проверяйте `is_always_lock_free`) — под капотом мьютекс; у чисел набор `fetch_*`-методов богаче. `wait` / `notify` — обёртка над `futex` (C++20), лучше не трогать.
- **По умолчанию — `memory_order::seq_cst`**: программа ведёт себя как будто все потоки исполняются в каком-то едином порядке на одном ядре; `relaxed` гарантирует только атомарность операции, а не порядок видимости.
- Весь lock-free код — это **CAS-цикл**: `load` → `compare_exchange_weak(prev, value)` → при неудаче `prev` обновился, повторяем. CAS — самая дорогая атомарная операция, но через неё выражается всё остальное.
- Lock-free стек: атомарная голова-указатель, push/pop через CAS; главные грабли — неинициализированный атомик, `delete` узла, который конкурентный поток мог уже удалить (**hazard pointers**), и **ABA-проблема** (переиспользование адреса аллокатором; лечится `tagged pointers`).

Орг. моменты

В конце лекции объявлена домашка: **сегодня выдаётся JPEG-декодер**. Также напомнено, что мьютекс через `futex` — тоже домашняя работа. Отдельная рекомендация: тема атомиков и моделей памяти в полсеместра не помещается, углубляться — к курсу Ромы Липовского; `hazard pointers` и `tagged pointers` подробно разберут на семинарах.

Лекция 12 — Корутины

Лекция посвящена корутинам (coroutines) в C++20: зачем они нужны, какими бывают, и как устроен их (довольно страшный) интерфейс в стандарте. Лектор сразу предупреждает: тема сложная, он сам её постоянно забывает, поэтому нужно внимательно «следить за руками» и спрашивать сразу, если что-то непонятно. Большая часть лекции — живое кодирование: с нуля пишется сначала минимальная корутина «hello world», потом полноценный генератор чисел, потом собственный `awaiter`.

Зачем всё это нужно: проблема утилизации CPU

Начнём с интуиции. Вы пишете многопоточную программу и хотите, чтобы процессор — довольно дорогой ресурс — был утилизирован максимально. Но обычно так не происходит: если посмотреть на любую программу, скорее всего, она не утилизует весь доступный ей CPU. Полностью нагружают процессор лишь единицы программ, и обычно это программы, которые делают что-то бессмысленное — например, перебирают хэши.

Почему так? Первая мысль — «программа ждёт память». Да, программы ждут память, но это ожидание **синхронное**: процессор крутится в spin-loop и ждёт, пока память ответит. С точки зрения операционной системы это выглядит как использование процессора, и на уровне железа даже нетривиально отличить «использование памяти» от «вычислений». Так что память — не главная причина.

Главная причина — **ввод-вывод**. Любая программа, которая делает что-то нетривиальное, взаимодействует с внешним миром, а взаимодействие со внешним миром всегда идёт через интерфейсы операционной

системы, в первую очередь через ввод-вывод. В категорию «ввод-вывод» обычно складывают всё: сеть, диск, запуск процессов, ожидание (буквально `sleep`), взаимодействие с ОС. Абстракции там одни и те же.

Пример: программа читает матрицы с диска, перемножает их и записывает обратно. Пока она перемножает — используются вычисления; в начале и в конце — ввод-вывод. Если программа перемножает много пар матриц, то каждый конкретный поток часть времени что-то считает, а часть времени **блокируется и ждёт**, пока ОС скажет, что запись завершилась. Поток часто спит, часто ждёт, пока кто-то дёрнет за `condition variable` — то есть CPU он максимально не использует почти никогда.

Чем это плохо? Вы не понимаете, **сколько потоков завести**, чтобы максимально утилизировать доступные ядра: сколько ядер? В два раза больше? В три? В восемь? Может, надо эластично подбирать? Люди много лет придумывали разные эвристики, и работало плохо. До появления более современных концепций люди, например, очень любили заводить поток на каждый чих — так работали HTTP-серверы образца 2005 года: на каждый новый запрос создавался поток и разрушался в конце. Это довольно дорого, и память используется неконтролируемо.

Отсюда необходимость: **разнести вычисления и исполнителей (потоки)**. Разбить программу на атомарные независимые вычисления, которые можно исполнять на разных потоках, а потом плотно упаковать их в «идеальное» количество потоков. В идеале — когда вы спите, читаете с диска или из сети, вы в этот момент **не занимаете поток**.

Теоретически это можно было бы делать внутри операционной системы — она для этого и есть: создать кучу потоков и надеяться, что ОС хорошо ими разрулит. Но проблема в том, что это дорого, и ОС слишком мало знает про вашу программу, чтобы управлять потоками достаточно хорошо. (Гугл, по слухам, отскейлил себе Linux так, что можно заводить кучу потоков и дешево между ними переключаться, — но современный Linux «из коробки» не такой.) Поэтому люди пошли другим путём — возможно, ошибочным, но точно другим. Это и есть корутины.

Ключевая идея: у вас есть вычисление, и в какой-то его точке вы понимаете, что нужно подождать внешнего события — кто-то выполнил другую работу, кто-то прочитал с диска, пришло уведомление, что угодно. Вы хотите ждать **неблокирующе, не занимая поток исполнения**: остановить вычисление, куда-то сохранить его состояние и продолжить только тогда, когда событие произошло. Это очень напоминает то, как устроены сами потоки, — по сути, мы реализуем **планировщик потоков в userspace**.

Почему не ядро? Переключение потока — это всегда несколько «приключений» в `kernel space`, это единицы микросекунд, дорого. Потоки едят много памяти на стек (не вся выделяется сразу, но резерв — большая), внутри ядра есть фиксированные накладные расходы, миллион потоков завести очень дорого. Плюс, когда вы отдаёте переключение операционной системе, вы **не контролируете порядок исполнения** потоков: у ОС очень ограниченные механизмы приоритетов, а когда решаете вы сами — можете выставлять приоритеты, придумывать свои эвристики. Ещё важная терминология: ядро делает **вытесняющую (preemptive) многозадачность** — оно в любой момент может снять произвольный поток с ядра. В `userspace` это делать сложно, поэтому используется **кооперативная многозадачность**: корутины сами говорят «в этих точках меня можно усыпить». Это позволяет лучше контролировать, какие ресурсы и сколько ядер вы используете и где происходит взаимодействие с ОС.

Первый пример: генератор, и почему на C++ это громоздко

Как выглядит абстракция «остановить функцию и продолжить её потом»? Классический пример — **генератор**: бесконечная последовательность чисел, которую потребитель вытягивает по одному значению.

На C++ «школьный» ответ — написать `range` с `begin / end`. Идея простая, но решение громоздкое: нужно завести **два типа** — сам `range` и итератор внутри него, — и набегает строк пятьдесят не очень понятного кода (все это писали в домашке первого или второго модуля). Работает, но «по-нормальному» это громоздкая конструкция.

А вот как это выглядит на Python — и это уже честная реализация:

```
def gen():
    num = 0
    while True:
```

```
yield num
num += 1
```

В функции объявляется переменная `num = 0`, дальше в бесконечном цикле — некое ключевое слово `yield` и увеличение числа. Вызываем функцию снаружи и «достаём» из неё значения:

```
gen = gen()
print(next(gen))    # 0
print(next(gen))    # 1
print(next(gen))    # 2
```

В REPL это выглядит так: `gen()` — ошибка `'generator' object is not callable` (объект-генератор нельзя «позвать» как функцию), а вот `next(gen)` каждый раз возвращает следующее число: 0, 1, 2, ...

Что произошло? Мы начали исполнять функцию, дошли до строчки с `yield` — и **почему-то вернули исполнение родительской функции**. Родитель поисполнялся, вывел наше значение, потом мы вернулись обратно и продолжили исполнение нашей функции до следующего `yield`. То есть `yield` — это такой `return`, который возвращает значение из функции, **но не останавливает саму функцию**: она после этого может быть продолжена вызовом `next`.

Забавная деталь про Python: под капотом `next(g)` на самом деле делает `g.send(None)` — у генератора есть странный метод `send`, в который можно «отправить» что-то (в данном случае `None`), и он продолжит исполнение корутины.

Это выглядит магично, потому что это **не функция в математическом смысле**: она постоянно «выплёвывает» из себя значения, не прекращая исполнение.

Формализм: четыре примитива

Давайте введём небольшой формализм. Что вообще происходит, когда вы вызываете обычную функцию? На ассемблере вызов функции — это сохранение всего состояния, которое было в момент вызова (все нужные регистры, всё состояние текущей функции — куда-нибудь сохраняется, хоть вызывающим, хоть вызываемым), и переключение на новую функцию. Возврат обратно — уничтожение текущего состояния (локальные переменные разрушаются) и восстановление контекста вызывающей функции.

Назовём четыре примитива:

- **suspend** — сохранение текущего состояния;
- **activate** — переход на следующее состояние (переключение контекста);
- **terminate** — разрушение текущего состояния;
- **yield** — возврат в родительский контекст.

Обычный вызов функции — это `suspend + activate`. Возврат — `terminate + yield`. А что будет, если мы их **перемешаем**? Получатся сопрограммы — корутины. (Слово «файберы», `green threads`, го-рутины — всё это, по сути, одно и то же: разновидности корутин.)

Два «нелегальных» для обычных функций сочетания:

- **переключение в родителя**: делаем `suspend` (как при вызове дочерней функции — сохраняем своё состояние), но потом прыгаем не в ребёнка, а **обратно в родителя**. То есть мы не вызываем дочернюю функцию, а сохраняем своё состояние и возвращаем управление родителю — и можем потом продолжиться;
- **переключение в ребёнка**: симметричная конструкция `suspend + yield` — сохраняем текущее состояние и прыгаем в уже подготовленное состояние.

Именно это происходит в генераторе: когда мы делаем `yield`, мы сохраняем текущее состояние исполнения (внутри функции `gen` сохраняются все локальные переменные, например `num`), передаём управление родительской функции — она крутится, делает полезную работу — и только потом, когда родитель вызовет `next`, мы продолжаем исполнение корутины. В обычной функции так нельзя: из функции всегда нужно вернуть что-то «насовсем». Здесь же вы сохраняете весь локальный стейт и потом продолжаете его исполнять.

Корутины оказываются очень полезными при работе с ввод-выводом, то есть практически во всех программах. Но для этого нужна **сильная поддержка со стороны рантайма**: рантайм Python много знает про то, что это корутина, хитрым образом «прикапывает» все локальные переменные — на самом деле он преобразует эту функцию под капотом в класс, у которого значения переменных лежат в полях. То есть за вас делается много магии ради синтаксического сахара, чтобы вы видели красивый код.

Как это используется в реальной жизни: во всех точках, где вы хотите что-то прочитать (обратиться к ОС, к вводу-выводу), вы свою корутину **останавливаете** и передаёте управление тому, кто готов поисполняться прямо сейчас. У вас, скорее всего, есть очередь других корутин — задач, которые уже готовы исполняться, а вы сами сейчас не готовы. Когда ваша корутина «проснётся» и станет готова, управление вернут ей. Тем самым получается существенно лучшая утилизация CPU.

Stackful vs stackless

Ключевая классификация корутин — **stackful** и **stackless** (есть ещё деление на симметричные/несимметричные, но это неинтересно, обсудим в конце, если успеем).

Stackful-корутины очень похожи на обычные потоки — это как будто вы реализовали потоки в userspace. Слово stackful значит, что для каждой корутины выделяется **стек**. Stackful-корутины также известны как **файберы** (fiber — «волокно», в отличие от thread — «нить»; «тонкий поток», потому что он целиком в userspace). Вы сами выделяете стек под каждую корутину и дальше работаете с ней как с обычным потоком.

Сколько стоит переключение контекста между файберами (если забыть про переключение в ядро)? Вы поисполняли корутину и понимаете, что пора её усыпить. Что сохранить? **Все регистры** — потому что вы могли заснуть в любом месте, и никто не знает, кто будет исполняться после вас и что ему нужно будет восстанавливать. Регистров немного: по грубой оценке, это порядка 100–150 байт, которые надо записать на стек и прочитать обратно. Это **не очень дорого** — сильно дешевле, чем переключение в ядро. Поэтому корутины можно переключать много: их создают **миллионами**, и никто не мешает создать миллион корутин и легко между ними переключаться.

Основная проблема stackful — **наличие стека**. Стек — вообще довольно дорогая конструкция: даже если вы его не весь используете, при создании файбера вы делаете не 8 мегабайт (как у потоков), а что-то сильно меньше — типично около 256 килобайт, — но память всё равно расходуется. Плюс переключение можно было бы сделать ещё дешевле: в конкретной функции реально нужных регистров может быть всего два-три (плюс адрес текущей инструкции) — итого ~24 байта вместо 150. То есть здесь всегда есть **фиксированный оверхед переключения, который мог бы быть меньше**.

Важная приятная черта: stackful-корутины можно **написать самим** — без поддержки компилятора, строк 30 на ассемблере для переключения контекста, и всё. (Домашку про это в этом году не дадут — «вас пожалели», но, может быть, дадут в следующем году; это реально можно сделать за несколько вечеров.) Забавно, что код, исполняющийся в stackful-корутине, **может вообще не знать**, что он исполняется в корутине, — как обычный поток.

Stackless-корутины — это то, что похоже на наш питоновский пример. Из названия следует, что стека нет. Как они устроены? Когда вы пишете функцию-генератор, вы заранее знаете: какие в ней есть точки возврата в родителя и **какое у неё локальное состояние**. Чтобы реализовать этот `gen` на низком уровне, достаточно завести класс с **одним полем** `num` — и всё. Компилятор (именно компилятор, не рантайм, в случае C++) может понять, какое состояние корутины нужно сохранять, и обеспечивать ровно его и только его: при остановке корутины все нужные локальные переменные складываются в сгенерированную под капотом структуру, при продолжении — восстанавливаются оттуда, и исполнение идёт дальше.

Это потенциально **эффективнее стека**: локального состояния обычно меньше, чем все регистры. Но чтобы это работало, нужен умный компилятор — нетривиальная конструкция. До C++20 stackless-корутин не было: были только странные неудобные попытки (что-то было в Boost на старых стандартах), а вот stackful люди писали много лет во множестве проектов. В C++20 появились именно **stackless-корутины**: компилятор научился разворачивать их в «структуру»-класс с минимально необходимым набором полей. Они красивые с точки зрения производительности и **безумно сложные** с точки зрения интерфейса — на них и посмотрим.

Практическое следствие: stackless-корутины — «инвазивные». Они сильно «загрязняют ваш код корутинностью» — сейчас увидим как. Stackful, наоборот, прозрачны для кода.

Три ключевых слова и почему они такие страшные

В C++20 корутина — это **любая функция, в теле которой есть хотя бы одно из трёх ключевых слов: `co_yield`, `co_return` или `co_await`**. Больше никаких синтаксических отличий нет — ну, кроме возвращаемого типа, о котором ниже.

Семантика:

- **`co_return`** — вместо обычного `return` внутри корутины используется он;
- **`co_yield`** — переключение на родителя: вернуть значение наружу, сохранив состояние и продолжившись потом;
- **`co_await`** — «усыпить текущую корутину» и вернуть управление (родителю или ещё кому-то — как получится). В отличие от `co_yield`, он сам по себе ничего не возвращает наружу. При этом `co_yield` и `co_return` **выражаются через `co_await`**: `co_yield expr` — это `co_await promise.yield_value(expr)`, а `co_return expr` — это `co_await promise.return_value(expr)`. То есть `co_await` — базовый, фундаментальный способ усыпить корутину и потом её пробудить.

А теперь о дизайне. Вместо нормального ключевого слова `yield` мы имеем уродливый префикс `co_` + подчёркивание — «максимально ужасно», настолько, что даже не хочется использовать корутины из C++20. Почему так? Потому что C++20 — это, ну, пять лет назад как вышло, по всему миру написаны миллиарды строк кода, и в них **заведомо есть много переменных с именем `yield`**. А C++ заявляет (на бумаге) обратную совместимость: при обновлении до C++20 вы не хотите, чтобы работающий код сломался от нового ключевого слова. Антон Полухин (автор фреймворка в Яндексе, поверх которого работают такси и значительный кусок Яндекса) долго защищал в комитете позицию «слово всё равно может быть занято, всё это не работает, давайте просто введём `yield` и починим код» — и даже нашли пару проектов с переменной `yield`, но аргумент «не зашёл»: видимо, те проекты решили, что неважные. Так появились `co_yield`, `co_await` и `co_return`. Придётся страдать: добавлять ключевые слова в C++ сложно, поэтому слово `static`, например, значит пять разных вещей в разных контекстах.

Практическое правило: не делайте `#define yield co_yield` — сломается `std::this_thread::yield()` и подобное. Это ребячество; надо жить и привыкать.

Идеал лекции — написать тот самый генератор, который в начале был на слайде:

```
nonstd::generator<int> numbers() {
    int num = 0;
    while (true) {
        co_yield num++;
    }
}

for (int num : numbers())
    printf("%d\n", num);
```

Можно потратить две лекции, чтобы написать то, что в начале выглядело в одну строчку. Зато красиво. И логика внутри ровно та же, что в питоновском примере: локальная переменная, постоянные `co_yield`, которые как-то возвращают значение наружу.

Устройство корутины в C++20: три объекта

Чем корутина синтаксически отличается от остальных функций? Только наличием ко-ключевых слов и — **возвращаемым типом**. Именно возвращаемый тип делает функцию корутиной.

Корутина определяется возвращаемым объектом. Важно: возвращаемый объект — это **не вся корутина**. Чтобы функция считалась корутиной, возвращаемый объект должен удовлетворять безумному количеству требований — в `srpreference` про это огромная статья со множеством точек расширения и настройки. Всё настраивается через этот объект.

Компилятор за вас очень много чего генерирует и много методов у этих объектов вызывает — примерно как `range-based for`, где красивый код раскрывается в некрасивый. В корутине фигурируют **три объекта**:

1. **Return object** — тот объект, который формально возвращается из функции; через него производится вся настройка корутины. Это обычный тип — ограничений на него почти нет, кроме одного: внутри него

должен быть объявлен **публичный alias** `promise_type`. Да, строго с именем `promise_type`, даже если у вас другой style guide.

2. **Promise type** (точнее, объект `promise`) — это **состояние корутины**. Его пишете вы: это класс с кучей обязательных методов, которые компилятор дёргает в правильном порядке. `Promise` инициализируется аргументами корутины, и из него получается `return object`.
3. **`std::coroutine_handle`** — стандартный объект, «ручка» (джойстик), за которую можно управлять корутиной. Он параметризуется типом `promise`. Умеет: `resume()` — продолжить корутину, `destroy()` — разрушить её, `done()` — проверить, завершилась ли она. Он очень базовый.

Связи между ними: `promise` определяется из `return object` (через alias `promise_type`), но сам `return object` получается **из `promise`** вызовом метода `get_return_object()` на `promise`. Почему два объекта, а не один, станет понятнее дальше; пока надо просто собрать картинку в голове и запомнить: есть `return object` и `promise`, которые пишете вы, и стандартный `coroutine handle`.

Всё живёт в заголовочном файле `<coroutine>`.

Порядок работы компилятора при старте корутины: создаются эти объекты (создаётся именно `promise`, а не «генератор»), аргументы корутины передаются в `promise`, из `promise` через `get_return_object()` получается возвращаемое значение — и оно возвращается вызывающему **ещё до исполнения тела** (если обещано, что корутина стартует остановленной — см. `initial_suspend` ниже).

Пишем минимальную корутину с нуля

Начнём писать. Цель — функция, возвращающая свой тип `suspendable`:

```
#include <coroutine>

struct suspendable {
    // ...
};

suspendable greet() {
    std::cout << "Hello, ";
    co_await std::suspend_always{};
    std::cout << "world!" << std::endl;
}
```

(`co_await` здесь — просто чтобы функция формально стала корутиной; IDE радостно подсвечивает «Unknown type name „suspendable“» и «Use of undeclared identifier „co_await“», пока мы не добавим `#include <coroutine>` и определение типа.)

Что нужно от `promise`? Он должен инициализироваться аргументами корутины (здесь их нет) и уметь возвращать `return object`. Начинаем:

```
struct suspendable {
    struct promise_type {
        promise_type() {}

        suspendable get_return_object() {
            auto handle = std::coroutine_handle<promise_type>::from_promise(*this);
            return suspendable{handle};
        }

        std::suspend_never initial_suspend() { return {}; }
        std::suspend_always final_suspend() noexcept { return {}; }
        void unhandled_exception() { std::terminate(); }
    };

    std::coroutine_handle<promise_type> handle;
};
```

Компилятор по мере написания подсказывает, чего не хватает:

- `initial_suspend` — метод, определяющий, **каким образом корутина будет запущена**: либо она сразу стартует остановленной (ни строчки вашего кода не исполнилось, её можно куда-то передать и начать исполнять там), либо сразу начинает исполняться до первого места, где засыпает. В стандарте есть два встроенных механизма: `std::suspend_always` — корутина сразу заснёт при старте, и `std::suspend_never` — не заснёт и будет исполняться.
- `final_suspend` — по аналогии: вызывается **в конце** корутины и позволяет в самом конце зачем-то заснуть. Мы весь код исполнили, заснули — и потом, когда нас кто-то разбудит, завершаемся совсем. Он исполняется после `co_return`, в самом хвосте.
- `unhandled_exception` — что делать, если из корутины вылетело необработанное исключение; пока «умрём молча» — `std::terminate()`.

И — сюрприз — требуется ещё метод `resume` у самого `suspendable`. Пишем пустой:

```
void resume() {}
```

Компилируется. Что выведет программа?

```
int main() {
    auto coro = greet();
    coro.resume();
}
```

Ничего! Точнее, почти: она напечатает `Hello`, и заснёт на `co_await`, а пустой `resume` ничего не делает. Как сделать настоящий `resume`? Из `main` у нас нет никакого контекста корутины. Но мы на C++ — будем танцевать от того, что есть: `suspendable` создаётся внутри `get_return_object` у `promise`, и **в этом месте у нас есть доступ к корутине**. Поэтому кладем в `return object` хендл:

```
suspendable get_return_object() {
    auto handle = std::coroutine_handle<promise_type>::from_promise(*this);
    return suspendable{handle};
}
```

`std::coroutine_handle<promise_type>::from_promise(*this)` — ключевая связка, которой все объекты провязываются между собой. На `return object` ограничений почти нет (должен определять `promise_type` — и всё), это обычная структура; а вот на `promise` накладывается куча требований. Дальше `resume` делается через хендл:

```
void resume() {
    if (handle.done()) {
        handle.destroy();
        return;
    }
    handle.resume();
}
```

Теперь работает: `Hello`, печатается сразу (при `initial_suspend = suspend_never`), потом корутина засыпает на `co_await std::suspend_always{}`, а `coro.resume()` будит её, и печатается `world!`.

Полезный лайфхак от лектора: одна из ключевых методик написания такого кода — **автокомплит**. Смотрите, какие методы есть у типа: у хендла это `resume`, `destroy`, `done` — и всё, других нет.

Грабли с `done` и `final_suspend`

Пробуем крутить корутину в цикле:

```
int main() {
    suspendable coro = greet();
    std::cout << "coroutine ";
}
```

```

while (coro.resume()) {}
}

```

И — падение на `handle.resume()`. Почему? Оказывается, `done()` возвращает `true` только тогда, когда корутина достигла `final_suspend`. Если `final_suspend` возвращает `suspend_never`, то корутина, исполнив весь код, **завершается совсем**: локальный объект `promise` разрушается, корутина сама себя разрушает, и хендл «протухает» — на нём нельзя вызывать даже методы проверки состояния вроде `done()`. Хендл в `return object` живёт как обычное поле структуры, и после самоуничтожения корутины он указывает в никуда.

Вывод: если вы хотите снаружи понимать, завершилась корутина или нет, — `final_suspend` должен возвращать `suspend_always`. Тогда корутина, исполнив весь код, засыпает «на выходе», остаётся живой, `done()` честно возвращает `true`, и вы сами решаете, что делать дальше: `destroy()` её или больше не трогать. `destroy()` точно можно использовать, когда хочется разрушить корутину **раньше** её естественного завершения; корректно ли звать `destroy()` после `final_suspend` — лектор честно говорит, что без `srpreference` не готов ответить, «я бы проверил по референсу».

Генератор: добавляем `co_yield`

Теперь из этого можно сделать генератор. Переименуем тип в `generator`, сделаем его шаблонным, а в `promise` добавим поле под значение:

```

template <typename T>
struct generator {
    struct promise_type {
        T value{};

        promise_type() {}

        generator get_return_object() {
            auto handle = std::coroutine_handle<promise_type>::from_promise(*this);
            return generator{handle};
        }

        std::suspend_always initial_suspend() { return {}; }
        std::suspend_always final_suspend() noexcept { return {}; }
        void unhandled_exception() { std::terminate(); }

        std::suspend_always yield_value(T v) {
            value = v;
            return {};
        }
    };

    std::optional<T> resume() {
        if (handle.done()) {
            handle.destroy();
            return std::nullopt;
        }
        handle.resume();
        return handle.promise().value;
    }

    std::coroutine_handle<promise_type> handle;
};

```

Пояснения по ходу кодирования:

- Пишем в корутине `co_yield` — компилятор говорит, что у `promise` **не хватает метода** `yield_value`. Именно его мы и добавили: он сохраняет значение в `promise` (то самое «поле `value`» сгенерированной

структуры) и возвращает `awaiter`, определяющий, засыпать ли дальше. Возвращать `void` из него не надо — нужен `awaitable`, здесь снова `std::suspend_always`.

- Хендл корутины позволяет добраться до её `promise`: `handle.promise().value` — так потребитель вытягивает последнее `co_yield`-нутое значение.
- `resume()` у генератора возвращает `std::optional<T> : std::nullopt`, когда корутина закончилась (тогда её заодно разрушаем), иначе — очередное значение.
- Тонкость с `initial_suspend`: для генератора **проще сделать `initial_suspend = suspend_always`** — не начинать исполнять генератор, пока его не позвали. Если оставить `suspend_never`, корутина сразу исполняется до первого `co_yield`, записывает значение — и тогда «резюм приходится выворачивать наизнанку»: первый вызов `resume()` должен не продолжать корутину, а только забрать уже готовое значение, и лишь последующие — реально продолжать. Неудобно; ленивый старт (`suspend_always`) снимает проблему.

Использование:

```
generator<int> greet() {
    for (int i = 0; i < 10; ++i)
        co_yield i;
}

int main() {
    generator coro = greet();
    while (auto value = coro.resume())
        std::cout << *value << " ";
}
```

Итого мы написали генератор чисел в ~50 строк — по объёму это сравнимо с итераторной реализацией, «зато сильно более весело». Логика та же, что на слайде с `nonstd::generator` и `co_yield num++`.

`co_await` и `awaiter`: три метода

Осталась нераскрытая магия: что такое `std::suspend_always`? И самое важное — **вы можете написать свой собственный `suspend_always`**. (Чтобы никто нас не спалил, посмотрим, как он устроен в `libc++`: там это структура с `await_ready` / `await_suspend` / `await_resume` — три метода.)

Когда вы делаете `co_await` на некоторой сущности, эта сущность (она называется **awaitable**) должна обладать набором специальных методов — ровно как `promise`. Их три:

- **`await_ready()`** — возвращает `bool`: **готово ли уже событие**. Семантика `co_await`: «я хочу подождать внешнюю операцию/событие». `suspend_never`-подобное поведение (`await_ready() == true`) значит: событие, как будто, уже случилось — засыпать нет смысла, корутина **не засыпает** и продолжает исполняться дальше, как будто `co_await` её и не усыплял. `false` — событие ещё не случилось, нужно подождать.
- **`await_suspend(handle)`** — вызывается, только если `await_ready()` вернул `false`. Сюда передаётся хендл текущей корутины; здесь мы готовим корутину ко сну. Например, **хендл можно положить в thread pool** — тем самым легко сделать корутину, которая «телепортирует» вас в thread pool: вы исполняете функцию в обычном потоке, вам нужно исполнить кусок кода в пуле — делаете `co_await jump_to_thread_pool`, и в `await_suspend` хендл кладётся в очередь пула (туда кладётся функция, которая сделает `resume()` на нашем хендле). Обычно так и делают: не крутят резюм в цикле, как в наших учебных примерах, а продолжают исполнять корутину в thread pool'e, когда она готова.
- **`await_resume()`** — вызывается сразу после того, как корутина пробудили; здесь можно что-то подготовить/вернуть.

Пример собственного аналога `std::suspend_always`:

```
struct suspend_always {
    bool await_ready() const noexcept { return false; } // событие точно не случилось — засыпай
    void await_suspend(std::coroutine_handle<> handle) const noexcept {
        // тут можно, например, отправить handle в thread pool
    }
}
```

```
void await_resume() const noexcept {}
};
```

`std::suspend_never` отличается **ровно одной строчкой**: `await_ready()` возвращает `true` — считаем, что событие всегда уже случилось, засыпать нет смысла.

`await_ready` — это оптимизация. Можно было бы обойтись без него, но тогда каждое ожидание требовало бы честно сохранять/восстанавливать состояние корутины, а это дорого. Если `await_ready()` вернул `true`, то не вызывается ни `await_suspend`, ни пробуждение — исполнение просто продолжается. Живой пример: вы делаете `co_await sleep(until_10_50)`, а время уже 10:51 — `awaiter` говорит `await_ready() == true`, и спать не надо: зачем засыпать, если вы уже опаздываете на вторую пару? Если же вернулся `false`, то `co_await` **гарантирует**, что корутину потом кто-то разбудит (тот, кому вы передали хендл в `await_suspend`).

Также полезно знать: `co_await` может и **не остановить** исполнение — «как пойдёт», в зависимости от `awaitable`; в него можно запихнуть любой `awaitable`, то есть любые свои типы с этими тремя методами.

Кратко

- Программы не утилизируют CPU полностью, потому что ждут ввод-вывод (диск, сеть, сон, ОС); число потоков для «идеальной» утилизации подобрать нельзя.
- Корутина — способ **остановить функцию, сохранив её состояние, и продолжить позже**, не занимая поток: планировщик задач в `userspace`, кооперативная многозадачность (в отличие от вытесняющей в ядре).
- Формализм: `suspend` (сохранить состояние), `activate` (переключиться), `terminate` (разрушить), `yield` (вернуться в родителя); корутины — это «нелегальные» комбинации: `suspend`→в родителя и `suspend`→в подготовленного ребёнка.
- **Stackful**-корутины (файберы) — поток в `userspace`: у каждой свой стек, переключение = сохранение всех регистров (~100–150 байт), можно написать самому на ассемблере; можно создать миллионы, но стек есть память.
- **Stackless**-корутины: компилятор знает локальное состояние функции и сохраняет **только его** в сгенерированную структуру — дешевле, но нужна поддержка компилятора и код становится «инвазивным».
- В C++20 корутинa — любая функция с `co_yield`, `co_return` или `co_await`; `co_yield` / `co_return` выражаются через `co_await` (`promise.yield_value` / `promise.return_value`).
- Страшный префикс `co_` появился ради обратной совместимости: слово `yield` занято миллиардами строк существующего кода.
- Корутинa определяется **возвращаемым типом**: в нём обязан быть `alias promise_type`; `promise` — состояние корутины с обязательными методами (`get_return_object`, `initial_suspend`, `final_suspend`, `unhandled_exception`, `yield_value` ...), а `std::coroutine_handle::from_promise` провязывает всё вместе.
- `initial_suspend` решает, стартует ли корутинa остановленной (`suspend_always` — удобно для генераторов) или сразу исполняется (`suspend_never`).
- `done()` корректен только при `final_suspend = suspend_always`: иначе корутинa разрушает себя сама, и хендл протухает.
- `Awaitable` для `co_await` — это три метода: `await_ready()` (событие уже случилось? тогда не засыпаем), `await_suspend(handle)` (усыпить; сюда можно отдать хендл в `thread pool`), `await_resume()` (пробуждение).

Орг. моменты

Домашних «накрутинок» по этой теме, к сожалению, не будет — возможно, выкатят одну маленькую бонусную задачку, надо её «пощупать». **Stackful**-корутины (файберы) — кандидат на домашку в следующем году; как устроено переключение контекста и сохранение регистров, возможно, посмотрят на отдельной лекции. На следующей лекции — немного «новогодняя» и не очень сложная тема: тулинг, `perf`, как правильно замерять производительность и дебажить.

Лекция 13 — Fibers

Рекап: stackless-корутины и почему ими не всё решается

На прошлой лекции мы разбирали **stackless-корутины** — встроенную в язык механику C++20 с ключевыми словами `co_await` / `co_yield`. Мы выписывали простые генераторы и надеялись, что эти «простые классики» обобщаются на более широкий круг задач. Главное применение корутин — работа с вводом-выводом и внешними по отношению к программе данными, которые ОС передаёт не очень быстро: в программе нужно **эффективно спать**, и пока спишь — не блокировать поток, а выполнять на нём что-то ещё. Stackless-корутина для этого классный подход: один раз написали — и дальше внутри доступен `co_yield`, «выплёвывающий» значение из функции.

Но у stackless-корутин есть три существенные проблемы.

1. Где лежит состояние корутины. Мы никак не обсуждали, а где физически находится state корутины: это промис (тот самый «волшебный» тип, который определяет возвращаемое значение) плюс набор локальных переменных. State у корутины ненулевой, и **обычно он выделяется на кучу**. Можно управлять этим: для конкретной корутины написать перегрузку `operator new` и указать, где и как выделять память. Но реально ~99% корутин выделяются на кучу.

2. Сложный интерфейс для автора библиотеки. Когда вы пишете сами корутины, надо очень много помнить: слишком много движущихся частей и взаимодействующих сущностей. Лектор признаётся, что читает эту лекцию не первый год, но это то место, которое без повторения невозможно воспроизвести. К этой сложности можно привыкнуть, и она почти вся оправдана — но это действительно боль.

3. Раскраска функций — главная проблема. Представьте: корутина выполняет кусок кода и хочет заснуть — например, читает диск асинхронно и ждёт, пока чтение произойдёт, чтобы проснуться и вернуть результат. Но когда вы делаете `co_await`, родительская функция обязана вас дождаться: `co_await` возвращает управление родителю, значит родитель должен *знать*, что вы корутина, и тоже уметь засыпать. На практике современный код на корутинах выглядит так, что вы делаете `co_await` на саму вызываемую функцию — и постепенно **вся программа покрывается `co_await`**.

Функции делятся на два типа: обычные («синие») и корутины («красные»). Программа перекрашивается в один из двух цветов, и **цвета между собой плохо совместимы**: синяя функция читает файл «по-человечески» — блокирует поток; для корутины блокировать поток — супер вредно. Из-за этого:

- нельзя взять и принести корутины в существующий большой проект — придётся либо очень изолированно оборачивать их в отдельный модуль (неудобно, профит небольшой), либо переписывать всю программу под корутины (больно);

Пусть обычные функции — синие, корутины — красные; это и есть «раскраска функций» (функциональная раскраска, red/blue functions).

Альтернатива: stackful-корутины = файберы

Раз было *stack-less* coroutine, существует и *stack-full* coroutine. И stackful-корутины, в отличие от stackless, **делаются тривиально: не нужно никакой поддержки со стороны компилятора**. Лектор показывает свой маленький coroutine runtime — **nanofibers**: по `cloc` это 721 строка всего, из которых ~542 содержательные: 58 строк ассемблера (`context.S`), 339 строк заголовков, 262 строки C++. Собирается одной командой:

```
clang++ *.cpp *.S -std=c++20 -O2 -ggdb -o nanofibers -I. -stdlib=libc++
```

Состав проекта: `context.S`, `context.h`, `context.x86`, `fiber.h`, `fiber.cpp`, `intrusive_list.h`, `mutex.h`, `scheduler.h`, `stack.h`, `nanofibers.cpp` (демо).

Терминология

В литературе вы будете встречать похожие слова для похожих сущностей: **корутина, файбер, сопрограмма, goroutine (го-рутина), go** и т.п. — абстрактно все они означают «функции, которые умеют засыпать». Когда говорят о конкретном рантайме, обычно разделяют: **файберы — это stackful-корутины**,

а когда в контексте современных плюсов говорят «корутины» — имеют в виду `stackless`. Небольшая путаница, к которой стоит привыкнуть.

Пример программы на фиберах

Все сущности живут в namespace `nanofibers`. Ключевой класс — `Scheduler`, который исполняет переданные ему корутины:

```
int main() {
    nanofibers::Mutex mtx;
    nanofibers::Scheduler sched;

    sched.Spawn([&mtx]() -> void {
        while (true) {
            nanofibers::Yield();
            std::scoped_lock lock{mtx};
            std::cout << "Hello from fiber 1, thread id: "
                      << std::this_thread::get_id() << std::endl;
            nanofibers::Yield();
            std::cout << "Hello from fiber 1, thread id: "
                      << std::this_thread::get_id() << std::endl;
            nanofibers::Yield();
            std::cout << "Hello from fiber 1, thread id: "
                      << std::this_thread::get_id() << std::endl;
            nanofibers::Yield();
        }
    });

    sched.Spawn([&mtx]() -> void {
        while (true) {
            nanofibers::Yield();
            std::scoped_lock lock{mtx};
            // ... аналогично: три печати "Hello from fiber 2", между ними Yield()
        }
    });

    sched.Run();
}
```

Метод `Spawn` принимает функцию, которая будет исполняться в отдельном фибере (в кадре — лямбда, но сюда можно передать более-менее всё что угодно: обычная функция, ничем не примечательная). Внутри фиберов — `nanofibers::Yield()` (переключиться на другой файбер), блокировка мьютекса, три печати в поток, повторение итерации. В конце `sched.Run()` — запускает шедулер и исполняет, пока есть что исполнять.

Вывод программы: три раза `Hello from fiber 1`, потом три раза `Hello from fiber 2` — по три раза каждый, и **везде поток один**.

На что это похоже? **На обычные потоки**. Единственное отличие — класс `Scheduler`: если переименовать его в `ThreadPool`, никто ничего не заметит. Записываем таски, делаем `run` — он их исполняет; вместо `yield` — `std::this_thread::yield()`, вместо файберного мьютекса — `std::mutex`. И действительно: **файберы (stackful-корутины) — это реализация потоков в userspace**. Именно поэтому слово «файбер»: поток по-английски `thread` (нить), а `fiber` — волокно. Это **легковесный поток**.

Что это даёт:

- штука полноценно работает в `userspace`: код, который написал сам, практически не взаимодействует с ОС — и это хорошо;
- можно самому выбирать, в каком порядке исполнять файберы, троттлить слишком прожорливые, писать сложные политики шедулинга;
- **при каждом переключении файберов не нужно переключаться в ядро** — а переключение в ядро дорогая штука.

Чего мы лишаемся: вытесняющая многозадачность

Как переключаются обычные потоки? На одноядерной системе с десятью потоками работает `completely fair scheduler`, который честно раздаёт процессорное время. Когда поток своё время потратил, ядро его останавливает. Останавливает не через `signal stop` (это юзерспейсный способ), а **прерыванием (`interrupt`)**: прерывание вырабатывается всегда в контексте ядра, и уже в его обработчике ядро может усыпить поток и пробудить другой. В произвольном месте пользовательской программы «просто так» переключиться в ядро нельзя — вы складываете числа в цикле, никаких системных вызовов нет; поэтому ядро *прерывает* поток.

При использовании обычных потоков это называется **вытесняющая многозадачность (`preemptive multitasking`)**: ядро само решает, когда вас снять с исполнения, а вы не особо можете на это влиять.

Файберы и корутины — это **кооперативная многозадачность (`cooperative multitasking`)**: задачи должны дружить друг с другом и *сами* разрешать друг другу поисполняться. Следствия:

- Если запустить файбер с бесконечным `while (true)` без переключений — скорее всего, весь корутинный рантайм сломается (никто, кроме этого файбера, не получит управление).
- Есть продвинутые рантаймы вроде Go, которые пытаются делать а-ля вытесняющую многозадачность, но на практике это не очень распространено.
- Теоретически прерывать файберы можно и сигналами: посылаете любой сигнал со своим обработчиком и хитрым образом достаёте из обработчика контекст потока (можно подпатчить, куда вы вернётесь после выхода из обработчика сигнала). Это реально делается — но в общем случае останавливать произвольные потоки в `userspace` сильно сложнее.

Кстати, переключение файберов очень похоже на то, что делает линуксовый планировщик, только происходит внутри юзерспейса и чуть легче (не надо в ядро). Для ядра вся программа — один поток. Многопоточный файберовый рантайм сделать можно (все так и делают — содержательные части файберов исполняются в обычном тредпуле), но формально C++ такое запрещает — об этом в конце лекции.

Устройство рантайма: Scheduler

Ключевой класс, через который происходит всё взаимодействие с рантаймом (кадр из `scheduler.h`):

```
#pragma once
#include "context.h"
#include "fiber.h"
#include "intrusive_list.h"
#include <cassert>

namespace nanofibers {

class Scheduler {
public:
    void Spawn(std::function<void()> routine) {
        Fiber* fiber = new Fiber{std::move(routine)};
        Schedule(fiber);
    }

    // Put fiber to the queue
    void Schedule(Fiber* fiber) {
        fiber->SetState(EFiberState::Runnable);
        queue_.PushBack(fiber);
    }

    // Pause current fiber and switch to scheduler
    void Yield() {
        Switch(from: current_fiber_->GetContext(), to: &main_context_);
    }
    // ...
};
```

Что здесь происходит:

- `Spawn` создаёт объект `Fiber` (ещё один ключевой класс) и передаёт ему функцию; файбер — наследник `IntrusiveListItem<Fiber>`.
- `Schedule` помечает файбер как **Runnable** (у файбера есть перечисление состояний) и кладёт его в очередь. Очередь — обычный **интрузивный список** (можно было бы `std::queue`, но интрузивный список удобнее).
- `Run` — супертривиальный цикл: пока есть что исполнять, берём первый файбер из очереди (на этом месте можно написать любую нетривиальную логику выбора), переключаемся на него, выходим из него, повторяем.

`yield` здесь — свободная функция, и это «чит» ради удобства: по смыслу она говорит шедулеру «усыпь прямо сейчас этот файбер и разбуди когда-нибудь потом» — буквально кладёт себя в конец той самой очереди исполнения. Если бы не свободная функция, пришлось бы везде протаскивать ссылку на шедулер. Реализация: `Scheduler::GetCurrentScheduler()` возвращает **текущий шедулер, привязанный к потоку** — на старте `Run` локальная переменная «текущий шедулер» запоминается в `thread_local`. Это глобальная переменная по сути, уникальная для каждого потока (потоково-глобальная внутри потока). Аналогично есть `current_fiber` — знание шедулера о том, какой файбер сейчас исполняется.

Переключение контекста: `SaveContext` и `JumpContext`

Вся магия рантайма — в механизме переключения контекста. Демонстрация (функция `Ctx` из `nanofibers.cpp`):

```
void Ctx() {
    Context ctx;
    if (SaveContext(&ctx) == ESaveContextResult::Resumed) {
        std::cout << "Hehe" << std::endl;
    } else {
        std::cout << "Not hehe" << std::endl;
    }
    JumpContext(&ctx);
}

int main() {
    Ctx();
    return 0;
}
```

Что выведет эта программа? Сначала `Not hehe`, а потом бесконечно `Hehe`. Это очень похоже на `goto`. Через всё работает **две функции**: `SaveContext` и `JumpContext` (можно сделать и одной, но двумя нагляднее). И появляется сущность **контекст** — буквально контекст исполнения.

Что лежит в контексте

Набор регистров. На ARM нужно сохранить ~12 регистров, чтобы запомнить состояние исполнения; из них два ключевых — **instruction pointer (rip)** и **stack pointer (rsp)**, остальные — general purpose регистры, которые важно не забыть. Всего контекст — порядка 100 байт.

`SaveContext`

Функция «магическая»: на ней висит attribute, которым мы говорим компилятору, что из этой функции можно выйти два раза. Реализация (x86, по кадрам/словам лектора):

- сохраняем все регистры, которые мы должны сохранить со своей стороны;
- **адрес возврата запоминаем забавным образом**: при вызове `SaveContext` на стеке первые 8 байт — это сохранённый адрес возврата, то, куда мы прыгнем после выхода из функции. Поэтому просто *разыменовываем* `rsp` — ровно в этом месте лежит адрес возврата — и сохраняем это значение в первое поле контекста (поле `rip`);
- дальше сохраняем `rsp` во второе поле, затем остальные регистры;
- возвращаем `0`, то есть `ESaveContextResult::Saved`.

Казалось бы, функция всегда возвращает 0 — зачем enum? А вот зачем: после `SaveContext` в контексте лежат `rsp` на момент вызова, остальные регистры, а `rip` указывает на следующую строчку после вызова. Мы вернули 0 → печатаем `Not hehe` → идём дальше → зовём `JumpContext`.

JumpContext

Происходит всё то же самое, только в обратную сторону:

- восстанавливаем `rsp` и все сохранённые регистры;
- **перезаписываем адрес возврата** — это внезапно можно сделать: берём и пишем сохранённый адрес возврата туда, где он должен лежать. По-хорошему адрес возврата `JumpContext` — это выход из `Ctx`; но на функции висит `[[noreturn]]` (говорим компилятору, что функция не возвращает, чтобы он нормально оптимизировал и не сходил с ума);
- в регистр `rax` кладем 1 — это `ESaveContextResult::Resumed`;
- делаем `ret`.

Как устроена инструкция `ret`? Она берёт сохранённые на стеке первые 8 байт (адрес возврата) и делает на них `jump`. Мы только что перезаписали адрес возврата на тот, который был у `SaveContext` — поэтому `ret` выходит «не из нашей функции, а как будто из `SaveContext`», и прыгает в ту ветку `if`, которая проверяет результат: там лежит 1 → `Resumed` → печатаем `Hehe` → снова зовём `JumpContext` → бесконечный цикл печатает `Hehe`.

Вся магия — в этих двух строчках: **перезапись адреса возврата**.

А это вообще валидный C++?

Вопрос с подвохом: корректен ли такой код с точки зрения C++, нет ли тут `undefined behavior`? Ответ: мы пишем на ассемблере — правила C++ к левой части (сам ассемблер) неприменимы. А к правой (плюсовая обёртка) применимы, и вопрос, разрешает ли C++ такие функции, — **открытый**: он их формально не запрещает, но и не разрешает. К счастью, это уже не запретить — слишком много кода написано.

Switch: как прыгать между файберами

Когда есть `SaveContext` / `JumpContext`, как сделать файберы? Переключения бывают двух видов: **из шедулера в файбер** и **из файбера обратно в шедулера**. Файберы здесь асимметричны: они всё время возвращаются в шедулера. Есть основной контекст потока (`main_context_` — контекст шедулера) и контексты разных файберов; файбер поисполнялся — вернулся в шедулера, шедулера выбирает следующий.

Для удобства есть функция `Switch` (абстракция над `SaveContext` + `JumpContext`):

```
// Псевдокод по словам лектора:
void Switch(Context& from, Context& to) {
    if (SaveContext(&from) == ESaveContextResult::Saved) {
        JumpContext(&to);
    }
}
```

Функции даём два контекста: текущий, который надо сохранить (`from`), и тот, куда хотим прыгнуть (`to`). Сохраняем себя во `from` и прыгаем на `to` — подразумевая, что если кто-то потом прыгнет на наш сохранённый контекст, всё будет хорошо.

В `Run`: взяли файбер из очереди → `Switch(from: main_context_, to: fiber->GetContext())` — запомнили контекст шедулера, прыгнули в файбер. Файбер поисполнялся, понял, что пора спать (например, позвал `yield`) → `Switch(from: current_fiber->GetContext(), to: &main_context_)` — вернулись в шедулера.

Почему первый прыжок в файбер вообще работает? Ведь файбер ещё ни разу не запускался — на что прыгаем? Ответ — в конструкторе файбера (кадр из `fiber.h`):

```
class Fiber : public IntrusiveListItem<Fiber> {
    static constexpr size_t kDefaultStackSize = 4 * 4096; // 16 КБ

public:
    explicit Fiber(std::function<void()> routine)
```

```

        : stack_{kDefaultStackSize}
        , routine_{std::move(routine)}
    {
        context_.sp = stack_.Top();
        context_.ip = reinterpret_cast<void*>(Trampoline);
    }

    EFiberState GetState() const {
        return state_;
    }
    // ...
};

```

Делается «прям супер красиво»:

1. выделяем стек — **16 килобайт** (4 страницы по 4096 байт; «нормальный стек, наверное, всем хватит»); структура `Stack` просто делает `operator new` и выделяет достаточно памяти;
2. сохраняем `std::function`, которую хотим исполнять;
3. **руками заполняем регистры**, которые нам интересны: `sp` — вершина только что выделенного стека, `ip` — адрес функции `trampoline`.

Трамплин — это функция, которая исполняется первым делом при старте фибера: мы всегда прыгаем на неё «в пустом» контексте фибера. Она берёт (через `GetCurrentFiber` — глобальное/тредлокальное состояние) текущий шедюлер, из него — текущий фибер и запускает его рутину. Таким образом контекст заполнен так, что первое переключение на него «оживляет» фибер.

Почему нельзя прыгнуть сразу на пользовательскую функцию

Шикарный вопрос с лекции: зачем трамплин? Почему не записать в instruction pointer сразу адрес той функции, которую хотим исполнить?

- **`std::function` — это объект (класс), а не код.** У нас фон-неймановская архитектура: в одной памяти лежит и код, и данные, поэтому указатели бывают и на код, и на данные. Если взять адрес функции — по этому адресу лежат буквально закодированные инструкции для процессора, и прыжок туда осмыслен. Но адрес `std::function` — это адрес данных: обычный плюсовый класс с полями, никак не связанными с исполняемым кодом. Сказать процессору «исполни данные этого класса» нельзя.
- **Лямбда с захватами.** Лямбда под капотом — класс с `operator()`. Если у лямбды есть захваты, в объекте появляется полюшка, где лежат захваченные переменные (например, указатель на рутину). Если взять адрес `operator()` и прыгнуть туда — **не будет инстанса класса**: не передастся информация о том, где лежит эта полюшка. Оператор — свойство всего класса, а у объекта есть ещё данные.
- В общем: мы пытаемся запихнуть **больше данных, чем влезает в один указатель** — кроме самого кода ещё и аргументы/контекст. Естественно, не получается.

Поэтому трамплин — какая-то глобальная функция, которая ничего не принимает и не возвращает, — нужен в любом случае. А данные протаскиваются **через глобальное состояние — `thread_local`**:

`GetCurrentFiber()` читает тредлокальную переменную, из неё берём шедюлер и текущий фибер, и уже его рутину исполняем. (Кстати, почему `thread_local`-переменная объявляется `inline`: чтобы не выносить определение в .cpp файл — `inline` значит «инициализируем сразу по месту». И `thread_local` бывают только глобальные/статические переменные — «редлокальных полей классов не бывает».)

Альтернативно можно было бы подпатчить регистры так, чтобы трамплин на старте получал аргументом ссылку на фибер — но проще через тредлокалку.

Fiber-примитивы синхронизации: `mutex` и `condition variable`

На фиберах написано очень много всего — больше, чем на корутинах, потому что фиберы можно было использовать и до C++20. В рантайме есть фиберный `Mutex` и — «очень смешной, очень красивый» — фиберный `ConditionVariable`.

Fiber mutex (без него рантайм ~400 строк, с ним ~500): у него внутри — **очередь фиберов**. `Lock`: если мьютекс занят, текущий фибер усыпляется и кладётся в очередь этого мьютекса (не в очередь шедюлера). `Unlock`: достаём фибер из очереди и будим. Поскольку рантайм в примерах однопоточный,

однопоточный мьютекс сделать легко — атомарность почти не нужна. А вот **хороший многопоточный файберный мьютекс — очень нетривиальная вещь**: по факту надо написать и обычный мьютекс, и чтобы он знал про файберы.

Fiber condition variable: полюшка — опять же очередь из файберов; методы `Wait`, `NotifyOne`, `NotifyAll`. `Wait`: «вы файбером ждёте какого-то события» — говорите шедулера «не надо меня будить, меня кто-то другой пробудит» и кладёте себя **в очередь кондвара, а не шедулера**. `NotifyAll` просто удлиняет очередь шедулера содержимым очереди кондвара; `NotifyOne` берёт один элемент и перекладывает его на исполнение. Ровно то, как кондвар ведёт себя в ОС, только помещается на экран. (В ядре, скорее всего, устроено хитрее — там удобнее иногда дренировать очередь независимо от того, было событие или нет.)

Почему в ОС бывают **spurious wakeup’ы**? Честный ответ лектора: хорошего источника он не нашёл; в мейлинг-листах 15-летней давности коллеги говорят «так было проще код написать». Текущая версия: это просто **дополнительное облегчение гарантий для ядра** — операционная система и так предоставляет очень много гарантий пользовательскому коду, и чем меньше фиксировать того, что не обязательно фиксировать, тем больше шансов написать более эффективный код. Spurious wakeups ничего не меняют для корректного кода, зато потенциально позволяют ядру быть эффективнее.

Опасность: обычные примитивы синхронизации в файберах

Что будет, если залочить обычный `std::mutex` в файберном рантайме? Если разлочить до переключения — всё шикарно. А если залочить, переключиться на другой файбер, и тот тоже попытается залочить — **дедлок**. Поэтому:

- **Использовать стандартные примитивы синхронизации в файберных рантаймах надо очень аккуратно: чётко понимать, где какой файбер блокируется, и гарантировать, что между lock и unlock мьютекса не произойдёт переключения контекста.**
- Проблема неизбежна, потому что вы используете чужой код, который не знает про файберы: например, аллокатор или стандартная библиотека. Обычные мьютексы в файберных рантаймах применяются, но **очень изолированно**: что-то сделали под мьютексом — сразу отпустили, без шансов на переключение внутри.

Stackful vs stackless: цена и практика

Сравнение двух подходов:

- **Stackful чуть удобнее**: вы пишете более-менее обычный код, только не забываете использовать правильные (файберные) примитивы, когда они возникают.
- **Stackful чуть дороже**: во-первых, надо выделять стеки, причём достаточно большого размера, чтобы можно было исполнять произвольный код. Во-вторых, нужно сохранять **все регистры** — контекст порядка 100 байт, тогда как у большинства C++20-корутин стоит, скорее всего, меньше 100 байт. Сохранение регистров эффективно по простоте кода, но прожорливо по производительности.
- **Если пишете совсем хайлоад**, где надо выжать каждую микросекунду — скорее всего, осмыслены stackless-корутины. Если пишете много кода, который должен быть удобен в написании, чтении, использовании и дебаге — **используйте stackful, это сильно понятнее**.

Размер стека — отдельная боль. У файберов 16 КБ, у обычных потоков — ~8 МБ (туда помещается много кода, и это многое упрощает). Это странное ограничение, на котором весь мир работает «на честном слове»: **никто не проверяет, сколько стека реально использует программа** — ни браузер, ни что-либо ещё; это трудно проверить, и это никто нормально не трекает. Если стек переполнится — программа падает, и всем нормально. С файберами то же самое, только стек мельче: сделали где-нибудь большой массив на стеке (даже не очень большой) — всё лопнуло.

Продакшен-практика: на файберах написано очень много всего, потому что их можно было использовать до C++20. Есть классный опенсорсный файберовый движок **userver**, на котором работает значительная часть Яндекса; похожих фреймворков внутри используется несколько, и в опенсорсе их довольно много — вы с ними столкнётесь. Прямо сейчас весь поиск ведётся на файберах с ожиданием профита относительно тредовой модели, и много больших сервисов работают на файберах исключительно. По проверенности это, наверное, одна из самых надёжных механик; с C++20-корутинами всё сложнее и непонятнее.

Главные грабли: `thread_local` и миграция файберов между потоками

Формально в C++ **запрещено** делать файбер, который начал исполняться на одном потоке, потом уснул и был пробуждён на другом. **На практике все так делают и надеются, что не сломается. Оно ломается.** Сценарий: многопоточный шедюлер, 5 потоков, каждый пытается исполнить любой готовый файбер — файберы мигрируют между потоками. Логично завести **один большой тредпул, в который напиханы файберы, и жить внутри него**. Можно биндить каждый файбер к конкретному потоку — но вы теряете перф: один поток исполняет что-то длинное, а остальные стоят, хотя могли бы исполнять чужие файберы.

В чём именно проблема

Не в синхронизации очередей (взятие файбера под мьютексом — это аккуратно пишется). Основная проблема — **thread-локальные переменные**. Компиляторы делают очень забавную вещь: если внутри одной функции вы прочитали тредлокальную переменную, сделали несколько `yield` (переключились между файберами и вернулись), а потом прочитали её ещё раз — компилятор видит, что это всё внутри одной функции, и предполагает, что **каждая функция исполняется в одном потоке** (так написано в стандарте!). Поэтому он кэширует первое чтение и, например, выкидывает проверку как «всегда true».

Пример из лекции (файбер исполняется, читает «номер своего потока»):

```
// thread_local int threadIndex; — на старте каждого потока в неё
// запоминается номер потока: 0, 1, 2, 3, ...

void FiberBody() {
    int copy = threadIndex;      // прочитали: исполняемся на потоке 0
    // ... полезная работа ...
    nanofibers::Yield();         // уснули; шедюлер мог разбудить
    nanofibers::Yield();         // нас на ДРУГОМ потоке
    nanofibers::Yield();

    // здесь threadIndex уже может быть, например, 3,
    // а в copy лежит сохранённый 0
    if (copy == threadIndex) {   // компилятор может оптимизировать
        // ...                  // эту проверку в "всегда true"!
    }
}
```

У вас 4 потока — значит, в программе 4 экземпляра тредлокалки, у каждой свой адрес и своё значение. Файбер начал исполняться на потоке 0 (`copy == 0`), после трёх `yield`-ов может продолжиться на потоке 3 (`threadIndex == 3`) — но компилятор про это не знает и оптимизирует в предположении «всё в одном потоке». Важно: **локальные (обычные) переменные файбера при этом живут нормально** — они лежат на стеке файбера, и в этом вся прелесть файбера. Проблема только в тредлокалках — они являются свойством потока, а не файбера.

«Корутина» здесь выступает как обычная функция, и компилятор считает, что исполнение одной функции не мигрирует между потоками.

Почему это не чинят

- **Стандарт:** там написано, что функция исполняется в одном потоке. То, что делают файберные рантаймы, формально запрещено.
- **Компании:** раз в год представители крупных компаний приходят на форум LLVM: «у нас есть файберовый движок, на нём всё работает, пожалуйста, не ломайте нам код». Ответ коллег из LLVM: «в стандарте C++ написано, что вы делаете запрещённое, поэтому мы будем вам ломать код». И уходит ещё на год. **Политика.**
- **Конкуренция компиляторов:** если предположить, что тредлокалки нельзя кэшировать, компиляторы будут генерировать менее эффективный код для программ, которые не используют файберы (пришлось бы перечитывать тредлокалку каждый раз). Ни один компилятор в одиночку не хочет на это соглашаться — GCC и Clang конкурируют и боятся проиграть в бенчмарках. Даже флаг сделать никто не торопится («флаг вам в руки — сделайте, все будут очень благодарны»).
- **Страдают все:** важные тредлокалки в файберных движках обмазывают `volatile` и директивами «не оптимизируй чтение этой переменной» — выглядит довольно безумно. Хуже: **вы можете даже не знать**,

что где-то под капотом используемой вами библиотеки есть тредлокалка; собираете с LTO, компилятор видит всю программу, кэширует чтения — и код ломается.

- Коллеги из компиляторов не готовы это поддерживать; переписывать стандарт надо, но никто не договорился. Это нерешённая проблема современных C++.

Практические правила

- Файберы — это потоки в userspace: легковесные, с дешёвым переключением без входа в ядро, с подконтрольным вам порядком исполнения.
- Файберы — кооперативная многозадачность: любой файбер обязан «делиться» (yield, блокировки); while (true) без переключений ломает рантайм.
- В файберном коде используйте файберные примитивы синхронизации (nanofibers::Mutex, nanofibers::ConditionVariable), а не std::mutex/std::condition_variable.
- Если всё же используете обычный мьютекс — между lock и unlock не должно быть ни одного переключения файбера, иначе дедлок.
- Чужой код (аллокаторы, стандартная библиотека) про файберы не знает — обычные примитивы там допустимы только в очень изолированных коротких критических секциях.
- Не кладите на стек файбера большие объекты/массивы: стек файбера — порядка 16 КБ против 8 МБ у потока; переполнение — падение программы.
- Никто не проверяет расход стека — ни для потоков, ни для файберов; это держится «на честном слове».
- Для читаемого и удобного в дебаге кода выбирайте stackful (файберы); для предельного хайлоада — stackless-корутины.
- Не кэшируйте значения thread_local переменных в файберах: файбер может уснуть на одном потоке и проснуться на другом.
- Не рассчитывайте, что thread_local внутри файберного кода корректен: компилятор оптимизирует чтения в предположении «одна функция — один поток».
- Многопоточный файберовый рантайм формально запрещён стандартом, но все его используют; будьте готовы к тонким поломкам (LTO усугубляет).

Кратко

1. Stackless-корутины C++20 страдают от трёх вещей: стоит на куче, сложный интерфейс библиотечного автора и раскраска функций — co_await расползается по всей программе, функции делятся на два плохо совместимых цвета, внедрить корутины в готовый проект тяжело.
2. Альтернатива — stackful-корутины (файберы): потоки в userspace, не требуют поддержки компилятора; весь рантайм nanofibers — ~700 строк, из них 58 строк ассемблера.
3. Файберы = кооперативная многозадачность: переключения происходят только когда файбер сам «отдаётся» (yield, блокировка); вытеснить его, как это делает ядро через interrupt, нельзя (можно эмулировать сигналами).
4. Сердце рантайма — контекст (набор из ~12 регистров, ~100 байт: rip, rsp + general purpose) и две функции: SaveContext (сохраняет регистры, в т.ч. адрес возврата со стека, возвращает 0/Saved) и JumpContext (восстанавливает регистры, перезаписывает адрес возврата, возвращает 1/Resumed) — поэтому SaveContext «возвращается дважды».
5. Switch(from, to) = SaveContext в from + JumpContext в to; файберы асимметричны — всегда возвращаются в шедюлер, у которого есть свой main_context_.
6. Первый запуск файбера работает через trampoline: в конструкторе файбера руками заполняются context_.sp (вершина выделенного стека, 16 КБ) и context_.ip (адрес трамплина). Прыгать сразу на std::function /лямбду нельзя — это данные, а не код; захваты не влезают в один указатель, контекст протаскивается через thread_local.
7. У файберов свои примитивы синхронизации — мьютекс и condvar, внутри которых просто очереди файберов; wait кладёт файбер в очередь кондвара, notify — переключает в очередь шедюлера.
8. Обычные примитивы в файберах опасны: залоченный std::mutex + переключение файбера = дедлок; чужой код без файбер-осведомлённости — источник таких дедлоков.
9. Stackful удобнее stackless, но дороже: выделение стеков (16 КБ) + сохранение всех регистров (~100 байт); в проде файберы проверены десятилетиями (userver в Яндексе, поиск на файберах), stackless-корутины пока «сложнее и непонятнее».

10. **Главная нерешённая проблема:** файбер, уснувший на одном потоке и проснувшийся на другом, формально запрещён стандартом (функция выполняется в одном потоке), компиляторы кэшируют чтения `thread_local` и ломают код; все всё равно используют многопоточные файберы, обмазывают тредлокалки `volatile` и надеются — а LLVM и GCC договариваться не спешат.

Орг. моменты

Следующее занятие — **коллоквиум**: устный, «на колоколе», стартовали в 9:30 и шли «пока не закончится». Лектор в шутку пообещал бонус (+100 баллов) тому, кто сделает флаг компилятора, отключающий кэширование `thread_local`, и скинет ссылку на форум LLVM, где обсуждается проблема. Лекция началась с небольшой задержки (технические заминки с зарядкой ноутбука — на скриншотах виден Mission Control с зарядом 9%); в конце лектор поблагодарил слушавших и напомнил о коллоквиуме.